

Padrão de Verificação de Segurança de Aplicações

Versão 5.0.0



Maio de 2025

Contents

Frontispício	7
Sobre o Padrão	7
Direitos Autorais e Licença	7
Líderes do Projeto	7
Grupo de Trabalho	7
Outros Principais Contribuidores	7
Outros Contribuidores e Revisores	8
Prefácio	8
Introdução	8
Princípios por trás da versão 5.0	8
Olhando para o futuro	9
O que é o ASVS?	9
Escopo do ASVS	10
Aplicação	10
Segurança	10
Verificação	11
Padrão	11
Requisito	11
Decisões de segurança documentadas	11
Níveis de Verificação de Segurança de Aplicações	13
Avaliação de Nível	13
Nível 1	13
Nível 2	14
Nível 3	14
Qual nível alcançar	14
Como usar o ASVS	15
A estrutura do ASVS	15
Estratégia de Lançamento	15
Flexibilidade com o ASVS	15
Como Referenciar os Requisitos do ASVS	16
Fazendo Fork do ASVS	16
Casos de uso para o ASVS	17
Como Guia Detalhado de Arquitetura de Segurança	17
Como Referência Especializada de Codificação Segura	17
Como Guia para Testes Automatizados de Unidade e Integração	17
Para Treinamento de Desenvolvimento Seguro	18
Como Framework para Guiar a Aquisição de Software Seguro	18
Aplicando o ASVS na Prática	18

Avaliação e Certificação	18
A Postura da OWASP sobre Certificações e Selos de Confiança do ASVS	18
Como Verificar a Conformidade com o ASVS	19
Relatório de Verificação	19
Escopo da Verificação	19
Mecanismos de Verificação	19
Alterações em Relação à Versão 4.x	21
Introdução	21
Filosofia dos Requisitos	21
Escopo e Foco	21
Ênfase nos Objetivos de Segurança Sobre os Mecanismos	21
Decisões de Segurança Documentadas	22
Mudanças Estruturais e Novos Capítulos	22
Remoção de Mapeamentos Diretos para Outros Padrões	23
Redução do Acoplamento com as Diretrizes de Identidade Digital do NIST	23
Afastamento da Common Weakness Enumeration (CWE)	23
Repensando as Definições de Nível	23
Nível de Entrada Mais Fácil	23
A Falácia da Testabilidade	24
Não se Trata Apenas de Risco	24
V1 Codificação e Sanitização	24
Objetivo de Controle	24
V1.1 Arquitetura de Codificação e Sanitização	25
V1.2 Prevenção de Injeção	25
V1.3 Sanitização	27
V1.4 Memória, String e Código Não Gerenciado (Unmanaged Code)	29
V1.5 Desserialização Segura	29
Referências	30
V2 Validação e Lógica de Negócios	31
Objetivo de Controle	31
V2.1 Documentação de Validação e Lógica de Negócios	31
V2.2 Validação de Entrada	31
V2.3 Segurança da Lógica de Negócios	33
V2.4 Antiautomação	33
Referências	34
V3 Segurança de Frontend Web	34
Objetivo de Controle	34
V3.1 Documentação de Segurança de Frontend Web	34

V3.2 Interpretação Não Intencional de Conteúdo	35
V3.3 Configuração de Cookies	35
V3.4 Cabeçalhos de Mecanismos de Segurança do Navegador	36
V3.5 Separação de Origem no Navegador	38
V3.6 Integridade de Recursos Externos	40
V3.7 Outras Considerações de Segurança do Navegador	40
Referências	41
V4 API e Web Service	41
Objetivo de Controle	41
V4.1 Segurança Genérica de Web Service	41
V4.2 Validação da Estrutura da Mensagem HTTP	42
V4.3 GraphQL	43
V4.4 WebSocket	44
Referências	44
V5 Manipulação de Arquivos	45
Objetivo de Controle	45
V5.1 Documentação de Manipulação de Arquivos	45
V5.2 Upload de Arquivo e Conteúdo	45
V5.3 Armazenamento de Arquivos	46
V5.4 Download de Arquivos	47
Referências	47
V6 Autenticação	48
Objetivo de Controle	48
V6.1 Documentação de Autenticação	48
V6.2 Segurança de Senha	49
V6.3 Segurança Geral de Autenticação	50
V6.4 Ciclo de Vida e Recuperação do Fator de Autenticação	52
V6.5 Requisitos Gerais de Autenticação Multifator	52
V6.6 Mecanismos de Autenticação Fora de Banda (Out-of-Band)	54
V6.7 Mecanismos de Autenticação Criptográfica	55
V6.8 Autenticação com um Provedor de Identidade (Identity Provider)	56
Referências	57
V7 Gerenciamento de Sessão	57
Objetivo de Controle	57
V7.1 Documentação de Gerenciamento de Sessão	58
V7.2 Segurança Fundamental do Gerenciamento de Sessão	59
V7.3 Tempo Limite de Sessão (Session Timeout)	59
V7.4 Encerramento da Sessão	60

V7.5 Defesas Contra o Abuso de Sessão	61
V7.6 Reautenticação Federada	61
Referências	62
V8 Autorização	62
Objetivo de Controle	62
V8.1 Documentação de Autorização	62
V8.2 Design Geral de Autorização	63
V8.3 Autorização em Nível de Operação	64
V8.4 Outras Considerações sobre Autorização	64
Referências	65
V9 Tokens Autocontidos (Self-contained Tokens)	65
Objetivo de Controle	65
V9.1 Origem e Integridade do Token	65
V9.2 Conteúdo do Token	66
Referências	67
V10 OAuth e OIDC	67
Objetivo de Controle	67
V10.1 Segurança Genérica de OAuth e OIDC	69
V10.2 Cliente OAuth	70
V10.3 Servidor de Recursos OAuth	70
V10.4 Servidor de Autorização OAuth	71
V10.5 Cliente OIDC	74
V10.6 Provedor OpenID	74
V10.7 Gerenciamento de Consentimento	75
Referências	75
V11 Criptografia	76
Objetivo de Controle	76
V11.1 Inventário e Documentação Criptográfica	77
V11.2 Implementação Segura de Criptografia	78
V11.3 Algoritmos de Criptografia	79
V11.4 Hashing e Funções Baseadas em Hash	80
V11.5 Valores Aleatórios	80
V11.6 Criptografia de Chave Pública	81
V11.7 Criptografia de Dados Em Uso	81
Referências	82
V12 Comunicação Segura	82
Objetivo de Controle	82
V12.1 Orientação Geral de Segurança TLS	82

V12.2 Comunicação HTTPS com Serviços Voltados Para a Internet (External Facing Services)	83
V12.3 Segurança da Comunicação Geral Serviço-a-Serviço	84
Referências	84
V13 Configuração	85
Objetivo de Controle	85
V13.1 Documentação da Configuração	85
V13.2 Configuração da Comunicação Backend	86
V13.3 Gerenciamento de Segredos	88
V13.4 Vazamento de Informações Não Intencionais (Unintended Information Leakage) . .	90
Referências	95
V14 Proteção de Dados	95
Objetivo de Controle	95
V14.1 Documentação de Proteção de Dados	96
V14.2 Proteção Geral de Dados	96
V14.3 Proteção de Dados no Lado do Cliente (Client-side)	98
Referências	98
V15 Codificação e Arquitetura Seguras	99
Objetivo de Controle	99
V15.1 Documentação de Codificação e Arquitetura Seguras	99
V15.2 Arquitetura de Segurança e Dependências	101
V15.3 Codificação Defensiva	103
V15.4 Concorrência Segura	106
Referências	106
V16 Registro de Segurança e Tratamento de Erros	107
Objetivo de Controle	107
V16.1 Documentação de Registros de Segurança (Security Logging)	107
V16.2 Registro (Logging) Geral	107
V16.3 Eventos de Segurança	108
V16.4 Proteção de Logs	109
V16.5 Tratamento de Erros	110
Referências	112
V17 WebRTC	112
Objetivo de Controle	112
V17.1 Servidor TURN	113
V17.2 Mídia	114
V17.3 Sinalização	117
Referências	118

Apêndice A: Glossário	118
Apêndice B: Referências	125
Projetos Principais da OWASP (Core Projects)	125
Projeto OWASP Cheat Sheet Series	125
Projetos Relacionados à Segurança Mobile	126
Projetos Relacionados à Internet das Coisas (IoT) da OWASP	126
Projetos Serverless da OWASP	126
Outros	126
Apêndice C: Padrões de Criptografia	126
Inventário e Documentação Criptográfica	127
Forças Equivalentes dos Parâmetros Criptográficos	127
Valores Aleatórios	128
Algoritmos de Cifra	129
Modos de Cifra AES	130
Empacotamento de Chaves (Key Wrapping)	130
Criptografia Autenticada	131
Funções Hash	132
Funções Hash para Casos de Uso Gerais	132
Funções Hash para Armazenamento de Senhas	133
Funções de Derivação de Chaves (KDFs)	134
Funções de Derivação de Chaves Gerais	134
Funções de Derivação de Chaves Baseadas em Senha	134
Mecanismos de Troca de Chaves	135
Esquemas KEX	135
Grupos Diffie-Hellman	136
Códigos de Autenticação de Mensagens (Message Authentication Codes - MAC)	137
Assinaturas Digitais	137
Padrões de Criptografia Pós-Quântica (Post-Quantum Encryption Standards)	138
Apêndice D: Recomendações	138
Introdução	138
Mecanismos recomendados, dentro do escopo	138
Princípios de Segurança de Software	139
Processos de Segurança de Software	139
Apêndice E - Contribuidores	140

Frontispício

Sobre o Padrão

O Padrão de Verificação de Segurança de Aplicações (Application Security Verification Standard - ASVS) é uma lista de requisitos de segurança de aplicações que arquitetos, desenvolvedores, testadores, profissionais de segurança, fornecedores de ferramentas e consumidores podem usar para definir, construir, testar e verificar aplicações seguras.

Direitos Autorais e Licença

Versão 5.0.0, Maio de 2025



Figure 1: license

Copyright © 2008-2025 The OWASP Foundation.

Este documento foi lançado sob a Licença Internacional Creative Commons Attribution-ShareAlike 4.0.

Para qualquer reutilização ou distribuição, você deve comunicar claramente os termos de licença desta obra a terceiros.

Líderes do Projeto

Elar Lang	Josh C Grossman
Jim Manico	Daniel Cuthbert

Grupo de Trabalho

Tobias Ahnoff	Ralph Andalis	Ryan Armstrong	Gabriel Corona
Meghan Jacquot	Shanni Prutchi	Iman Sharafaldin	Eden Yardeni

Outros Principais Contribuidores

Sjoerd Langkemper	Isaac Lewis
Mark Carney	Sandro Gauci

Outros Contribuidores e Revisores

Incluímos uma lista dos demais contribuidores no Apêndice E.

Se algum crédito estiver faltando na lista de créditos da versão 5.x, por favor, abra um ticket no GitHub para ser reconhecido em futuras atualizações 5.x.

O Padrão de Verificação de Segurança de Aplicações baseia-se no trabalho dos envolvidos no ASVS 1.0 (2008) até a 4.0 (2019). Grande parte da estrutura e muitos dos itens de verificação que permanecem no ASVS hoje foram originalmente escritos por Andrew van der Stock, Mike Boberski, Jeff Williams e Dave Wichers, entre vários outros contribuidores. Obrigado a todos que contribuíram no passado. Para uma lista abrangente de contribuidores anteriores, por favor, consulte cada versão anterior. Preface.md

Prefácio

Bem-vindo à Versão 5.0 do Padrão de Verificação de Segurança de Aplicações (ASVS).

Introdução

Lançado originalmente em 2008 por meio de uma colaboração da comunidade global, o ASVS define um conjunto abrangente de requisitos de segurança para projetar, desenvolver e testar aplicações e serviços web modernos.

Após o lançamento do ASVS 4.0 em 2019 e sua pequena atualização (v4.0.3) em 2021, a Versão 5.0 representa um marco significativo —modernizada para refletir os mais recentes avanços na segurança de software.

O ASVS 5.0 é o resultado de extensas contribuições de líderes de projetos, membros do grupo de trabalho e da comunidade OWASP em geral para atualizar e melhorar este importante padrão.

Princípios por trás da versão 5.0

Esta grande revisão foi desenvolvida com vários princípios-chave em mente:

- **Escopo e Foco Refinados:** Esta versão do padrão foi projetada para se alinhar mais diretamente com os pilares fundamentais em seu nome: Aplicação (Application), Segurança (Security), Verificação (Verification) e Padrão (Standard). Os requisitos foram reescritos para enfatizar a pre-

venção de falhas de segurança, em vez de exigir implementações técnicas específicas. Os textos dos requisitos pretendem ser autoexplicativos, justificando o motivo de sua existência.

- **Suporte para Decisões de Segurança Documentadas:** O ASVS 5.0 introduz requisitos para documentar as principais decisões de segurança. Isso melhora a rastreabilidade e oferece suporte a implementações sensíveis ao contexto, permitindo que as organizações adaptem sua postura de segurança às suas necessidades e riscos específicos.
- **Níveis Atualizados:** Embora o ASVS mantenha seu modelo de três camadas, as definições de nível evoluíram para tornar o ASVS mais fácil de adotar. O Nível 1 é projetado como o passo inicial para a adoção do ASVS, fornecendo a primeira camada de defesa. O Nível 2 representa uma visão abrangente das práticas padrão de segurança, e o Nível 3 aborda requisitos avançados e de alta garantia.
- **Conteúdo Reestruturado e Expandido:** O ASVS 5.0 inclui aproximadamente 350 requisitos distribuídos em 17 capítulos. Os capítulos foram reorganizados para maior clareza e usabilidade. Um mapeamento bidirecional entre as versões 4.0 e 5.0 é fornecido para facilitar a migração.

Olhando para o futuro

Assim como a segurança de uma aplicação nunca está verdadeiramente terminada, o ASVS também não está. Embora a Versão 5.0 seja um lançamento importante, o desenvolvimento continua. Este lançamento permite que a comunidade em geral se beneficie das melhorias e adições que foram acumuladas, mas também estabelece as bases para aprimoramentos futuros. Isso pode incluir esforços impulsionados pela comunidade para criar guias de implementação e verificação construídos sobre o conjunto central de requisitos.

O ASVS 5.0 foi projetado para servir como uma base confiável para o desenvolvimento seguro de software. A comunidade está convidada a adotar, contribuir e construir sobre este padrão para avançar coletivamente o estado da segurança de aplicações.

O que é o ASVS?

O Padrão de Verificação de Segurança de Aplicações (Application Security Verification Standard - ASVS) define requisitos de segurança para aplicações e serviços web, e é um recurso valioso para qualquer pessoa que tenha como objetivo projetar, desenvolver e manter aplicações seguras ou avaliar sua segurança.

Este capítulo descreve os aspectos essenciais do uso do ASVS, incluindo seu escopo, a estrutura de seus níveis baseados em prioridade e os principais casos de uso para o padrão.

Escopo do ASVS

O escopo do ASVS é definido pelo seu nome: Aplicação (Application), Segurança (Security), Verificação (Verification) e Padrão (Standard). Ele estabelece quais requisitos são incluídos ou excluídos, com o objetivo geral de identificar os princípios de segurança que devem ser alcançados. O escopo também considera os requisitos de documentação, que servem como base para os requisitos de implementação.

Não existe o conceito de escopo para atacantes. Portanto, os requisitos do ASVS devem ser avaliados em conjunto com as diretrizes para outros aspectos do ciclo de vida da aplicação, incluindo processos de CI/CD, hospedagem e atividades operacionais.

Aplicação

O ASVS define uma “aplicação” como o produto de software em desenvolvimento, no qual controles de segurança devem ser integrados. O ASVS não prescreve atividades do ciclo de vida de desenvolvimento ou dita como a aplicação deve ser construída através de um pipeline de CI/CD; em vez disso, ele especifica os resultados de segurança que devem ser alcançados dentro do próprio produto.

Componentes que servem, modificam ou validam tráfego HTTP, como Web Application Firewalls (WAFs), balanceadores de carga ou proxies, podem ser considerados parte da aplicação para esses propósitos específicos, pois alguns controles de segurança dependem diretamente deles ou podem ser implementados através deles. Estes componentes devem ser considerados para requisitos relacionados a respostas em cache, limitação de taxa (rate limiting) ou restrição de conexões de entrada e saída com base na origem e no destino.

Por outro lado, o ASVS geralmente exclui requisitos que não são diretamente relevantes para a aplicação ou onde a configuração está fora da responsabilidade da aplicação. Por exemplo, problemas de DNS são tipicamente gerenciados por uma equipe ou função separada.

Da mesma forma, embora a aplicação seja responsável por como ela consome entradas e produz saídas, se um processo externo interage com a aplicação ou seus dados, ele é considerado fora do escopo do ASVS. Por exemplo, o backup da aplicação ou de seus dados geralmente é responsabilidade de um processo externo e não é controlado pela aplicação ou por seus desenvolvedores.

Segurança

Todo requisito deve ter um impacto demonstrável na segurança. A ausência de um requisito deve resultar em uma aplicação menos segura, e a implementação do requisito deve reduzir a probabilidade ou o impacto de um risco de segurança.

Todas as outras considerações, como aspectos funcionais, estilo de código ou requisitos de política, estão fora do escopo.

Verificação

O requisito deve ser verificável, e a verificação deve resultar em uma decisão de “falha”(fail) ou “aprovação”(pass).

Padrão

O ASVS foi projetado para ser uma coleção de requisitos de segurança a serem implementados para estar em conformidade com o padrão. Isso significa que os requisitos se limitam a definir a meta de segurança para alcançar isso. Outras informações relacionadas podem ser construídas sobre o ASVS ou vinculadas por meio de mapeamentos.

Especificamente, a OWASP tem muitos projetos, e o ASVS evita deliberadamente a sobreposição com o conteúdo de outros projetos. Por exemplo, desenvolvedores podem ter a dúvida: “como implementar um requisito específico na minha tecnologia ou ambiente particular?”, e isso deve ser coberto pelo projeto Cheat Sheet Series. Verificadores podem ter a dúvida: “como testar este requisito neste ambiente?”, e isso deve ser coberto pelo projeto Web Security Testing Guide (WSTG).

Embora o ASVS não seja destinado apenas ao uso por especialistas em segurança, espera-se que o leitor tenha conhecimento técnico para entender o conteúdo ou a capacidade de pesquisar conceitos específicos.

Requisito

A palavra requisito é usada especificamente no ASVS pois descreve o que deve ser alcançado para satisfazê-lo. O ASVS contém apenas requisitos (deve / must) e não contém recomendações (deveria / should) como condição principal.

Em outras palavras, recomendações, sejam elas apenas uma das muitas opções possíveis para resolver um problema ou considerações de estilo de código, não satisfazem a definição para serem um requisito.

Os requisitos do ASVS pretendem abordar princípios específicos de segurança sem serem muito específicos a implementações ou tecnologias, e ao mesmo tempo, sendo autoexplicativos quanto ao motivo de existirem. Isso também significa que os requisitos não são construídos em torno de um método de verificação ou implementação particular.

Decisões de segurança documentadas

Na segurança de software, planejar o design de segurança e os mecanismos a serem usados logo no início levará a uma implementação mais consistente e confiável no produto ou funcionalidade final.

Além disso, para certos requisitos, a implementação será complicada e muito específica às necessidades de uma aplicação. Exemplos comuns incluem permissões, validação de entrada e controles de proteção em torno de diferentes níveis de dados sensíveis.

Para levar isso em conta, em vez de afirmações generalistas como “todos os dados devem ser criptografados” ou tentar cobrir todos os casos de uso possíveis em um requisito, foram incluídos requisitos de documentação que exigem que a abordagem e a configuração do desenvolvedor da aplicação para esses tipos de controles sejam documentadas. Isso pode então ser revisado quanto à adequação, e a implementação real pode ser comparada à documentação para avaliar se a implementação corresponde às expectativas.

Estes requisitos têm o objetivo de documentar as decisões que a organização que desenvolve a aplicação tomou em relação a como implementar determinados requisitos de segurança.

Os requisitos de documentação estão sempre na primeira seção de um capítulo (embora nem todo capítulo os tenha) e sempre têm um requisito de implementação relacionado onde as decisões documentadas devem realmente ser colocadas em prática. O ponto aqui é que verificar se a documentação está em vigor e se a implementação real ocorreu são duas atividades separadas.

Existem dois motivadores principais para incluir esses requisitos. O primeiro motivador é que um requisito de segurança muitas vezes envolverá a aplicação de regras, por exemplo, que tipo de arquivos são permitidos para upload, quais controles de negócios devem ser aplicados, quais são os caracteres permitidos para um campo específico. Essas regras serão diferentes para cada aplicação e, portanto, o ASVS não pode definir prescritivamente quais devem ser, nem um cheat sheet ou uma resposta mais detalhada ajudará neste caso. Da mesma forma, sem que essas decisões sejam documentadas, não será possível realizar a verificação dos requisitos que implementam essas decisões.

O segundo motivador é que, para certos requisitos, é importante fornecer ao desenvolvimento da aplicação flexibilidade sobre como abordar desafios de segurança particulares. Por exemplo, em versões anteriores do ASVS, as regras de timeout de sessão eram muito prescritivas. Na prática, muitas aplicações, especialmente aquelas voltadas para o consumidor, têm regras muito mais flexíveis e preferem implementar outros controles de mitigação em seu lugar. Os requisitos de documentação, portanto, permitem explicitamente flexibilidade em torno disso.

Claramente, não se espera que desenvolvedores individuais tomem e documentem essas decisões, mas sim que a organização como um todo tome essas decisões e garanta que sejam comunicadas aos desenvolvedores, que então se certificam de segui-las.

Fornecer aos desenvolvedores especificações e designs para novos recursos e funcionalidades é uma parte padrão do desenvolvimento de software. Da mesma forma, espera-se que os desenvolvedores usem componentes comuns e mecanismos de interface de usuário em vez de apenas tomar suas próprias decisões a cada vez. Sendo assim, estender isso para a segurança não deve ser visto como algo surpreendente ou controverso.

Também há flexibilidade sobre como alcançar isso. Decisões de segurança podem ser documentadas em um documento literal, ao qual se espera que os desenvolvedores consultem. Alternativamente, as decisões de segurança podem ser documentadas e implementadas em uma biblioteca de código

comum que todos os desenvolvedores são obrigados a usar. Em ambos os casos, o resultado desejado é alcançado.

Níveis de Verificação de Segurança de Aplicações

O ASVS define três níveis de verificação de segurança, com cada nível aumentando em profundidade e complexidade. O objetivo geral é que as organizações comecem pelo primeiro nível para lidar com as preocupações de segurança mais críticas e, em seguida, avancem para os níveis mais altos de acordo com as necessidades da organização e da aplicação. Os níveis podem ser apresentados como L1, L2 e L3 no documento e nos textos dos requisitos.

Cada nível do ASVS indica os requisitos de segurança que são obrigatórios para atingir aquele nível, com os requisitos restantes dos níveis superiores servindo como recomendações.

A fim de evitar requisitos duplicados ou requisitos que não são mais relevantes em níveis mais altos, alguns requisitos se aplicam a um nível específico, mas têm condições mais rigorosas para níveis superiores.

Avaliação de Nível

Os níveis são definidos por uma avaliação baseada em prioridade de cada requisito, fundamentada na experiência de implementação e teste de requisitos de segurança. O foco principal é comparar a redução de risco com o esforço para implementar o requisito. Outro fator-chave é manter uma barreira de entrada baixa.

A redução de risco considera até que ponto o requisito reduz o nível de risco de segurança dentro da aplicação, levando em conta os clássicos fatores de impacto de Confidencialidade, Integridade e Disponibilidade (CIA), bem como considerando se esta é uma camada primária de defesa ou se seria considerada defesa em profundidade (defense in depth).

As discussões rigorosas em torno dos critérios e das decisões de nivelamento resultaram em uma alocação que deve ser verdadeira para a grande maioria dos casos, embora se aceite que pode não ser 100% adequada para todas as situações. Isso significa que, em certos casos, as organizações podem desejar priorizar requisitos de um nível mais alto mais cedo, com base em suas próprias considerações de risco específicas.

Os tipos de requisitos em cada nível podem ser caracterizados da seguinte forma.

Nível 1

Este nível contém os requisitos mínimos a serem considerados ao proteger uma aplicação e representa um ponto de partida crítico. Este nível contém cerca de 20% dos requisitos do ASVS. O objetivo para este nível é ter o menor número possível de requisitos, para diminuir a barreira de entrada.

Esses requisitos geralmente são críticos ou básicos, requisitos de primeira camada de defesa para prevenir ataques comuns que não requerem outras vulnerabilidades ou pré-condições para serem exploráveis.

Além dos requisitos de primeira camada de defesa, alguns requisitos têm um impacto menor em níveis mais altos, como os requisitos relacionados a senhas. Eles são mais importantes para o Nível 1, pois a partir de níveis mais altos, os requisitos de autenticação multifator (MFA) tornam-se relevantes.

O Nível 1 não é necessariamente testável por intrusão (penetration test) por um testador externo sem acesso interno a documentação ou código (como em testes do tipo “caixa preta” ou “black box”), embora o menor número de requisitos deva torná-lo mais fácil de verificar.

Nível 2

A maioria das aplicações deveria se esforçar para atingir este nível de segurança. Cerca de 50% dos requisitos no ASVS são L2, o que significa que uma aplicação precisa implementar cerca de 70% dos requisitos no ASVS (todos os requisitos L1 e L2) a fim de estar em conformidade com o L2.

Esses requisitos geralmente se relacionam com ataques menos comuns ou proteções mais complicadas contra ataques comuns. Eles ainda podem ser uma primeira camada de defesa ou podem exigir certas pré-condições para que o ataque seja bem-sucedido.

Nível 3

Este nível deve ser a meta para aplicações que buscam demonstrar os mais altos níveis de segurança e fornece os cerca de 30% finais dos requisitos a serem cumpridos.

Os requisitos nesta seção geralmente são mecanismos de defesa em profundidade ou outros controles úteis, mas difíceis de implementar.

Qual nível alcançar

Os níveis baseados em prioridade destinam-se a fornecer um reflexo da maturidade de segurança de aplicações da organização e da aplicação. Em vez de o ASVS afirmar de forma prescritiva em qual nível uma aplicação deve estar, uma organização deve analisar seus riscos e decidir em qual nível acredita que deve estar, dependendo da sensibilidade da aplicação e, claro, das expectativas dos usuários da aplicação.

Por exemplo, uma startup em estágio inicial que coleta apenas dados sensíveis limitados pode decidir se concentrar no Nível 1 para seus objetivos de segurança iniciais, mas um banco pode ter dificuldade em justificar qualquer coisa inferior ao Nível 3 aos seus clientes para sua aplicação de internet banking.

Como usar o ASVS

A estrutura do ASVS

O ASVS é composto por um total de cerca de 350 requisitos, que são divididos em 17 capítulos, cada um dos quais é subdividido em seções.

O objetivo da divisão de capítulos e seções é simplificar a escolha ou filtragem de capítulos e seções com base no que é relevante para a aplicação. Por exemplo, para uma API máquina a máquina (machine-to-machine), os requisitos do capítulo V3 relacionados a frontends web não serão relevantes. Se não houver uso de OAuth ou WebRTC, esses capítulos também podem ser ignorados.

Estratégia de Lançamento

Os lançamentos do ASVS seguem o padrão “Major.Minor.Patch”(Maior.Menor.Correção) e os números fornecem informações sobre o que mudou na versão. Em uma versão principal (major), o primeiro número mudará; em uma versão menor (minor), o segundo número mudará; e em uma versão de correção (patch), o terceiro número mudará.

- Major release (Lançamento Maior) - Reorganização completa, quase tudo pode ter mudado, incluindo os números dos requisitos. A reavaliação de conformidade será necessária (por exemplo, 4.0.3 -> 5.0.0).
- Minor release (Lançamento Menor) - Requisitos podem ser adicionados ou removidos, mas a numeração geral permanecerá a mesma. A reavaliação de conformidade será necessária, mas deve ser mais fácil (por exemplo, 5.0.0 -> 5.1.0).
- Patch release (Lançamento de Correção) - Requisitos podem ser removidos (por exemplo, se forem duplicados ou desatualizados) ou tornados menos rigorosos, mas uma aplicação que estava em conformidade com o lançamento anterior também estará em conformidade com a versão de correção (por exemplo, 5.0.0 -> 5.0.1).

O acima relaciona-se especificamente aos requisitos do ASVS. Mudanças no texto ao redor e em outros conteúdos, como os apêndices, não serão consideradas uma mudança com quebra de compatibilidade (breaking change).

Flexibilidade com o ASVS

Vários dos pontos descritos acima, como requisitos de documentação e o mecanismo de níveis, fornecem a capacidade de usar o ASVS de forma mais flexível e específica da organização.

Além disso, as organizações são fortemente encorajadas a criar um fork (ramificação) específico da organização ou do domínio que ajuste os requisitos com base nas características e níveis de risco específicos de suas aplicações. No entanto, é importante manter a rastreabilidade para que ser aprovado no requisito 4.1.1 signifique o mesmo em todas as versões.

O ideal é que cada organização crie seu próprio ASVS personalizado, omitindo seções irrelevantes (por exemplo, GraphQL, WebSockets, SOAP, se não forem usados). Uma versão ou suplemento do ASVS específico da organização também é um bom lugar para fornecer orientações de implementação específicas da organização, detalhando bibliotecas ou recursos a serem usados ao cumprir os requisitos.

Como Referenciar os Requisitos do ASVS

Cada requisito tem um identificador no formato <capítulo>.<seção>.<requisito>, onde cada elemento é um número. Por exemplo, 1.11.3.

- O valor <capítulo> corresponde ao capítulo do qual o requisito se origina; por exemplo, todos os requisitos 1.#.# são do capítulo ‘Codificação e Sanitização’(Encoding and Sanitization).
- O valor <seção> corresponde à seção dentro daquele capítulo onde o requisito aparece, por exemplo: todos os requisitos 1.2.# estão na seção ‘Prevenção de Injeção’(Injection Prevention) do capítulo ‘Codificação e Sanitização’.
- O valor <requisito> identifica o requisito específico dentro do capítulo e seção, por exemplo, 1.2.5 que a partir da versão 5.0.0 deste padrão é:

Verifique se a aplicação protege contra injeção de comando no sistema operacional (OS) e se as chamadas de sistema operacional usam consultas de OS parametrizadas ou usam codificação de saída de linha de comando contextual.

Como os identificadores podem mudar entre versões do padrão, é preferível que outros documentos, relatórios ou ferramentas usem o seguinte formato: v<versão>-<capítulo>.<seção>.<requisito>, onde: ‘versão’ é a tag da versão do ASVS. Por exemplo: v5.0.0-1.2.5 seria entendido como especificamente o 5º requisito na seção ‘Prevenção de Injeção’ do capítulo ‘Codificação e Sanitização’ da versão 5.0.0. (Isso poderia ser resumido como v<versão>-<identificador_do_requisito>.)

Nota: A letra v que precede o número da versão no formato deve sempre ser minúscula.

Se os identificadores forem usados sem incluir o elemento v<versão>, deve-se presumir que eles se referem ao conteúdo mais recente do Application Security Verification Standard. À medida que o padrão cresce e muda, isso se torna problemático, e é por isso que escritores ou desenvolvedores devem incluir o elemento de versão.

As listas de requisitos do ASVS são disponibilizadas em formatos CSV, JSON e outros que podem ser úteis para referência ou uso programático.

Fazendo Fork do ASVS

As organizações podem se beneficiar da adoção do ASVS escolhendo um dos três níveis ou criando um fork específico do domínio que ajusta os requisitos de acordo com o nível de risco da aplicação. Esse

tipo de fork é encorajado, desde que mantenha a rastreabilidade para que a aprovação no requisito 4.1.1 signifique o mesmo em todas as versões.

Idealmente, cada organização deve criar seu próprio ASVS adaptado, omitindo seções irrelevantes (por exemplo, GraphQL, WebSockets, SOAP, se não forem usados). O forking deve começar com o Nível 1 do ASVS como base, avançando para os Níveis 2 ou 3 com base no risco da aplicação.

Casos de uso para o ASVS

O ASVS pode ser usado para avaliar a segurança de uma aplicação, e isso é explorado mais a fundo no próximo capítulo. No entanto, vários outros usos potenciais para o ASVS (ou uma versão em fork) foram identificados.

Como Guia Detalhado de Arquitetura de Segurança

Um dos usos mais comuns do Application Security Verification Standard é como um recurso para arquitetos de segurança. Existem recursos limitados disponíveis sobre como construir uma arquitetura de aplicação segura, especialmente para aplicações modernas. O ASVS pode ser usado para preencher essas lacunas, permitindo que os arquitetos de segurança escolham melhores controles para problemas comuns, como padrões de proteção de dados e estratégias de validação de entrada. Os requisitos de arquitetura e documentação serão particularmente úteis para isso.

Como Referência Especializada de Codificação Segura

O ASVS pode ser usado como base para preparar uma referência de codificação segura durante o desenvolvimento de aplicações, ajudando os desenvolvedores a garantirem que mantenham a segurança em mente ao construir software. Embora o ASVS possa ser a base, as organizações devem preparar suas próprias diretrizes específicas, que sejam claras e unificadas, e idealmente preparadas com base na orientação de engenheiros de segurança ou arquitetos de segurança. Como extensão a isso, as organizações são encorajadas, sempre que possível, a preparar mecanismos de segurança e bibliotecas aprovadas que possam ser referenciadas nas diretrizes e usadas pelos desenvolvedores.

Como Guia para Testes Automatizados de Unidade e Integração

O ASVS foi projetado para ser altamente testável. Algumas verificações serão técnicas, enquanto outros requisitos (como os requisitos arquitetônicos e de documentação) podem exigir revisão de documentação ou de arquitetura. Ao construir testes de unidade e de integração que testem e façam fuzzing para casos de abuso específicos e relevantes relacionados aos requisitos que são verificáveis por meios técnicos, deve ser mais fácil verificar se esses controles estão operando corretamente em cada build. Por exemplo, testes adicionais podem ser elaborados para a suíte de testes de um controlador de login, testando o parâmetro de nome de usuário para nomes de usuário padrão comuns,

enumeração de contas, força bruta, injeção de LDAP e SQL e XSS. Da mesma forma, um teste no parâmetro de senha deve incluir senhas comuns, comprimento de senha, injeção de byte nulo, remoção do parâmetro, XSS e muito mais.

Para Treinamento de Desenvolvimento Seguro

O ASVS também pode ser usado para definir as características de um software seguro. Muitos cursos de “codificação segura” são simplesmente cursos de hacking ético com uma leve camada de dicas de codificação. Isso pode não necessariamente ajudar os desenvolvedores a escreverem códigos mais seguros. Em vez disso, cursos de desenvolvimento seguro podem usar o ASVS com um forte foco nos mecanismos positivos encontrados no ASVS, em vez das 10 principais coisas negativas que não se deve fazer (Top 10). A estrutura do ASVS também fornece uma estrutura lógica para percorrer os diferentes tópicos ao proteger uma aplicação.

Como Framework para Guiar a Aquisição de Software Seguro

O ASVS é um excelente framework para ajudar na aquisição de software seguro ou na aquisição de serviços de desenvolvimento sob medida. O comprador pode simplesmente estabelecer um requisito de que o software que deseja adquirir deve ser desenvolvido no nível X do ASVS e solicitar que o vendedor comprove que o software satisfaz o nível X do ASVS.

Aplicando o ASVS na Prática

Ameaças diferentes têm motivações diferentes. Algumas indústrias possuem ativos de informação e tecnologia únicos, bem como requisitos de conformidade regulatória específicos de domínio.

As organizações são fortemente encorajadas a analisar profundamente suas características únicas de risco com base na natureza de seus negócios e, com base nesse risco e nos requisitos de negócios, determinar o nível apropriado do ASVS.

Avaliação e Certificação

A Postura da OWASP sobre Certificações e Selos de Confiança do ASVS

A OWASP, sendo uma organização sem fins lucrativos e neutra em relação a fornecedores, não certifica nenhum fornecedor, verificador ou software. Qualquer garantia, selo de confiança ou certificação que alegue conformidade com o ASVS não é oficialmente endossado pela OWASP, portanto, as organizações devem ser cautelosas com alegações de certificação do ASVS feitas por terceiros.

As organizações podem oferecer serviços de garantia (assurance services), desde que não aleguem certificação oficial da OWASP.

Como Verificar a Conformidade com o ASVS

O ASVS deliberadamente não é prescritivo sobre exatamente como verificar a conformidade no nível de um guia de testes. No entanto, é importante destacar alguns pontos-chave.

Relatório de Verificação

Os relatórios tradicionais de testes de intrusão (penetration testing) relatam problemas “por exceção”, listando apenas as falhas. No entanto, um relatório de certificação do ASVS deve incluir o escopo, um resumo de todos os requisitos verificados, os requisitos onde foram observadas exceções e orientações sobre como resolver os problemas. Alguns requisitos podem não ser aplicáveis (por exemplo, gerenciamento de sessão em APIs sem estado/stateless), e isso deve ser observado no relatório.

Escopo da Verificação

Uma organização que desenvolve uma aplicação geralmente não implementará todos os requisitos, pois alguns podem ser irrelevantes ou menos significativos com base na funcionalidade da aplicação. O verificador deve deixar o escopo da verificação claro, incluindo qual Nível a organização está tentando alcançar e quais requisitos foram incluídos. Isso deve ser feito sob a perspectiva do que foi incluído, em vez do que não foi incluído. Eles também devem fornecer uma opinião sobre a justificativa para a exclusão dos requisitos que não foram implementados.

Isso deve permitir que o consumidor de um relatório de verificação entenda o contexto da verificação e tome uma decisão informada sobre o nível de confiança que pode depositar na aplicação.

As organizações certificadoras podem escolher seus métodos de teste, mas devem divulgá-los no relatório, e idealmente, isso deve ser repetível. Diferentes métodos, como testes manuais de intrusão ou análise de código-fonte, podem ser usados para verificar aspectos como validação de entrada, dependendo da aplicação e dos requisitos.

Mecanismos de Verificação

Existem diversas técnicas diferentes que podem ser necessárias para verificar requisitos específicos do ASVS. Além dos testes de intrusão (usando credenciais válidas para obter cobertura total da aplicação), a verificação dos requisitos do ASVS pode exigir acesso à documentação, ao código-fonte, às configurações e às pessoas envolvidas no processo de desenvolvimento. Especialmente para verificar os requisitos L2 e L3. É prática padrão fornecer evidências robustas das descobertas com documentação detalhada, que pode incluir documentos de trabalho, capturas de tela (screenshots), scripts e logs de testes. Apenas executar uma ferramenta automatizada sem testes minuciosos é insuficiente para a certificação, pois cada requisito deve ser comprovadamente testado.

O uso de automação para verificar requisitos do ASVS é um tópico de constante interesse. Portanto, é importante esclarecer alguns pontos relacionados a testes automatizados e testes de caixa preta (black box).

O Papel das Ferramentas de Teste de Segurança Automatizado

Quando ferramentas automatizadas de teste de segurança, como as ferramentas DAST (Dynamic Application Security Testing) e SAST (Static Application Security Testing), são implementadas corretamente no pipeline de construção (build), elas podem ser capazes de identificar alguns problemas de segurança que nunca deveriam existir. No entanto, sem configuração e ajuste cuidadosos, elas não fornecerão a cobertura exigida, e o nível de ruído impedirá que problemas reais de segurança sejam identificados e mitigados.

Embora isso possa fornecer cobertura para alguns dos requisitos técnicos mais básicos e diretos, como os relacionados à codificação ou sanitização de saída, é fundamental notar que essas ferramentas serão incapazes de verificar inteiramente muitos dos requisitos mais complicados do ASVS ou aqueles que se relacionam à lógica de negócios e ao controle de acesso.

Para requisitos menos diretos, é provável que a automação ainda possa ser utilizada, mas verificações específicas da aplicação precisarão ser escritas para alcançar isso. Estas podem ser semelhantes aos testes de unidade e de integração que a organização já pode estar usando. Portanto, pode ser possível usar essa infraestrutura de automação de testes existente para escrever esses testes específicos do ASVS. Embora fazer isso exija investimento de curto prazo, os benefícios de longo prazo de ser capaz de verificar continuamente esses requisitos do ASVS serão significativos.

Em resumo, testável usando automação != executar uma ferramenta pronta para uso (off the shelf).

O Papel do Teste de Intrusão

Enquanto o L1 na versão 4.0 foi otimizado para a ocorrência de testes “caixa preta”(sem documentação e sem código-fonte), mesmo na época o padrão era claro de que esta não é uma atividade de garantia eficaz e deve ser ativamente desencorajada.

Testar sem acesso a informações adicionais necessárias é um mecanismo ineficiente e ineficaz para verificação de segurança, pois perde a possibilidade de revisar o código-fonte, identificar ameaças e controles ausentes, e realizar um teste muito mais completo em um prazo mais curto.

É fortemente encorajado realizar testes de intrusão guiados por documentação ou código-fonte (híbridos), que tenham acesso total aos desenvolvedores da aplicação e à documentação da aplicação, em vez de testes de intrusão tradicionais. Isso certamente será necessário para verificar muitos dos requisitos do ASVS.

Alterações em Relação à Versão 4.x

Introdução

Os usuários familiarizados com a versão 4.x do padrão podem achar útil revisar as principais alterações introduzidas na versão 5.0, incluindo atualizações de conteúdo, escopo e filosofia subjacente.

Dos 286 requisitos da versão 4.0.3, apenas 11 permanecem inalterados, enquanto 15 passaram por pequenos ajustes gramaticais sem alterar seu significado. No total, 109 requisitos (38%) não são mais requisitos separados na versão 5.0, com 50 simplesmente sendo excluídos, 28 removidos como duplicados e 31 mesclados em outros requisitos. O restante foi revisado de alguma forma. Mesmo os requisitos que não foram substancialmente modificados têm identificadores diferentes devido à reordenação ou reestruturação.

Para facilitar a adoção da versão 5.0, documentos de mapeamento são fornecidos para ajudar os usuários a rastrear como os requisitos da versão 4.x correspondem aos da versão 5.0. Estes mapeamentos não estão vinculados ao versionamento de lançamentos e podem ser atualizados ou esclarecidos conforme necessário.

Filosofia dos Requisitos

Escopo e Foco

A versão 4.x incluía requisitos que não se alinhavam com o escopo pretendido do padrão; estes foram removidos. Requisitos que não atendiam aos critérios de escopo para a versão 5.0 ou não eram verificáveis também foram excluídos.

Ênfase nos Objetivos de Segurança Sobre os Mecanismos

Na versão 4.x, muitos requisitos se concentravam em mecanismos específicos, em vez dos objetivos de segurança subjacentes. Na versão 5.0, os requisitos estão centrados nas metas de segurança, fazendo referência a mecanismos particulares apenas quando são a única solução prática, ou fornecendo-os como exemplos ou orientação complementar.

Essa abordagem reconhece que podem existir vários métodos para atingir um determinado objetivo de segurança e evita uma prescritividade desnecessária que poderia limitar a flexibilidade organizacional.

Além disso, requisitos que abordam a mesma preocupação de segurança foram consolidados quando apropriado.

Decisões de Segurança Documentadas

Embora o conceito de decisões de segurança documentadas possa parecer novo na versão 5.0, ele é uma evolução de requisitos anteriores relacionados à aplicação de políticas e modelagem de ameaças na versão 4.0. Anteriormente, alguns requisitos exigiam implicitamente análises para informar a implementação de controles de segurança, como a determinação de conexões de rede permitidas.

Para garantir que as informações necessárias estejam disponíveis para implementação e verificação, essas expectativas agora são explicitamente definidas como requisitos de documentação, tornando-os claros, acionáveis e verificáveis.

Mudanças Estruturais e Novos Capítulos

Vários capítulos na versão 5.0 introduzem conteúdo totalmente novo:

- OAuth e OIDC –Dada a ampla adoção desses protocolos para delegação de acesso e login único (single sign-on), requisitos dedicados foram adicionados para abordar os diversos cenários que os desenvolvedores podem encontrar. Essa área pode eventualmente evoluir para um padrão autônomo, semelhante ao tratamento dos requisitos de Mobile e IoT em versões anteriores.
- WebRTC –À medida que essa tecnologia ganha popularidade, suas considerações e desafios de segurança exclusivos agora são abordados em uma seção dedicada.

Esforços também foram feitos para garantir que capítulos e seções sejam organizados em torno de conjuntos coerentes de requisitos relacionados.

Esta reestruturação levou à criação de capítulos adicionais:

- Tokens Autocontidos (Self-contained Tokens) –Anteriormente agrupados no gerenciamento de sessão, os tokens autocontidos agora são reconhecidos como um mecanismo distinto e um elemento fundamental para comunicação sem estado (como no OAuth e OIDC). Devido às suas implicações de segurança exclusivas, eles são abordados em um capítulo dedicado, com alguns novos requisitos introduzidos na versão 5.x.
- Segurança de Frontend Web –Com a crescente complexidade das aplicações baseadas em navegador e a ascensão de arquiteturas exclusivas de API (API-only), os requisitos de segurança de frontend foram separados em seu próprio capítulo.
- Codificação e Arquitetura Seguras –Novos requisitos que abordam práticas gerais de segurança que não se encaixavam nos capítulos existentes foram agrupados aqui.

Outras mudanças organizacionais na versão 5.0 foram feitas para esclarecer a intenção. Por exemplo, os requisitos de validação de entrada foram movidos para o lado da lógica de negócios, refletindo seu papel na aplicação de regras de negócios, em vez de serem agrupados com sanitização e codificação.

O antigo capítulo de Arquitetura (V1) foi removido. Sua seção inicial continha requisitos que estavam fora do escopo, enquanto as seções subsequentes foram redistribuídas para capítulos relevantes, com requisitos desduplicados e esclarecidos conforme necessário.

Remoção de Mapeamentos Diretos para Outros Padrões

Mapeamentos diretos para outros padrões foram removidos do corpo principal do padrão. O objetivo é preparar um mapeamento com o projeto OWASP Common Requirement Enumeration (CRE), que, por sua vez, vinculará o ASVS a uma série de projetos da OWASP e padrões externos.

Mapeamentos diretos para CWE e NIST não são mais mantidos, conforme explicado abaixo.

Redução do Acoplamento com as Diretrizes de Identidade Digital do NIST

As Diretrizes de Identidade Digital do NIST (SP 800-63) servem há muito tempo como referência para controles de autenticação e autorização. Na versão 4.x, certos capítulos estavam intimamente alinhados com a estrutura e terminologia do NIST.

Embora estas diretrizes continuem sendo uma referência importante, o alinhamento estrito introduziu desafios, incluindo terminologia menos amplamente reconhecida, duplicação de requisitos semelhantes e mapeamentos incompletos. A versão 5.0 se afasta dessa abordagem para melhorar a clareza e a relevância.

Afastamento da Common Weakness Enumeration (CWE)

A Common Weakness Enumeration (CWE) fornece uma taxonomia útil de fraquezas na segurança de software. No entanto, desafios como CWEs exclusivas de categoria, dificuldades em mapear requisitos para uma única CWE e a presença de mapeamentos imprecisos na versão 4.x levaram à decisão de descontinuar os mapeamentos diretos de CWE na versão 5.0.

Repensando as Definições de Nível

A versão 4.x descrevia os níveis como L1 (“Mínimo”), L2 (“Padrão”) e L3 (“Avançado”), com a implicação de que todas as aplicações que lidam com dados sensíveis deveriam atender pelo menos ao L2.

A versão 5.0 aborda vários problemas com essa abordagem, que são descritos nos parágrafos a seguir.

Como uma questão prática, enquanto a versão 4.x usava marcas de seleção (tick marks) para indicadores de nível, a 5.x usa um número simples em todos os formatos do padrão, incluindo markdown, PDF, DOCX, CSV, JSON e XML. Para compatibilidade com versões anteriores, versões legadas das saídas CSV, JSON e XML que ainda usam marcas de seleção também são geradas.

Nível de Entrada Mais Fácil

O feedback indicou que o grande número de requisitos do Nível 1 (~120), combinado com a sua designação como o nível “mínimo” que não é bom o suficiente para a maioria das aplicações, desencorajava

a adoção. A versão 5.0 visa diminuir essa barreira definindo o Nível 1 principalmente em torno de requisitos de primeira camada de defesa, resultando em requisitos mais claros e em menor número nesse nível. Para demonstrar isso numericamente, na versão 4.0.3 havia 128 requisitos L1 em um total de 278 requisitos, representando 46%. Na 5.0.0, existem 70 requisitos L1 de um total de 345 requisitos, representando 20%.

A Falácia da Testabilidade

Um fator-chave na seleção de controles para o Nível 1 na versão 4.x era a sua adequação para avaliação através de testes de intrusão externos tipo “caixa preta”(black box). No entanto, esta abordagem não estava totalmente alinhada com a intenção do Nível 1 como o conjunto mínimo de controles de segurança. Alguns usuários argumentaram que o Nível 1 era insuficiente para proteger as aplicações, enquanto outros o consideravam muito difícil de testar.

Confiar na testabilidade como um critério é algo relativo e, por vezes, enganoso. O fato de um requisito ser testável não garante que ele possa ser testado de maneira automatizada ou direta. Além disso, os requisitos mais facilmente testáveis nem sempre são aqueles com o maior impacto de segurança ou os mais simples de implementar.

Sendo assim, na versão 5.0, as decisões de nível foram tomadas principalmente com base na redução de risco e também mantendo em mente o esforço para implementar.

Não se Trata Apenas de Risco

O uso de níveis prescritivos e baseados em risco que exigem um nível específico para certas aplicações provou ser excessivamente rígido. Na prática, a priorização e a implementação de controles de segurança dependem de vários fatores, incluindo a redução de risco e o esforço necessário para a implementação.

Portanto, as organizações são encorajadas a alcançar o nível que acreditam que deveriam alcançar com base em sua maturidade e na mensagem que desejam transmitir aos seus usuários.

V1 Codificação e Sanitização

Objetivo de Controle

Este capítulo aborda as fraquezas de segurança de aplicações web mais comuns relacionadas ao processamento inseguro de dados não confiáveis. Tais fraquezas podem resultar em várias vulnerabilidades técnicas, onde dados não confiáveis são interpretados de acordo com as regras de sintaxe do interpretador relevante.

Para aplicações web modernas, é sempre melhor usar APIs mais seguras, como consultas parametrizadas, auto-escaping ou frameworks de template. Caso contrário, a codificação de saída,

o escaping ou a sanitização realizados de forma cuidadosa tornam-se críticos para a segurança da aplicação.

A validação de entrada serve como um mecanismo de defesa em profundidade para proteger contra conteúdo inesperado ou perigoso. No entanto, como seu objetivo principal é garantir que o conteúdo recebido corresponda às expectativas funcionais e de negócios, os requisitos relacionados a isso podem ser encontrados no capítulo “Validação e Lógica de Negócios”(Validation and Business Logic).

V1.1 Arquitetura de Codificação e Sanitização

Nas seções abaixo, são fornecidos requisitos específicos de sintaxe ou específicos de interpretador para o processamento seguro de conteúdo inseguro, a fim de evitar vulnerabilidades de segurança. Os requisitos nesta seção cobrem a ordem em que esse processamento deve ocorrer e onde ele deve ser realizado. Eles também visam garantir que, sempre que os dados forem armazenados, eles permaneçam em seu estado original e não sejam armazenados em formato codificado ou com escape (por exemplo, codificação HTML), para evitar problemas de codificação dupla (double encoding).

#	Descrição	Nível
1.1.1	Verifique se a entrada é decodificada ou desescapada (unescaped) para uma forma canônica apenas uma vez, se é decodificada apenas quando os dados codificados nessa forma são esperados, e se isso é feito antes de processar a entrada posteriormente (por exemplo, não é realizado após a validação ou sanitização da entrada).	2
1.1.2	Verifique se a aplicação realiza a codificação e o escaping de saída como uma etapa final antes de ser usada pelo interpretador ao qual se destina, ou pelo próprio interpretador.	2

V1.2 Prevenção de Injeção

A codificação ou escaping de saída, realizada próxima ou adjacente a um contexto potencialmente perigoso, é crítica para a segurança de qualquer aplicação. Normalmente, a codificação e o escaping de saída não são persistidos, mas são usados para tornar a saída segura para uso imediato no interpretador apropriado. Tentar realizar isso muito cedo pode resultar em conteúdo malformatado ou tornar a codificação ou o escaping ineficazes.

Em muitos casos, as bibliotecas de software incluem funções seguras ou mais seguras que realizam isso automaticamente, embora seja necessário garantir que elas estejam corretas para o contexto atual.

#	Descrição	Nível
1.2.1	Verifique se a codificação de saída para uma resposta HTTP, documento HTML ou documento XML é relevante para o contexto exigido, como codificar os caracteres relevantes para elementos HTML, atributos HTML, comentários HTML, CSS ou campos de cabeçalho HTTP, para evitar alterar a mensagem ou a estrutura do documento.	1
1.2.2	Verifique se, ao construir URLs dinamicamente, os dados não confiáveis são codificados de acordo com seu contexto (por exemplo, codificação de URL ou codificação base64url para parâmetros de consulta ou de caminho). Garanta que apenas protocolos de URL seguros sejam permitidos (por exemplo, proibir javascript: ou data:).	1
1.2.3	Verifique se a codificação ou escaping de saída é usado ao construir conteúdo JavaScript dinamicamente (incluindo JSON), para evitar a alteração da mensagem ou da estrutura do documento (para evitar a injeção de JavaScript e JSON).	1
1.2.4	Verifique se a seleção de dados ou consultas a bancos de dados (por exemplo, SQL, HQL, NoSQL, Cypher) usam consultas parametrizadas, ORMs, entity frameworks ou são de outra forma protegidas contra injeção de SQL e outros ataques de injeção de banco de dados. Isso também é relevante ao escrever procedimentos armazenados (stored procedures).	1
1.2.5	Verifique se a aplicação protege contra injeção de comando no sistema operacional (OS) e se as chamadas de sistema operacional usam consultas de OS parametrizadas ou usam codificação de saída de linha de comando contextual.	1
1.2.6	Verifique se a aplicação protege contra vulnerabilidades de injeção de LDAP, ou se controles de segurança específicos para prevenir a injeção de LDAP foram implementados.	2
1.2.7	Verifique se a aplicação é protegida contra ataques de injeção de XPath através do uso de parametrização de consultas ou consultas pré-compiladas.	2
1.2.8	Verifique se os processadores LaTeX estão configurados com segurança (como não usar a flag “-shell-escape”) e se uma lista de permissões (allowlist) de comandos é usada para evitar ataques de injeção LaTeX.	2
1.2.9	Verifique se a aplicação escapa caracteres especiais em expressões regulares (geralmente usando uma barra invertida) para evitar que sejam mal interpretados como metacaracteres.	2

#	Descrição	Nível
1.2.10	Verifique se a aplicação está protegida contra injeção de CSV e Fórmulas. A aplicação deve seguir as regras de escaping definidas na RFC 4180 seções 2.6 e 2.7 ao exportar conteúdo CSV. Adicionalmente, ao exportar para CSV ou outros formatos de planilha (como XLS, XLSX ou ODF), caracteres especiais (incluindo '=', '+', '-', '@', '\t'(tab) e '\0'(caractere nulo)) devem ser escapados com uma aspa simples se aparecerem como o primeiro caractere em um valor de campo.	3

Nota: O uso de consultas parametrizadas ou o escape de SQL nem sempre é suficiente. Partes da consulta, como nomes de tabelas e colunas (incluindo nomes de colunas "ORDER BY"), não podem ser escapadas. A inclusão de dados escapados fornecidos pelo usuário nesses campos resulta em falhas nas consultas ou injeção de SQL.

V1.3 Sanitização

A proteção ideal contra o uso de conteúdo não confiável em um contexto inseguro é usar codificação ou escaping específicos do contexto, o que mantém o mesmo significado semântico do conteúdo inseguro, mas o torna seguro para uso naquele contexto específico, conforme discutido com mais detalhes na seção anterior.

Onde isso não for possível, a sanitização se torna necessária, removendo caracteres ou conteúdos potencialmente perigosos. Em alguns casos, isso pode alterar o significado semântico da entrada, mas, por razões de segurança, pode não haver alternativa.

#	Descrição	Nível
1.3.1	Verifique se todas as entradas HTML não confiáveis provenientes de editores WYSIWYG ou similares são sanitizadas usando uma biblioteca de sanitização HTML conhecida e segura ou um recurso de framework.	1
1.3.2	Verifique se a aplicação evita o uso de eval() ou de outros recursos de execução dinâmica de código, como a Spring Expression Language (SpEL). Onde não houver alternativa, qualquer entrada do usuário a ser incluída deve ser sanitizada antes de ser executada.	1
1.3.3	Verifique se os dados que estão sendo passados para um contexto potencialmente perigoso são sanitizados previamente para aplicar medidas de segurança, como permitir apenas caracteres seguros para esse contexto e cortar entradas muito longas (trimming).	2

#	Descrição	Nível
1.3.4	Verifique se o conteúdo de script Scalable Vector Graphics (SVG) fornecido pelo usuário é validado ou sanitizado para conter apenas tags e atributos (como gráficos de desenho) que são seguros para a aplicação, por exemplo, não conter scripts e foreignObject.	2
1.3.5	Verifique se a aplicação sanitiza ou desabilita o conteúdo fornecido pelo usuário para linguagens de template com suporte a scripts ou expressões, como Markdown, folhas de estilo CSS ou XSL, BBCode ou similares.	2
1.3.6	Verifique se a aplicação protege contra ataques de Falsificação de Solicitação do Lado do Servidor (Server-side Request Forgery - SSRF), validando dados não confiáveis em relação a uma lista de permissões (allowlist) de protocolos, domínios, caminhos e portas, e sanitizando caracteres potencialmente perigosos antes de usar os dados para chamar outro serviço.	2
1.3.7	Verifique se a aplicação protege contra ataques de injeção de template não permitindo que templates sejam criados com base em entradas não confiáveis. Onde não houver alternativa, qualquer entrada não confiável sendo incluída dinamicamente durante a criação do template deve ser sanitizada ou estritamente validada.	2
1.3.8	Verifique se a aplicação sanitiza apropriadamente as entradas não confiáveis antes de usá-las em consultas do Java Naming and Directory Interface (JNDI) e se o JNDI está configurado com segurança para evitar ataques de injeção JNDI.	2
1.3.9	Verifique se a aplicação sanitiza o conteúdo antes de ser enviado ao memcache para prevenir ataques de injeção.	2
1.3.10	Verifique se as format strings (strings de formatação), que podem ser resolvidas de maneira inesperada ou maliciosa quando usadas, são sanitizadas antes de serem processadas.	2
1.3.11	Verifique se a aplicação sanitiza a entrada do usuário antes de passá-la aos sistemas de e-mail para proteger contra injeção SMTP ou IMAP.	2
1.3.12	Verifique se as expressões regulares estão livres de elementos que causam retrocesso exponencial (exponential backtracking) e certifique-se de que a entrada não confiável seja sanitizada para mitigar ataques de ReDoS ou Runaway Regex.	3

V1.4 Memória, String e Código Não Gerenciado (Unmanaged Code)

Os requisitos a seguir abordam os riscos associados ao uso inseguro de memória, que geralmente se aplicam quando a aplicação usa uma linguagem de sistemas ou código não gerenciado (unmanaged code).

Em alguns casos, é possível conseguir isso definindo flags do compilador que habilitam proteções e avisos contra estouro de buffer (buffer overflow), incluindo randomização de pilha e prevenção de execução de dados, e que quebram a build se operações inseguras de ponteiro, memória, format string, inteiros ou strings forem encontradas.

#	Descrição	Nível
1.4.1	Verifique se a aplicação usa aritmética de ponteiros, cópia de memória mais segura e strings com segurança de memória para detectar ou prevenir estouros de pilha (stack), buffer ou heap.	2
1.4.2	Verifique se técnicas de validação de sinal, intervalo e entrada são usadas para evitar estouros de inteiros (integer overflows).	2
1.4.3	Verifique se a memória e os recursos alocados dinamicamente são liberados e se as referências ou ponteiros para a memória liberada são removidos ou definidos como nulos (null) para evitar ponteiros pendentes (dangling pointers) e vulnerabilidades de uso após a liberação (use-after-free).	2

V1.5 Desserialização Segura

A conversão de dados de uma representação armazenada ou transmitida em objetos reais da aplicação (desserialização) tem sido historicamente a causa de várias vulnerabilidades de injeção de código. É importante executar esse processo com cuidado e segurança para evitar esses tipos de problemas.

Em particular, certos métodos de desserialização foram identificados pela documentação da linguagem de programação ou do framework como inseguros e não podem se tornar seguros com dados não confiáveis. Para cada mecanismo em uso, deve-se realizar uma diligência cuidadosa.

#	Descrição	Nível
1.5.1	Verifique se a aplicação configura parsers XML para usar uma configuração restritiva e se recursos inseguros, como a resolução de entidades externas, estão desabilitados para prevenir ataques de Entidade Externa XML (XML eXternal Entity - XXE).	1

#	Descrição	Nível
1.5.2	Verifique se a desserialização de dados não confiáveis impõe tratamento seguro de entrada, como o uso de uma lista de permissões (allowlist) de tipos de objetos ou a restrição de tipos de objetos definidos pelo cliente, para evitar ataques de desserialização. Mecanismos de desserialização explicitamente definidos como inseguros não devem ser usados com entradas não confiáveis.	2
1.5.3	Verifique se diferentes parsers usados na aplicação para o mesmo tipo de dado (por exemplo, parsers JSON, parsers XML, parsers de URL), realizam o parsing de forma consistente e usam o mesmo mecanismo de codificação de caracteres para evitar problemas como vulnerabilidades de Interoperabilidade JSON ou comportamento diferente na análise de URI ou arquivos sendo explorados em ataques de Inclusão Remota de Arquivo (RFI) ou Falsificação de Solicitação do Lado do Servidor (SSRF).	3

Referências

Para mais informações, veja também:

- OWASP LDAP Injection Prevention Cheat Sheet
- OWASP Cross Site Scripting Prevention Cheat Sheet
- OWASP DOM Based Cross Site Scripting Prevention Cheat Sheet
- OWASP XML External Entity Prevention Cheat Sheet
- OWASP Web Security Testing Guide: Client-Side Testing
- OWASP Java Encoding Project
- DOMPurify - Client-side HTML Sanitization Library
- RFC4180 - Common Format and MIME Type for Comma-Separated Values (CSV) Files

Para obter mais informações, especificamente sobre problemas de desserialização ou parsing, consulte:

- OWASP Deserialization Cheat Sheet
- An Exploration of JSON Interoperability Vulnerabilities
- Orange Tsai - A New Era of SSRF Exploiting URL Parser In Trending Programming Languages

V2 Validação e Lógica de Negócios

Objetivo de Controle

Este capítulo tem como objetivo garantir que uma aplicação verificada atenda aos seguintes objetivos de alto nível:

- A entrada recebida pela aplicação corresponde às expectativas de negócios ou funcionais.
- O fluxo da lógica de negócios é sequencial, processado em ordem e não pode ser burlado (bypassed).
- A lógica de negócios inclui limites e controles para detectar e prevenir ataques automatizados, como transferências contínuas de pequenos fundos ou a adição de um milhão de amigos, um de cada vez.
- Fluxos de lógica de negócios de alto valor consideraram casos de abuso e atores mal-intencionados, e possuem proteções contra ataques de falsificação (spoofing), adulteração (tampering), divulgação de informações e elevação de privilégios.

V2.1 Documentação de Validação e Lógica de Negócios

A documentação da lógica de negócios e da validação deve definir claramente os limites da lógica de negócios, regras de validação e consistência contextual de itens de dados combinados, para que fique claro o que precisa ser implementado na aplicação.

#	Descrição	Nível
2.1.1	Verifique se a documentação da aplicação define regras de validação de entrada para saber como verificar a validade de itens de dados em relação a uma estrutura esperada. Pode ser formatos de dados comuns, como números de cartão de crédito, endereços de e-mail, números de telefone, ou pode ser um formato de dados interno.	1
2.1.2	Verifique se a documentação da aplicação define como validar a consistência lógica e contextual de itens de dados combinados, como verificar se o bairro e o CEP (ZIP code) correspondem.	2
2.1.3	Verifique se as expectativas em relação aos limites da lógica de negócios e às validações estão documentadas, incluindo tanto as restrições por usuário quanto as aplicadas globalmente em toda a aplicação.	2

V2.2 Validação de Entrada

Controles eficazes de validação de entrada aplicam expectativas de negócios ou funcionais em torno do tipo de dado que a aplicação espera receber. Isso garante boa qualidade de dados e reduz a su-

perfície de ataque. No entanto, isso não remove ou substitui a necessidade de usar codificação, parametrização ou sanitização corretas ao usar os dados em outro componente ou ao apresentá-los na saída.

Neste contexto, “entrada” pode vir de uma ampla variedade de fontes, incluindo campos de formulário HTML, requisições REST, parâmetros de URL, campos de cabeçalho HTTP, cookies, arquivos no disco, bancos de dados e APIs externas.

Um controle da lógica de negócios pode verificar se uma entrada específica é um número menor que 100. Uma expectativa funcional pode verificar se um número está abaixo de um determinado limite, já que esse número controla quantas vezes um determinado loop ocorrerá, e um número alto pode levar a processamento excessivo e uma possível condição de negação de serviço (Denial of Service).

Embora a validação de esquema não seja explicitamente obrigatória, ela pode ser o mecanismo mais eficaz para cobertura total de validação de APIs HTTP ou outras interfaces que usam JSON ou XML.

Por favor, observe os seguintes pontos sobre Validação de Esquema (Schema Validation):

- A “versão publicada” da especificação de validação do JSON Schema é considerada pronta para produção, mas não é estritamente falando “estável”. Ao usar a validação do JSON Schema, garanta que não haja lacunas com a orientação nos requisitos abaixo.
- Quaisquer bibliotecas de validação JSON Schema em uso também devem ser monitoradas e atualizadas se necessário assim que o padrão for formalizado.
- A validação de DTD não deve ser usada, e a avaliação de DTD no framework deve ser desativada, para evitar problemas com ataques XXE contra DTDs.

#	Descrição	Nível
2.2.1	Verifique se a entrada é validada para impor expectativas funcionais ou de negócios para aquela entrada. Isso deve usar validação positiva em relação a uma lista de permissões (allowlist) de valores, padrões e intervalos, ou ser baseado na comparação da entrada com uma estrutura esperada e limites lógicos de acordo com regras predefinidas. Para L1, o foco pode ser na entrada que é usada para tomar decisões específicas de negócios ou segurança. Para L2 e superior, isso deve se aplicar a todas as entradas.	1
2.2.2	Verifique se a aplicação é projetada para aplicar a validação de entrada em uma camada de serviço confiável. Embora a validação no lado do cliente (client-side validation) melhore a usabilidade e deva ser incentivada, não se deve confiar nela como um controle de segurança.	1
2.2.3	Verifique se a aplicação garante que combinações de itens de dados relacionados sejam razoáveis de acordo com as regras predefinidas.	2

V2.3 Segurança da Lógica de Negócios

Esta seção considera os principais requisitos para garantir que a aplicação imponha os processos de lógica de negócios da maneira correta e não seja vulnerável a ataques que explorem a lógica e o fluxo da aplicação.

#	Descrição	Nível
2.3.1	Verifique se a aplicação processará fluxos da lógica de negócios para o mesmo usuário apenas na ordem sequencial de etapas esperada e sem pular etapas.	1
2.3.2	Verifique se os limites da lógica de negócios são implementados conforme a documentação da aplicação para evitar que falhas da lógica de negócios sejam exploradas.	2
2.3.3	Verifique se as transações estão sendo usadas no nível da lógica de negócios, de tal forma que uma operação da lógica de negócios tenha sucesso na sua totalidade ou seja revertida (rolled back) para o estado correto anterior.	2
2.3.4	Verifique se mecanismos de bloqueio (locking) no nível da lógica de negócios são usados para garantir que recursos de quantidade limitada (como assentos de teatro ou horários de entrega) não possam ser reservados duplamente pela manipulação da lógica da aplicação.	2
2.3.5	Verifique se fluxos de lógica de negócios de alto valor exigem aprovação multiusuário para evitar ações não autorizadas ou acidentais. Isso pode incluir, mas não se limita a, grandes transferências financeiras, aprovações de contratos, acesso a informações confidenciais ou anulações de segurança (safety overrides) na manufatura.	3

V2.4 Antiautomação

Esta seção inclui controles de antiautomação para garantir que interações semelhantes a ações humanas sejam necessárias e que solicitações automatizadas excessivas sejam evitadas.

#	Descrição	Nível
2.4.1	Verifique se há controles de antiautomação em vigor para proteger contra chamadas excessivas a funções da aplicação que podem levar a exfiltração de dados, criação de dados lixo (garbage-data), esgotamento de cotas, violações de limite de taxa (rate-limit), negação de serviço ou uso excessivo de recursos caros.	2

#	Descrição	Nível
2.4.2	Verifique se os fluxos da lógica de negócios requerem temporização humana realista, evitando envios de transações excessivamente rápidos.	3

Referências

Para mais informações, veja também:

- OWASP Web Security Testing Guide: Input Validation Testing
- OWASP Web Security Testing Guide: Business Logic Testing
- Anti-automation can be achieved in many ways, including the use of the OWASP Automated Threats to Web Applications
- OWASP Input Validation Cheat Sheet
- JSON Schema

V3 Segurança de Frontend Web

Objetivo de Controle

Esta categoria concentra-se em requisitos projetados para proteger contra ataques executados via um frontend web. Estes requisitos não se aplicam a soluções máquina a máquina (machine-to-machine).

V3.1 Documentação de Segurança de Frontend Web

Esta seção descreve os recursos de segurança do navegador que devem ser especificados na documentação da aplicação.

#	Descrição	Nível
3.1.1	Verifique se a documentação da aplicação declara os recursos de segurança esperados que os navegadores que usam a aplicação devem suportar (como HTTPS, HTTP Strict Transport Security (HSTS), Content Security Policy (CSP) e outros mecanismos de segurança HTTP relevantes). Deve também definir como a aplicação deve se comportar quando alguns desses recursos não estiverem disponíveis (como avisar o usuário ou bloquear o acesso).	3

V3.2 Interpretação Não Intencional de Conteúdo

Renderizar conteúdo ou funcionalidade em um contexto incorreto pode resultar na execução ou exibição de conteúdo malicioso.

#	Descrição	Nível
3.2.1	Verifique se há controles de segurança em vigor para evitar que navegadores renderizem conteúdo ou funcionalidades de respostas HTTP em um contexto incorreto (por exemplo, quando uma API, um arquivo enviado pelo usuário ou outro recurso é solicitado diretamente). Possíveis controles poderiam incluir: não servir o conteúdo a menos que campos de cabeçalho da requisição HTTP (como Sec-Fetch-*) indiquem que é o contexto correto, usar a diretiva sandbox do campo de cabeçalho Content-Security-Policy ou usar o tipo de disposição attachment no campo de cabeçalho Content-Disposition.	1
3.2.2	Verifique se o conteúdo que deve ser exibido como texto, e não renderizado como HTML, é processado usando funções de renderização seguras (como <code>createTextNode</code> ou <code>textContent</code>) para evitar a execução não intencional de conteúdo como HTML ou JavaScript.	1
3.2.3	Verifique se a aplicação evita a sobrescrita do DOM (DOM clobbering) ao usar JavaScript no lado do cliente (client-side), empregando declarações explícitas de variáveis, realizando verificação estrita de tipo, evitando armazenar variáveis globais no objeto <code>document</code> e implementando isolamento de namespace.	3

V3.3 Configuração de Cookies

Esta seção descreve os requisitos para configurar cookies sensíveis com segurança, a fim de fornecer um maior nível de garantia de que foram criados pela própria aplicação e para evitar que seus conteúdos vazem ou sejam modificados inadequadamente.

#	Descrição	Nível
3.3.1	Verifique se os cookies possuem o atributo 'Secure' definido, e caso o prefixo '___Host-' não seja usado no nome do cookie, o prefixo '___Secure-' deve ser usado para o nome do cookie.	1

#	Descrição	Nível
3.3.2	Verifique se o valor do atributo 'SameSite' de cada cookie é configurado de acordo com o propósito do cookie, para limitar a exposição a ataques de distorção de interface de usuário (user interface redress) e ataques de falsificação de requisição baseados em navegador, comumente conhecidos como falsificação de requisição entre sites (Cross-Site Request Forgery - CSRF).	2
3.3.3	Verifique se os cookies possuem o prefixo '__Host-' para o nome do cookie, a menos que sejam explicitamente projetados para serem compartilhados com outros hosts.	2
3.3.4	Verifique se o valor de um cookie não deve ser acessível a scripts do lado do cliente (como um token de sessão), o cookie deve ter o atributo 'HttpOnly' definido e o mesmo valor (ex. token de sessão) deve ser transferido ao cliente apenas através do campo de cabeçalho 'Set-Cookie'.	2
3.3.5	Verifique se quando a aplicação grava um cookie, o comprimento combinado do nome e do valor do cookie não ultrapassa 4096 bytes. Cookies excessivamente grandes não serão armazenados pelo navegador e, portanto, não serão enviados com requisições, impedindo o usuário de utilizar as funcionalidades da aplicação que dependem desse cookie.	3

V3.4 Cabeçalhos de Mecanismos de Segurança do Navegador

Esta seção descreve quais cabeçalhos de segurança devem ser definidos nas respostas HTTP para habilitar os recursos e as restrições de segurança do navegador ao processar as respostas da aplicação.

#	Descrição	Nível
3.4.1	Verifique se o campo de cabeçalho Strict-Transport-Security está incluído em todas as respostas para forçar uma política HTTP Strict Transport Security (HSTS). A validade máxima (maximum age) de pelo menos 1 ano deve ser definida e, para L2 e superior, a política também deve se aplicar a todos os subdomínios.	1

#	Descrição	Nível
3.4.2	Verifique se o campo de cabeçalho Cross-Origin Resource Sharing (CORS) Access-Control-Allow-Origin possui um valor fixo atribuído pela aplicação ou, se o valor do campo de cabeçalho Origin da requisição HTTP for usado, ele é validado com base em uma lista de permissões (allowlist) de origens confiáveis. Quando 'Access-Control-Allow-Origin: *' precisar ser usado, verifique se a resposta não inclui informações sensíveis.	1
3.4.3	Verifique se as respostas HTTP incluem um campo de cabeçalho de resposta Content-Security-Policy que define diretivas para garantir que o navegador apenas carregue e execute conteúdos ou recursos confiáveis, a fim de limitar a execução de JavaScript malicioso. Como mínimo, uma política global deve ser usada, a qual inclui as diretivas object-src 'none' e base-uri 'none', e define uma lista de permissões ou usa nonces ou hashes. Para uma aplicação L3, deve ser definida uma política por resposta com nonces ou hashes.	2
3.4.4	Verifique se todas as respostas HTTP contêm um campo de cabeçalho 'X-Content-Type-Options: nosniff'. Isso instrui os navegadores a não utilizarem a adivinhação do tipo MIME (content sniffing) para a resposta dada, e exige que o valor do campo de cabeçalho Content-Type da resposta corresponda ao recurso de destino. Por exemplo, a resposta para a requisição de um estilo só é aceita se o Content-Type da resposta for 'text/css'. Isso também permite o uso da funcionalidade Cross-Origin Read Blocking (CORB) pelo navegador.	2
3.4.5	Verifique se a aplicação define uma política de referência (referrer policy) para evitar o vazamento de dados tecnicamente sensíveis para serviços de terceiros através do campo de cabeçalho HTTP da requisição 'Referer'. Isso pode ser feito usando o campo de cabeçalho de resposta HTTP Referrer-Policy ou por meio de atributos de elemento HTML. Dados sensíveis podem incluir caminho e dados de consulta na URL e, para aplicações internas não públicas, o nome do host (hostname) também.	2
3.4.6	Verifique se a aplicação web usa a diretiva frame-ancestors do campo de cabeçalho Content-Security-Policy para toda resposta HTTP para garantir que ela não possa ser incorporada (embedded) por padrão, e que a incorporação de recursos específicos seja permitida apenas quando necessário. Note que o campo de cabeçalho X-Frame-Options, embora suportado por navegadores, está obsoleto e não deve ser confiável.	2
3.4.7	Verifique se o campo de cabeçalho Content-Security-Policy especifica um local para relatar violações.	3

#	Descrição	Nível
3.4.8	Verifique se todas as respostas HTTP que iniciam a renderização de um documento (como respostas com Content-Type text/html) incluem o campo de cabeçalho Cross-Origin-Opener-Policy com a diretiva same-origin ou a diretiva same-origin-allow-popups, conforme exigido. Isso evita ataques que abusam do acesso compartilhado a objetos Window, como tabnabbing e contagem de frames (frame counting).	3

V3.5 Separação de Origem no Navegador

Ao aceitar uma solicitação de funcionalidade sensível no lado do servidor, a aplicação precisa garantir que a solicitação seja iniciada pela própria aplicação ou por uma parte confiável e não tenha sido forjada por um atacante.

Funcionalidades sensíveis neste contexto podem incluir o recebimento de posts de formulários para usuários autenticados e não autenticados (como uma solicitação de autenticação), operações que mudam o estado (state-changing) ou funcionalidades que demandam recursos (como exportação de dados).

As principais proteções aqui são as políticas de segurança do navegador, como a Same Origin Policy (Política de Mesma Origem) para JavaScript e também a lógica do SameSite para cookies. Outra proteção comum é o mecanismo de CORS preflight. Esse mecanismo será fundamental para os endpoints projetados para serem chamados de uma origem diferente, mas também pode ser um mecanismo útil de prevenção contra falsificação de solicitações para endpoints que não foram projetados para serem chamados de uma origem diferente.

#	Descrição	Nível
3.5.1	Verifique se, caso a aplicação não dependa do mecanismo de CORS preflight para impedir que requisições cross-origin não permitidas usem funcionalidades sensíveis, essas requisições sejam validadas para garantir que sejam originadas na própria aplicação. Isso pode ser feito usando e validando tokens antifalsificação ou exigindo campos de cabeçalho HTTP adicionais que não sejam campos de cabeçalho listados como seguros (CORS-safelisted). O objetivo é defender contra ataques de falsificação de solicitações baseadas no navegador, conhecidos como falsificação de solicitação entre sites (Cross-Site Request Forgery - CSRF).	1

#	Descrição	Nível
3.5.2	Verifique se, caso a aplicação dependa do mecanismo de CORS preflight para evitar o uso cross-origin não permitido de funcionalidades sensíveis, não seja possível chamar a funcionalidade com uma solicitação que não acione uma requisição de CORS-preflight. Isso pode exigir a verificação dos valores dos campos de cabeçalho de requisição 'Origin' e 'Content-Type' ou o uso de um campo de cabeçalho extra que não seja CORS-safelisted.	1
3.5.3	Verifique se as solicitações HTTP a funcionalidades sensíveis usam métodos HTTP apropriados como POST, PUT, PATCH ou DELETE, e não métodos definidos pela especificação HTTP como "seguros" (safe), como HEAD, OPTIONS ou GET. Alternativamente, uma validação estrita dos campos de cabeçalho de requisição Sec-Fetch-* pode ser usada para garantir que a solicitação não tenha se originado de uma chamada cross-origin inadequada, de uma solicitação de navegação ou do carregamento de um recurso (como fonte de imagem) onde isso não é esperado.	1
3.5.4	Verifique se as aplicações distintas estão hospedadas em hostnames diferentes para aproveitar as restrições fornecidas pela política de mesma origem (same-origin policy), incluindo a forma como os documentos ou scripts carregados por uma origem podem interagir com os recursos de outra origem e as restrições baseadas no hostname dos cookies.	2
3.5.5	Verifique se as mensagens recebidas pela interface postMessage são descartadas caso a origem da mensagem não seja confiável, ou se a sintaxe da mensagem for inválida.	2
3.5.6	Verifique se a funcionalidade JSONP não está habilitada em nenhuma parte da aplicação para evitar ataques de Inclusão de Script Entre Sites (Cross-Site Script Inclusion - XSSI).	3
3.5.7	Verifique se os dados que requerem autorização não estão incluídos em respostas de recursos de script, como arquivos JavaScript, para evitar ataques de Inclusão de Script Entre Sites (Cross-Site Script Inclusion - XSSI).	3
3.5.8	Verifique se os recursos autenticados (como imagens, vídeos, scripts e outros documentos) podem ser carregados ou incorporados (embedded) em nome do usuário apenas quando pretendido. Isso pode ser alcançado através da validação estrita dos campos de cabeçalho de requisição HTTP Sec-Fetch-* para garantir que a requisição não se originou de uma chamada cross-origin inapropriada, ou através da definição de um campo de cabeçalho restritivo de resposta HTTP Cross-Origin-Resource-Policy para instruir o navegador a bloquear o conteúdo retornado.	3

V3.6 Integridade de Recursos Externos

Esta seção fornece orientações para a hospedagem segura de conteúdo em sites de terceiros.

#	Descrição	Nível
3.6.1	Verifique se ativos do lado do cliente (client-side assets), como bibliotecas JavaScript, CSS ou fontes da web (web fonts), são hospedados externamente (ex., em uma Content Delivery Network - CDN) apenas se o recurso for estático e versionado, e se a Integridade de Sub-recurso (Subresource Integrity - SRI) for usada para validar a integridade do ativo. Se isso não for possível, deve haver uma decisão de segurança documentada para justificar isso para cada recurso.	3

V3.7 Outras Considerações de Segurança do Navegador

Esta seção inclui vários outros controles de segurança e recursos de segurança de navegadores modernos exigidos para a segurança do navegador do lado do cliente (client-side).

#	Descrição	Nível
3.7.1	Verifique se a aplicação usa apenas tecnologias do lado do cliente (client-side) que ainda são suportadas e consideradas seguras. Exemplos de tecnologias que não atendem a este requisito incluem plug-ins NSAPI, Flash, Shockwave, ActiveX, Silverlight, NACL ou miniaplicativos (applets) Java do lado do cliente.	2
3.7.2	Verifique se a aplicação redirecionará automaticamente o usuário para um hostname ou domínio diferente (que não seja controlado pela aplicação) apenas se o destino constar em uma lista de permissões (allowlist).	2
3.7.3	Verifique se a aplicação exibe uma notificação quando o usuário está sendo redirecionado para um URL fora do controle da aplicação, com a opção de cancelar a navegação.	3
3.7.4	Verifique se o domínio de nível superior da aplicação (ex., site.tld) é adicionado à lista pública de pré-carregamento (preload list) para o HTTP Strict Transport Security (HSTS). Isso garante que o uso de TLS para a aplicação seja integrado diretamente aos principais navegadores, em vez de depender apenas do campo de cabeçalho de resposta Strict-Transport-Security.	3

#	Descrição	Nível
3.7.5	Verifique se a aplicação se comporta conforme o documentado (como avisar o usuário ou bloquear o acesso) caso o navegador usado para acessar a aplicação não suporte os recursos de segurança esperados.	3

Referências

Para mais informações, veja também:

- Set-Cookie __Host- prefix details
- OWASP Content Security Policy Cheat Sheet
- OWASP Secure Headers Project
- OWASP Cross-Site Request Forgery Prevention Cheat Sheet
- HSTS Browser Preload List submission form
- OWASP DOM Clobbering Prevention Cheat Sheet

V4 API e Web Service

Objetivo de Controle

Diversas considerações se aplicam especificamente a aplicações que expõem APIs para uso por navegadores web ou outros consumidores (comumente usando JSON, XML ou GraphQL). Este capítulo abrange as configurações e os mecanismos de segurança relevantes que devem ser aplicados.

Note que as preocupações com autenticação, gerenciamento de sessão e validação de entrada de outros capítulos também se aplicam a APIs, portanto, este capítulo não pode ser tirado de contexto ou testado isoladamente.

V4.1 Segurança Genérica de Web Service

Esta seção aborda considerações gerais de segurança de web services e, consequentemente, práticas básicas de higiene de web services.

#	Descrição	Nível
4.1.1	Verifique se toda resposta HTTP com um corpo de mensagem contém um campo de cabeçalho Content-Type que corresponda ao conteúdo real da resposta, incluindo o parâmetro charset para especificar uma codificação de caracteres segura (ex., UTF-8, ISO-8859-1) de acordo com os Tipos de Mídia da IANA, como “text/”, “/+xml” e “/xml”.	1
4.1.2	Verifique se apenas endpoints voltados ao usuário (destinados ao acesso manual por navegador web) redirecionam automaticamente de HTTP para HTTPS, enquanto outros serviços ou endpoints não implementam redirecionamentos transparentes. Isso serve para evitar uma situação em que um cliente envia erroneamente requisições HTTP não criptografadas, mas como as requisições são redirecionadas automaticamente para HTTPS, o vazamento de dados sensíveis passa despercebido.	2
4.1.3	Verifique se qualquer campo de cabeçalho HTTP usado pela aplicação e definido por uma camada intermediária, como um balanceador de carga, um proxy web ou um serviço backend-for-frontend, não pode ser substituído (overridden) pelo usuário final. Exemplos de cabeçalhos podem incluir X-Real-IP, X-Forwarded-* ou X-User-ID.	2
4.1.4	Verifique se apenas os métodos HTTP explicitamente suportados pela aplicação ou por sua API (incluindo OPTIONS durante requisições preflight) podem ser usados e se os métodos não utilizados estão bloqueados.	3
4.1.5	Verifique se assinaturas digitais por mensagem são usadas para fornecer garantia adicional, além das proteções de transporte, para requisições ou transações que são altamente sensíveis ou que atravessam vários sistemas.	3

V4.2 Validação da Estrutura da Mensagem HTTP

Esta seção explica como a estrutura e os campos de cabeçalho de uma mensagem HTTP devem ser validados para evitar ataques como falsificação de requisição (request smuggling), divisão de resposta (response splitting), injeção de cabeçalho e negação de serviço por meio de mensagens HTTP excessivamente longas.

Esses requisitos são relevantes para o processamento e a geração geral de mensagens HTTP, mas são especialmente importantes ao converter mensagens HTTP entre diferentes versões do HTTP.

#	Descrição	Nível
4.2.1	Verifique se todos os componentes da aplicação (incluindo balanceadores de carga, firewalls e servidores de aplicação) determinam os limites das mensagens HTTP de entrada usando o mecanismo apropriado para a versão do HTTP, a fim de evitar falsificação de requisição HTTP (HTTP request smuggling). No HTTP/1.x, se um campo de cabeçalho Transfer-Encoding estiver presente, o cabeçalho Content-Length deve ser ignorado conforme a RFC 2616. Ao usar HTTP/2 ou HTTP/3, se um campo de cabeçalho Content-Length estiver presente, o receptor deve garantir que seja consistente com o comprimento dos frames DATA.	2
4.2.2	Verifique se, ao gerar mensagens HTTP, o campo de cabeçalho Content-Length não entra em conflito com o comprimento do conteúdo determinado pelo enquadramento (framing) do protocolo HTTP, a fim de evitar ataques de request smuggling.	3
4.2.3	Verifique se a aplicação não envia nem aceita mensagens HTTP/2 ou HTTP/3 com campos de cabeçalho específicos da conexão, como Transfer-Encoding, para evitar ataques de response splitting e injeção de cabeçalho.	3
4.2.4	Verifique se a aplicação aceita apenas requisições HTTP/2 e HTTP/3 onde os campos de cabeçalho e valores não contêm nenhuma sequência CR (\r), LF (\n) ou CRLF (\r\n), para evitar ataques de injeção de cabeçalho.	3
4.2.5	Verifique se, caso a aplicação (backend ou frontend) construa e envie requisições, ela usa validação, sanitização ou outros mecanismos para evitar a criação de URIs (como para chamadas de API) ou campos de cabeçalho de requisição HTTP (como Authorization ou Cookie) que sejam longos demais para serem aceitos pelo componente receptor. Isso poderia causar uma negação de serviço, como ao enviar uma requisição excessivamente longa (ex., um campo de cabeçalho de cookie longo), resultando no servidor sempre respondendo com um status de erro.	3

V4.3 GraphQL

O GraphQL está se tornando mais comum como uma forma de criar clientes ricos em dados que não são fortemente acoplados a uma variedade de serviços de backend. Esta seção aborda as considerações de segurança para o GraphQL.

#	Descrição	Nível
4.3.1	Verifique se uma lista de permissões (allowlist) de consultas, limitação de profundidade, limitação de quantidade ou análise de custo de consulta é usada para evitar Negação de Serviço (DoS) de expressão na camada de dados ou GraphQL como resultado de consultas aninhadas e custosas.	2
4.3.2	Verifique se as consultas de introspecção do GraphQL estão desabilitadas no ambiente de produção, a menos que a API do GraphQL seja destinada a ser usada por terceiros.	2

V4.4 WebSocket

O WebSocket é um protocolo de comunicações que fornece um canal de comunicação bidirecional simultâneo em uma única conexão TCP. Ele foi padronizado pela IETF como RFC 6455 em 2011 e é distinto do HTTP, embora seja projetado para funcionar nas portas HTTP 443 e 80.

Esta seção fornece os principais requisitos de segurança para prevenir ataques relacionados à segurança da comunicação e ao gerenciamento de sessão que exploram especificamente esse canal de comunicação em tempo real.

#	Descrição	Nível
4.4.1	Verifique se o WebSocket over TLS (WSS) é usado para todas as conexões WebSocket.	1
4.4.2	Verifique se, durante o handshake inicial do WebSocket HTTP, o campo de cabeçalho Origin é verificado em relação a uma lista de origens permitidas para a aplicação.	2
4.4.3	Verifique se, caso o gerenciamento de sessão padrão da aplicação não possa ser usado, tokens dedicados estão sendo usados para isso, os quais cumprem os requisitos de segurança relevantes de Gerenciamento de Sessão.	2
4.4.4	Verifique se os tokens dedicados de gerenciamento de sessão do WebSocket são inicialmente obtidos ou validados por meio da sessão HTTPS previamente autenticada ao fazer a transição de uma sessão HTTPS existente para um canal WebSocket.	2

Referências

Para mais informações, veja também:

- OWASP REST Security Cheat Sheet

- Recursos sobre Autorização GraphQL em graphql.org e Apollo.
- OWASP Web Security Testing Guide: GraphQL Testing
- OWASP Web Security Testing Guide: Testing WebSockets

V5 Manipulação de Arquivos

Objetivo de Controle

O uso de arquivos pode apresentar uma variedade de riscos à aplicação, incluindo negação de serviço, acesso não autorizado e esgotamento de armazenamento. Este capítulo inclui requisitos para abordar esses riscos.

V5.1 Documentação de Manipulação de Arquivos

Esta seção inclui um requisito para documentar as características esperadas dos arquivos aceitos pela aplicação, como uma pré-condição necessária para desenvolver e verificar verificações de segurança relevantes.

#	Descrição	Nível
5.1.1	Verifique se a documentação define os tipos de arquivos permitidos, as extensões de arquivo esperadas e o tamanho máximo (incluindo o tamanho descompactado) para cada recurso de upload. Além disso, certifique-se de que a documentação especifique como os arquivos são tornados seguros para que os usuários finais façam download e os processem, como, por exemplo, o comportamento da aplicação quando um arquivo malicioso é detectado.	2

V5.2 Upload de Arquivo e Conteúdo

A funcionalidade de upload de arquivo é uma fonte primária de arquivos não confiáveis. Esta seção descreve os requisitos para garantir que a presença, o volume ou o conteúdo desses arquivos não possam prejudicar a aplicação.

#	Descrição	Nível
5.2.1	Verifique se a aplicação aceitará apenas arquivos de um tamanho que possa processar sem causar perda de desempenho ou um ataque de negação de serviço.	1

#	Descrição	Nível
5.2.2	Verifique se, quando a aplicação aceita um arquivo, seja individualmente ou dentro de um arquivo compactado como um zip, ela verifica se a extensão do arquivo corresponde a uma extensão esperada e valida se o conteúdo corresponde ao tipo representado pela extensão. Isso inclui, mas não se limita a, verificar os 'magic bytes' (bytes mágicos) iniciais, realizar a reescrita de imagem e usar bibliotecas especializadas para validação de conteúdo de arquivo. Para L1, o foco pode ser apenas nos arquivos que são usados para tomar decisões específicas de negócios ou segurança. Para L2 e superior, isso deve se aplicar a todos os arquivos sendo aceitos.	1
5.2.3	Verifique se a aplicação verifica os arquivos compactados (ex., zip, gz, docx, odt) em relação ao tamanho máximo não compactado permitido e em relação ao número máximo de arquivos antes de descompactar o arquivo.	2
5.2.4	Verifique se uma cota de tamanho de arquivo e um número máximo de arquivos por usuário são aplicados para garantir que um único usuário não possa encher o armazenamento com muitos arquivos ou arquivos excessivamente grandes.	3
5.2.5	Verifique se a aplicação não permite o upload de arquivos compactados contendo links simbólicos (symlinks), a menos que isso seja especificamente exigido (nesse caso, será necessário impor uma lista de permissões dos arquivos que podem ser vinculados via symlink).	3
5.2.6	Verifique se a aplicação rejeita imagens enviadas com um tamanho de pixel maior que o máximo permitido, para evitar ataques de inundação de pixels (pixel flood).	3

V5.3 Armazenamento de Arquivos

Esta seção inclui requisitos para evitar que arquivos sejam executados indevidamente após o upload, para detectar conteúdo perigoso e para evitar que dados não confiáveis sejam usados para controlar onde os arquivos estão sendo armazenados.

#	Descrição	Nível
5.3.1	Verifique se os arquivos carregados ou gerados por entrada não confiável e armazenados em uma pasta pública não são executados como código de programa no lado do servidor quando acessados diretamente por uma requisição HTTP.	1

#	Descrição	Nível
5.3.2	Verifique se, quando a aplicação cria caminhos de arquivo para operações de arquivo, em vez de usar nomes de arquivo enviados pelo usuário, ela usa dados gerados internamente ou confiáveis. Se os nomes de arquivo enviados pelo usuário ou os metadados do arquivo precisarem ser usados, validação e sanitização estritas devem ser aplicadas. Isso visa proteger contra ataques de path traversal, inclusão de arquivo local ou remoto (LFI, RFI) e falsificação de solicitação do lado do servidor (SSRF).	1
5.3.3	Verifique se o processamento de arquivos no lado do servidor, como a descompactação de arquivos, ignora as informações de caminho fornecidas pelo usuário para evitar vulnerabilidades como zip slip.	3

V5.4 Download de Arquivos

Esta seção contém requisitos para mitigar riscos ao servir arquivos a serem baixados, incluindo ataques de path traversal e injeção. Isso também inclui certificar-se de que eles não contêm conteúdo perigoso.

#	Descrição	Nível
5.4.1	Verifique se a aplicação valida ou ignora os nomes de arquivo enviados pelo usuário, inclusive em um JSON, JSONP ou parâmetro de URL, e especifica um nome de arquivo no campo de cabeçalho Content-Disposition na resposta.	2
5.4.2	Verifique se os nomes de arquivos servidos (por exemplo, em campos de cabeçalho de resposta HTTP ou anexos de e-mail) são codificados ou sanitizados (por exemplo, seguindo a RFC 6266) para preservar a estrutura do documento e prevenir ataques de injeção.	2
5.4.3	Verifique se os arquivos obtidos de fontes não confiáveis são verificados por scanners antivírus para evitar a veiculação de conteúdo malicioso conhecido.	2

Referências

Para mais informações, veja também:

- OWASP File Upload Cheat Sheet
- Example of using symlinks for arbitrary file read
- Explanation of “Magic Bytes” from Wikipedia

V6 Autenticação

Objetivo de Controle

Autenticação é o processo de estabelecer ou confirmar a autenticidade de um indivíduo ou dispositivo. Envolve a verificação de reivindicações (claims) feitas por uma pessoa ou sobre um dispositivo, garantindo a resistência à falsificação de identidade (impersonation) e evitando a recuperação ou interceptação de senhas.

O NIST SP 800-63 é um padrão moderno, baseado em evidências, que é valioso para organizações em todo o mundo, mas é particularmente relevante para agências dos EUA e para aqueles que interagem com elas.

Embora muitos dos requisitos neste capítulo baseiem-se na segunda seção do padrão (conhecido como NIST SP 800-63B “Digital Identity Guidelines - Authentication and Lifecycle Management”), o capítulo foca em ameaças comuns e fraquezas de autenticação frequentemente exploradas. Ele não tenta cobrir de forma abrangente todos os pontos do padrão. Para casos em que a conformidade total com o NIST SP 800-63 for necessária, consulte o NIST SP 800-63.

Além disso, a terminologia do NIST SP 800-63 pode, às vezes, ser diferente, e este capítulo frequentemente usa uma terminologia mais comumente compreendida para melhorar a clareza.

Um recurso comum de aplicações mais avançadas é a capacidade de adaptar os estágios de autenticação exigidos com base em vários fatores de risco. Esse recurso é abordado no capítulo “Autorização”, uma vez que esses mecanismos também precisam ser considerados para decisões de autorização.

V6.1 Documentação de Autenticação

Esta seção contém requisitos que detalham a documentação de autenticação que deve ser mantida para uma aplicação. Isso é crucial para implementar e avaliar como os controles de autenticação relevantes devem ser configurados.

#	Descrição	Nível
6.1.1	Verifique se a documentação da aplicação define como os controles, como limitação de taxa (rate limiting), antiautomação e resposta adaptativa, são usados para se defender contra ataques como preenchimento de credenciais (credential stuffing) e força bruta de senha. A documentação deve deixar claro como esses controles são configurados e impedir o bloqueio malicioso de contas (account lockout).	1

#	Descrição	Nível
6.1.2	Verifique se uma lista de palavras específicas do contexto está documentada para impedir o seu uso em senhas. A lista pode incluir permutações de nomes da organização, nomes de produtos, identificadores de sistema, codinomes de projeto, nomes de departamentos ou funções, e similares.	2
6.1.3	Verifique se, caso a aplicação inclua vários caminhos de autenticação, todos estejam documentados juntamente com os controles de segurança e a força de autenticação que devem ser aplicados consistentemente em todos eles.	2

V6.2 Segurança de Senha

As senhas, chamadas de “Segredos Memorizados”(Memorized Secrets) pelo NIST SP 800-63, incluem senhas, frases secretas (passphrases), PINs, padrões de desbloqueio e escolher o gatinho correto ou outro elemento de imagem. Geralmente, são consideradas “algo que você sabe” e costumam ser usadas como um mecanismo de autenticação de fator único.

Como tal, esta seção contém requisitos para garantir que as senhas sejam criadas e tratadas de forma segura. A maioria dos requisitos é L1, pois eles são mais importantes nesse nível. Do L2 em diante, exigem-se mecanismos de autenticação multifator, onde as senhas podem ser um desses fatores.

Os requisitos nesta seção relacionam-se principalmente à § 5.1.1.2 do Guia do NIST.

#	Descrição	Nível
6.2.1	Verifique se as senhas definidas pelo usuário têm pelo menos 8 caracteres de comprimento, embora um mínimo de 15 caracteres seja fortemente recomendado.	1
6.2.2	Verifique se os usuários podem alterar sua senha.	1
6.2.3	Verifique se a funcionalidade de alteração de senha exige a senha atual e a nova senha do usuário.	1
6.2.4	Verifique se as senhas enviadas durante o registro da conta ou na alteração da senha são verificadas em um conjunto disponível de, pelo menos, as 3000 senhas mais comuns que correspondem à política de senha da aplicação, por exemplo, o comprimento mínimo.	1
6.2.5	Verifique se senhas de qualquer composição podem ser usadas, sem regras que limitem o tipo de caracteres permitidos. Não deve haver requisito para um número mínimo de letras maiúsculas ou minúsculas, números ou caracteres especiais.	1

#	Descrição	Nível
6.2.6	Verifique se os campos de entrada de senha usam type=password para mascarar a entrada. As aplicações podem permitir que o usuário visualize temporariamente toda a senha mascarada ou o último caractere digitado da senha.	1
6.2.7	Verifique se a funcionalidade de “colar”(paste), ajudantes de senha de navegador e gerenciadores de senha externos são permitidos.	1
6.2.8	Verifique se a aplicação verifica a senha do usuário exatamente como recebida dele, sem nenhuma modificação como truncamento ou transformação de maiúsculas e minúsculas (case transformation).	1
6.2.9	Verifique se senhas de pelo menos 64 caracteres são permitidas.	2
6.2.10	Verifique se a senha de um usuário permanece válida até que seja descoberta como comprometida ou o usuário a troque. A aplicação não deve exigir a rotação periódica de credenciais.	2
6.2.11	Verifique se a lista documentada de palavras específicas do contexto é usada para evitar que senhas fáceis de adivinhar sejam criadas.	2
6.2.12	Verifique se as senhas enviadas durante o registro da conta ou alterações de senha são verificadas em relação a um conjunto de senhas vazadas (breached passwords).	2

V6.3 Segurança Geral de Autenticação

Esta seção contém requisitos gerais para a segurança dos mecanismos de autenticação, além de estabelecer as diferentes expectativas para os níveis. Aplicações L2 devem forçar o uso de autenticação multifator (MFA). Aplicações L3 devem usar autenticação baseada em hardware, executada em um ambiente de execução atestado e confiável (TEE). Isso pode incluir passkeys vinculadas a dispositivos (device-bound), autenticadores aplicados com Nível de Garantia (LoA) Alto do eIDAS, autenticadores com nível de garantia NIST Authenticator Assurance Level 3 (AAL3) ou um mecanismo equivalente.

Embora esta seja uma postura relativamente agressiva em relação à MFA, é fundamental elevar o padrão em torno disso para proteger os usuários, e qualquer tentativa de relaxar esses requisitos deve ser acompanhada de um plano claro sobre como os riscos em torno da autenticação serão mitigados, levando em consideração as orientações e pesquisas do NIST sobre o tópico.

Observe que, no momento do lançamento, o NIST SP 800-63 considera o e-mail como um mecanismo de autenticação inaceitável (cópia arquivada).

Os requisitos nesta seção se relacionam a uma variedade de seções do Guia do NIST, incluindo: § 4.2.1, § 4.3.1, § 5.2.2 e § 6.1.2.

#	Descrição	Nível
6.3.1	Verifique se os controles para prevenir ataques, como preenchimento de credenciais e força bruta de senha, são implementados de acordo com a documentação de segurança da aplicação.	1
6.3.2	Verifique se contas de usuário padrão (ex., “root”, “admin” ou “sa”) não estão presentes na aplicação ou estão desativadas.	1
6.3.3	Verifique se um mecanismo de autenticação multifator ou uma combinação de mecanismos de autenticação de fator único deve ser usado para acessar a aplicação. Para o L3, um dos fatores deve ser um mecanismo de autenticação baseado em hardware que ofereça resistência contra comprometimento e falsificação (impersonation) contra ataques de phishing, enquanto verifica a intenção de autenticar por meio da exigência de uma ação iniciada pelo usuário (como o pressionamento de um botão em uma chave de hardware FIDO ou telefone celular). Relaxar qualquer uma das considerações neste requisito exige uma justificativa totalmente documentada e um conjunto abrangente de controles mitigadores.	2
6.3.4	Verifique se, caso a aplicação inclua vários caminhos de autenticação, não há caminhos não documentados e que os controles de segurança e a força de autenticação são impostos consistentemente.	2
6.3.5	Verifique se os usuários são notificados sobre tentativas suspeitas de autenticação (bem-sucedidas ou mal-sucedidas). Isso pode incluir tentativas de autenticação de um local ou cliente incomum, autenticação parcialmente bem-sucedida (apenas um dos múltiplos fatores), uma tentativa de autenticação após um longo período de inatividade ou uma autenticação bem-sucedida após várias tentativas malsucedidas.	3
6.3.6	Verifique se o e-mail não é usado como mecanismo de autenticação de fator único nem multifator.	3
6.3.7	Verifique se os usuários são notificados após atualizações de detalhes de autenticação, como redefinições de credenciais ou modificação do nome de usuário ou endereço de e-mail.	3
6.3.8	Verifique se usuários válidos não podem ser deduzidos de desafios de autenticação malsucedidos, como baseando-se em mensagens de erro, códigos de resposta HTTP ou diferentes tempos de resposta. A funcionalidade de registro e esquecimento de senha também deve ter essa proteção.	3

V6.4 Ciclo de Vida e Recuperação do Fator de Autenticação

Os fatores de autenticação podem incluir senhas, tokens de software, tokens de hardware e dispositivos biométricos. Lidar com o ciclo de vida desses mecanismos de forma segura é fundamental para a segurança de uma aplicação, e esta seção inclui requisitos relacionados a isso.

Os requisitos nesta seção relacionam-se principalmente à § 5.1.1.2 ou § 6.1.2.3 do Guia do NIST.

#	Descrição	Nível
6.4.1	Verifique se as senhas iniciais geradas pelo sistema ou códigos de ativação são gerados aleatoriamente de forma segura, seguem a política de senha existente e expiram após um curto período ou depois de serem usados inicialmente. Esses segredos iniciais não podem se tornar a senha de longo prazo.	1
6.4.2	Verifique se não estão presentes dicas de senha ou autenticação baseada em conhecimento (as chamadas “perguntas secretas”).	1
6.4.3	Verifique se um processo seguro para a redefinição de uma senha esquecida está implementado, o qual não ignora (bypass) nenhum mecanismo de autenticação multifator habilitado.	2
6.4.4	Verifique se, em caso de perda de um fator de autenticação multifator, a comprovação de identidade é realizada no mesmo nível que durante a inscrição (enrollment).	2
6.4.5	Verifique se as instruções de renovação para os mecanismos de autenticação que expiram são enviadas com tempo suficiente para serem realizadas antes que o antigo mecanismo expire, configurando lembretes automatizados se necessário.	3
6.4.6	Verifique se os usuários administrativos podem iniciar o processo de redefinição de senha para o usuário, mas que isso não permita que eles alterem ou escolham a senha do usuário. Isso evita uma situação em que eles conheçam a senha do usuário.	3

V6.5 Requisitos Gerais de Autenticação Multifator

Esta seção fornece orientações gerais que serão relevantes para vários métodos diferentes de autenticação multifator.

Os mecanismos incluem:

- Segredos de Consulta (Lookup Secrets)
- Senhas de Uso Único Baseadas em Tempo (TOTPs)

- Mecanismos Fora de Banda (Out-of-Band)

Os segredos de consulta são listas pré-geradas de códigos secretos, semelhantes a Números de Autorização de Transação (TAN), códigos de recuperação de mídia social ou uma grade contendo um conjunto de valores aleatórios. Este tipo de mecanismo de autenticação é considerado “algo que você possui” porque os códigos deliberadamente não são memoráveis, logo precisarão ser armazenados em algum lugar.

As Senhas de Uso Único Baseadas em Tempo (TOTPs) são tokens físicos ou de software que exibem um desafio pseudoaleatório de uso único em constante mudança. Este tipo de mecanismo de autenticação é considerado “algo que você possui”. TOTPs multifator são semelhantes aos TOTPs de fator único, mas requerem a inserção de um código PIN válido, desbloqueio biométrico, inserção de USB, emparelhamento NFC ou algum valor adicional (como calculadoras de assinatura de transação) para criar a Senha de Uso Único (OTP) final.

Detalhes sobre os mecanismos out-of-band serão fornecidos na próxima seção.

Os requisitos nestas seções relacionam-se principalmente à § 5.1.2, § 5.1.3, § 5.1.4.2, § 5.1.5.2, § 5.2.1 e § 5.2.3 do Guia do NIST.

#	Descrição	Nível
6.5.1	Verifique se os segredos de consulta, solicitações ou códigos de autenticação out-of-band e senhas de uso único baseadas em tempo (TOTPs) podem ser usados com sucesso apenas uma vez.	2
6.5.2	Verifique se, quando armazenados no backend da aplicação, os segredos de consulta com menos de 112 bits de entropia (19 caracteres alfanuméricos aleatórios ou 34 dígitos aleatórios) recebem hash usando um algoritmo aprovado de hash para armazenamento de senhas que incorpore um salt aleatório de 32 bits. Uma função de hash padrão pode ser usada se o segredo tiver 112 bits de entropia ou mais.	2
6.5.3	Verifique se os segredos de consulta, código de autenticação out-of-band e sementes de senha de uso único baseada em tempo são gerados usando um Gerador de Números Pseudoaleatórios Criptograficamente Seguro (CSPRNG) para evitar valores previsíveis.	2
6.5.4	Verifique se os segredos de consulta e os códigos de autenticação out-of-band têm um mínimo de 20 bits de entropia (normalmente 4 caracteres alfanuméricos aleatórios ou 6 dígitos aleatórios são suficientes).	2
6.5.5	Verifique se as solicitações de autenticação out-of-band, códigos ou tokens, bem como as senhas de uso único baseadas em tempo (TOTPs) têm uma vida útil (lifetime) definida. As solicitações out-of-band devem ter uma vida útil máxima de 10 minutos e as TOTPs uma vida útil máxima de 30 segundos.	2

#	Descrição	Nível
6.5.6	Verifique se qualquer fator de autenticação (incluindo dispositivos físicos) pode ser revogado em caso de roubo ou outra perda.	3
6.5.7	Verifique se os mecanismos de autenticação biométrica são usados apenas como fatores secundários em conjunto com algo que você possui ou algo que você sabe.	3
6.5.8	Verifique se as senhas de uso único baseadas em tempo (TOTPs) são checadas com base em uma fonte de tempo de um serviço confiável e não de um tempo não confiável ou fornecido pelo cliente.	3

V6.6 Mecanismos de Autenticação Fora de Banda (Out-of-Band)

Geralmente, isso envolve o servidor de autenticação comunicando-se com um dispositivo físico por meio de um canal secundário seguro. Por exemplo, enviando notificações por push para dispositivos móveis. Esse tipo de mecanismo de autenticação é considerado “algo que você possui”.

Mecanismos de autenticação out-of-band inseguros, como e-mail e VOIP, não são permitidos. A autenticação por PSTN e SMS é atualmente considerada um mecanismo de autenticação “restrito” pelo NIST e deve ser preterida em favor de Senhas de Uso Único Baseadas em Tempo (TOTPs), um mecanismo criptográfico, ou similar. A seção § 5.1.3.3 do NIST SP 800-63B recomenda tratar os riscos de troca de dispositivo, troca de SIM, portabilidade de número ou outro comportamento anormal, caso a autenticação telefônica ou SMS out-of-band deva absolutamente ser suportada. Embora esta seção do ASVS não exija isso como um requisito, não tomar essas precauções para um aplicativo L2 sensível ou um aplicativo L3 deve ser visto como um sinal de alerta vermelho significativo.

Observe que o NIST também forneceu recentemente orientações que desencorajam o uso de notificações push. Embora esta seção do ASVS não faça isso, é importante estar ciente dos riscos de “bombardeio de notificações push”(push bombing).

#	Descrição	Nível
6.6.1	Verifique se os mecanismos de autenticação usando a Rede Telefônica Pública Comutada (PSTN) para entregar Senhas de Uso Único (OTPs) via telefone ou SMS são oferecidos apenas quando o número de telefone foi validado previamente, se métodos alternativos e mais fortes (como Senhas de Uso Único Baseadas em Tempo) também são oferecidos e se o serviço fornece informações sobre seus riscos de segurança aos usuários. Para aplicações L3, telefone e SMS não devem estar disponíveis como opções.	2

#	Descrição	Nível
6.6.2	Verifique se as solicitações de autenticação out-of-band, códigos ou tokens estão vinculados à solicitação de autenticação original para a qual foram gerados e não são utilizáveis para uma solicitação anterior ou subsequente.	2
6.6.3	Verifique se um mecanismo de autenticação out-of-band baseado em código está protegido contra ataques de força bruta usando limitação de taxa (rate limiting). Considere também o uso de um código com pelo menos 64 bits de entropia.	2
6.6.4	Verifique se, quando notificações push são usadas para autenticação multifator, a limitação de taxa é usada para evitar ataques de bombardeio de push (push bombing). A correspondência de números (number matching) também pode mitigar esse risco.	3

V6.7 Mecanismos de Autenticação Criptográfica

Mecanismos de autenticação criptográfica incluem smart cards ou chaves FIDO, onde o usuário precisa conectar ou emparelhar o dispositivo criptográfico ao computador para concluir a autenticação. O servidor de autenticação enviará um nonce de desafio ao dispositivo ou software criptográfico, e o dispositivo ou software calcula uma resposta baseada em uma chave criptográfica armazenada de forma segura. Os requisitos nesta seção fornecem orientações específicas de implementação para esses mecanismos, sendo que a orientação sobre algoritmos criptográficos é abordada no capítulo “Criptografia”.

Quando chaves compartilhadas ou secretas são usadas para autenticação criptográfica, elas devem ser armazenadas usando os mesmos mecanismos de outros segredos do sistema, conforme documentado na seção “Gerenciamento de Segredos” no capítulo “Configuração”.

Os requisitos nesta seção relacionam-se principalmente à § 5.1.7.2 do Guia do NIST.

#	Descrição	Nível
6.7.1	Verifique se os certificados usados para verificar afirmações (assertions) de autenticação criptográfica são armazenados de forma que os proteja contra modificações.	3
6.7.2	Verifique se o nonce de desafio tem pelo menos 64 bits de comprimento, e é estatisticamente único ou único durante toda a vida útil do dispositivo criptográfico.	3

V6.8 Autenticação com um Provedor de Identidade (Identity Provider)

Provedores de Identidade (IdPs) oferecem identidade federada para os usuários. Os usuários frequentemente terão mais de uma identidade em vários IdPs, como uma identidade corporativa usando o Azure AD, Okta, Ping Identity ou Google, ou identidade de consumidor usando Facebook, Twitter, Google ou WeChat, para citar apenas algumas alternativas comuns. Esta lista não é um endosso a essas empresas ou serviços, mas simplesmente um encorajamento para que os desenvolvedores considerem a realidade de que muitos usuários possuem várias identidades estabelecidas. As organizações devem considerar a integração com identidades de usuários existentes, de acordo com o perfil de risco da força da comprovação de identidade do IdP. Por exemplo, é improvável que uma organização governamental aceite uma identidade de mídia social como login para sistemas sensíveis, já que é fácil criar identidades falsas ou descartáveis, enquanto uma empresa de jogos para dispositivos móveis pode precisar se integrar com grandes plataformas de mídia social para expandir sua base ativa de jogadores.

O uso seguro de provedores de identidade externos exige configuração e verificação cuidadosas para prevenir a falsificação de identidade ou asserções forjadas. Esta seção fornece requisitos para mitigar esses riscos.

#	Descrição	Nível
6.8.1	Verifique se, caso a aplicação suporte múltiplos provedores de identidade (IdPs), a identidade do usuário não pode ser falsificada (spoofed) por meio de outro provedor de identidade suportado (por exemplo, usando o mesmo identificador de usuário). A mitigação padrão seria a aplicação registrar e identificar o usuário usando uma combinação do ID do IdP (servindo como um namespace) e o ID do usuário no IdP.	2
6.8.2	Verifique se a presença e a integridade de assinaturas digitais em asserções de autenticação (por exemplo, em JWTs ou asserções SAML) são sempre validadas, rejeitando quaisquer asserções que não estejam assinadas ou possuam assinaturas inválidas.	2
6.8.3	Verifique se asserções SAML são processadas de maneira única e usadas apenas uma vez dentro do período de validade para prevenir ataques de repetição (replay attacks).	2

#	Descrição	Nível
6.8.4	Verifique se, caso uma aplicação use um Provedor de Identidade (IdP) separado e espere uma força de autenticação, métodos ou atualidade específicos para funções específicas, a aplicação verifique isso usando as informações retornadas pelo IdP. Por exemplo, se o OIDC for usado, isso pode ser obtido validando as claims (reivindicações) do ID Token, como 'acr', 'amr' e 'auth_time' (se presentes). Se o IdP não fornecer essas informações, a aplicação deve ter uma abordagem de contingência documentada que assume que o mecanismo de autenticação de força mínima foi usado (por exemplo, autenticação de fator único usando nome de usuário e senha).	2

Referências

Para mais informações, veja também:

- NIST SP 800-63 - Digital Identity Guidelines
- NIST SP 800-63B - Authentication and Lifecycle Management
- NIST SP 800-63 FAQ
- OWASP Web Security Testing Guide: Testing for Authentication
- OWASP Password Storage Cheat Sheet
- OWASP Forgot Password Cheat Sheet
- OWASP Choosing and Using Security Questions Cheat Sheet
- CISA Guidance on “Number Matching”
- Details on the FIDO Alliance

V7 Gerenciamento de Sessão

Objetivo de Controle

Os mecanismos de gerenciamento de sessão permitem que as aplicações correlacionem as interações do usuário e do dispositivo ao longo do tempo, mesmo ao usar protocolos de comunicação sem estado (como o HTTP). Aplicações modernas podem usar vários tokens de sessão com características e propósitos distintos. Um sistema de gerenciamento de sessão seguro é aquele que impede que invasores obtenham, utilizem ou, de outra forma, abusem da sessão de uma vítima. Aplicações que mantêm sessões devem garantir que os seguintes requisitos de alto nível de gerenciamento de sessão sejam atendidos:

- As sessões são exclusivas de cada indivíduo e não podem ser adivinhadas ou compartilhadas.
- As sessões são invalidadas quando não são mais necessárias e expiram durante períodos de inatividade.

Muitos dos requisitos neste capítulo estão relacionados aos controles selecionados do NIST SP 800-63 Digital Identity Guidelines, concentrando-se em ameaças comuns e fraquezas de autenticação frequentemente exploradas.

Observe que os requisitos para detalhes de implementação específicos de determinados mecanismos de gerenciamento de sessão podem ser encontrados em outras partes:

- Cookies HTTP são um mecanismo comum para proteger tokens de sessão. Os requisitos específicos de segurança para cookies podem ser encontrados no capítulo “Segurança de Frontend Web”(Web Frontend Security).
- Tokens autocontidos (Self-contained tokens) são frequentemente usados como uma forma de manter sessões. Requisitos de segurança específicos podem ser encontrados no capítulo “Tokens Autocontidos”(Self-contained Tokens).

V7.1 Documentação de Gerenciamento de Sessão

Não existe um padrão único que atenda a todas as aplicações. Portanto, não é viável definir fronteiras e limites universais que sirvam para todos os casos. Uma análise de risco com decisões de segurança documentadas relacionadas ao tratamento de sessões deve ser conduzida como um pré-requisito para implementação e teste. Isso garante que o sistema de gerenciamento de sessão seja adaptado aos requisitos específicos da aplicação.

Independentemente de ser escolhido um mecanismo de sessão stateful ou “stateless”(sem estado), a análise deve ser completa e documentada para demonstrar que a solução selecionada é capaz de satisfazer todos os requisitos de segurança relevantes. A interação com quaisquer mecanismos de Single Sign-on (SSO) em uso também deve ser considerada.

#	Descrição	Nível
7.1.1	Verifique se o tempo limite de inatividade da sessão do usuário e o tempo de vida (lifetime) máximo absoluto da sessão estão documentados, são apropriados em combinação com outros controles e se a documentação inclui justificativa para quaisquer desvios dos requisitos de reautenticação do NIST SP 800-63B.	2
7.1.2	Verifique se a documentação define quantas sessões concorrentes (paralelas) são permitidas para uma conta, bem como os comportamentos e as ações pretendidas a serem tomadas quando o número máximo de sessões ativas for alcançado.	2

#	Descrição	Nível
7.1.3	Verifique se todos os sistemas que criam e gerenciam sessões de usuário como parte de um ecossistema de gerenciamento de identidade federada (como sistemas SSO) estão documentados juntamente com os controles para coordenar tempos de vida das sessões, encerramento e quaisquer outras condições que exijam reautenticação.	2

V7.2 Segurança Fundamental do Gerenciamento de Sessão

Esta seção atende aos requisitos essenciais de sessões seguras verificando se os tokens de sessão são gerados e validados com segurança.

#	Descrição	Nível
7.2.1	Verifique se a aplicação realiza toda a verificação do token de sessão usando um serviço de backend confiável.	1
7.2.2	Verifique se a aplicação usa tokens autocontidos ou de referência gerados dinamicamente para o gerenciamento de sessão, ou seja, não usando chaves e segredos estáticos de API.	1
7.2.3	Verifique se, caso tokens de referência sejam usados para representar sessões de usuário, eles são únicos e gerados usando um gerador de números pseudoaleatórios criptograficamente seguro (CSPRNG) e possuem pelo menos 128 bits de entropia.	1
7.2.4	Verifique se a aplicação gera um novo token de sessão no momento da autenticação do usuário, incluindo a reautenticação, e encerra o token de sessão atual.	1

V7.3 Tempo Limite de Sessão (Session Timeout)

Mecanismos de timeout (tempo limite) de sessão servem para minimizar a janela de oportunidade para sequestro de sessão (session hijacking) e outras formas de abuso de sessão. Os timeouts devem satisfazer as decisões de segurança documentadas.

#	Descrição	Nível
7.3.1	Verifique se existe um tempo limite (timeout) de inatividade de modo que a reautenticação seja forçada de acordo com a análise de risco e as decisões de segurança documentadas.	2
7.3.2	Verifique se existe um tempo de vida (lifetime) máximo absoluto para a sessão, de forma que a reautenticação seja forçada de acordo com a análise de risco e as decisões de segurança documentadas.	2

V7.4 Encerramento da Sessão

O encerramento da sessão pode ser tratado pela própria aplicação ou pelo provedor de SSO, caso o provedor de SSO esteja lidando com o gerenciamento de sessão em vez da aplicação. Pode ser necessário decidir se o provedor de SSO está no escopo ao considerar os requisitos nesta seção, pois alguns deles podem ser controlados pelo provedor.

O encerramento da sessão deve resultar na exigência de reautenticação e ser eficaz em toda a aplicação, login federado (se presente) e quaisquer relying parties (partes confiáveis).

Para mecanismos de sessão stateful, o encerramento normalmente envolve a invalidação da sessão no backend. No caso de tokens autocontidos, medidas adicionais são necessárias para revogar ou bloquear esses tokens, pois de outra forma eles poderiam permanecer válidos até a expiração.

#	Descrição	Nível
7.4.1	Verifique se, quando o encerramento da sessão é acionado (como em caso de logout ou expiração), a aplicação proíbe qualquer uso adicional da sessão. Para tokens de referência ou sessões stateful, isso significa invalidar os dados da sessão no backend da aplicação. As aplicações que usam tokens autocontidos precisarão de uma solução como manter uma lista de tokens encerrados, proibir tokens produzidos antes de uma data e hora por usuário ou rotacionar a chave de assinatura por usuário.	1
7.4.2	Verifique se a aplicação encerra todas as sessões ativas quando uma conta de usuário é desativada ou excluída (como quando um funcionário deixa a empresa).	1
7.4.3	Verifique se a aplicação fornece a opção de encerrar todas as outras sessões ativas após uma mudança ou remoção bem-sucedida de qualquer fator de autenticação (incluindo alteração de senha via redefinição ou recuperação e, se presente, atualização de configurações de MFA).	2

#	Descrição	Nível
7.4.4	Verifique se todas as páginas que requerem autenticação têm um acesso fácil e visível à funcionalidade de logout.	2
7.4.5	Verifique se os administradores da aplicação conseguem encerrar sessões ativas para um usuário individual ou para todos os usuários.	2

V7.5 Defesas Contra o Abuso de Sessão

Esta seção fornece requisitos para mitigar o risco representado por sessões ativas que sejam sequestradas ou abusadas por meio de vetores que dependem da existência e das capacidades das sessões de usuário ativas. Por exemplo, usando a execução de conteúdo malicioso para forçar um navegador de vítima autenticado a executar uma ação utilizando a sessão da vítima.

Note que a orientação específica por nível no capítulo “Autenticação”(Authentication) deve ser levada em consideração ao avaliar os requisitos desta seção.

#	Descrição	Nível
7.5.1	Verifique se a aplicação exige uma reautenticação completa antes de permitir modificações em atributos sensíveis da conta que possam afetar a autenticação, como endereço de e-mail, número de telefone, configuração de MFA ou outras informações usadas na recuperação da conta.	2
7.5.2	Verifique se os usuários são capazes de visualizar e (tendo se autenticado novamente com pelo menos um fator) encerrar qualquer ou todas as sessões ativas no momento.	2
7.5.3	Verifique se a aplicação exige uma autenticação posterior com pelo menos um fator ou verificação secundária antes de executar transações ou operações altamente confidenciais.	3

V7.6 Reautenticação Federada

Esta seção se relaciona àqueles que escrevem o código da Relying Party (RP) ou do Provedor de Identidade (IdP). Esses requisitos são derivados do NIST SP 800-63C para Federação e Asserções.

#	Descrição	Nível
7.6.1	Verifique se o tempo de vida e o encerramento da sessão entre Relying Parties (RPs) e Provedores de Identidade (IdPs) se comportam conforme o documentado, exigindo a reautenticação conforme necessário, como, por exemplo, quando o tempo máximo entre os eventos de autenticação do IdP é atingido.	2
7.6.2	Verifique se a criação de uma sessão requer o consentimento do usuário ou uma ação explícita, prevenindo a criação de novas sessões na aplicação sem interação do usuário.	2

Referências

Para mais informações, veja também:

- OWASP Web Security Testing Guide: Session Management Testing
- OWASP Session Management Cheat Sheet

V8 Autorização

Objetivo de Controle

A autorização garante que o acesso seja concedido apenas aos consumidores permitidos (usuários, servidores e outros clientes). Para aplicar o Princípio do Menor Privilégio (Principle of Least Privilege - POLP), aplicações verificadas devem atender aos seguintes requisitos de alto nível:

- Documentar regras de autorização, incluindo fatores de tomada de decisão e contextos ambientais.
- Os consumidores devem ter acesso apenas aos recursos permitidos por seus direitos definidos.

V8.1 Documentação de Autorização

A documentação abrangente de autorização é essencial para garantir que as decisões de segurança sejam aplicadas de forma consistente, passíveis de auditoria e alinhadas com as políticas organizacionais. Isso reduz o risco de acesso não autorizado, tornando os requisitos de segurança claros e acionáveis para desenvolvedores, administradores e testadores.

#	Descrição	Nível
8.1.1	Verifique se a documentação de autorização define regras para restringir o acesso em nível de função e específico a dados com base nas permissões do consumidor e nos atributos do recurso.	1
8.1.2	Verifique se a documentação de autorização define regras para restrições de acesso em nível de campo (tanto para leitura quanto escrita) com base nas permissões do consumidor e atributos do recurso. Note que essas regras podem depender de outros valores de atributos do objeto de dados relevante, como estado ou status.	2
8.1.3	Verifique se a documentação da aplicação define os atributos ambientais e contextuais (incluindo, mas não se limitando a, hora do dia, localização do usuário, endereço IP ou dispositivo) que são usados na aplicação para tomar decisões de segurança, incluindo as pertinentes à autenticação e autorização.	3
8.1.4	Verifique se a documentação de autenticação e autorização define como fatores ambientais e contextuais são usados na tomada de decisões, além da autorização em nível de função, específica a dados e em nível de campo. Isso deve incluir os atributos avaliados, os limites de risco e as ações tomadas (ex., permitir, desafiar, negar, autenticação step-up).	3

V8.2 Design Geral de Autorização

A implementação de controles de autorização granulares no nível de função, dados e campos garante que os consumidores possam acessar apenas o que lhes foi explicitamente concedido.

#	Descrição	Nível
8.2.1	Verifique se a aplicação garante que o acesso em nível de função seja restrito a consumidores com permissões explícitas.	1
8.2.2	Verifique se a aplicação garante que o acesso específico a dados seja restrito a consumidores com permissões explícitas a itens de dados específicos para mitigar referência direta insegura a objeto (IDOR) e autorização em nível de objeto quebrada (BOLA).	1
8.2.3	Verifique se a aplicação garante que o acesso em nível de campo seja restrito a consumidores com permissões explícitas a campos específicos para mitigar autorização em nível de propriedade de objeto quebrada (BOPLA).	2

#	Descrição	Nível
8.2.4	Verifique se os controles de segurança adaptativos baseados nos atributos ambientais e contextuais do consumidor (como hora do dia, localização, endereço IP ou dispositivo) são implementados para decisões de autenticação e autorização, conforme definido na documentação da aplicação. Esses controles devem ser aplicados quando o consumidor tentar iniciar uma nova sessão e também durante uma sessão existente.	3

V8.3 Autorização em Nível de Operação

A aplicação imediata de mudanças de autorização na camada (tier) apropriada da arquitetura de uma aplicação é crucial para evitar ações não autorizadas, especialmente em ambientes dinâmicos.

#	Descrição	Nível
8.3.1	Verifique se a aplicação impõe as regras de autorização em uma camada de serviço confiável e não depende de controles que um consumidor não confiável possa manipular, como JavaScript no lado do cliente.	1
8.3.2	Verifique se as alterações aos valores sobre os quais as decisões de autorização são tomadas são aplicadas imediatamente. Nos casos onde as mudanças não podem ser aplicadas imediatamente (como quando se depende de dados em tokens autocontidos), deve haver controles mitigadores para alertar quando um consumidor realizar uma ação para a qual ele não está mais autorizado a fazer, e para reverter a mudança. Note que essa alternativa não mitigaria o vazamento de informações.	3
8.3.3	Verifique se o acesso a um objeto é baseado nas permissões do sujeito (ex., consumidor) originador, não nas permissões de qualquer intermediário ou serviço atuando em nome deles. Por exemplo, se um consumidor chama um web service usando um token autocontido para autenticação, e o serviço então solicita dados de um serviço diferente, o segundo serviço usará o token do consumidor, em vez de um token machine-to-machine do primeiro serviço, para tomar decisões de permissão.	3

V8.4 Outras Considerações sobre Autorização

Considerações adicionais de autorização, particularmente para interfaces administrativas e ambientes multilocatário (multi-tenant), ajudam a evitar acessos não autorizados.

#	Descrição	Nível
8.4.1	Verifique se aplicações multilocatário (multi-tenant) usam controles cruzados de inquilinos (cross-tenant) para garantir que as operações do consumidor nunca afetem inquilinos com os quais eles não têm permissão para interagir.	2
8.4.2	Verifique se o acesso a interfaces administrativas incorpora várias camadas de segurança, incluindo a verificação contínua da identidade do consumidor, avaliação da postura de segurança do dispositivo e análise de risco contextual, garantindo que a localização da rede ou endpoints confiáveis não sejam os únicos fatores para a autorização, embora possam reduzir a probabilidade de acesso não autorizado.	3

Referências

Para mais informações, veja também:

- OWASP Web Security Testing Guide: Authorization
- OWASP Authorization Cheat Sheet

V9 Tokens Autocontidos (Self-contained Tokens)

Objetivo de Controle

O conceito de um token autocontido (self-contained token) é mencionado no RFC 6749 original do OAuth 2.0 de 2012. Refere-se a um token contendo dados ou claims (reivindicações) nos quais um serviço receptor se baseará para tomar decisões de segurança. Isso deve ser diferenciado de um token simples contendo apenas um identificador, o qual um serviço receptor usa para pesquisar os dados localmente. Os exemplos mais comuns de tokens autocontidos são os JSON Web Tokens (JWTs) e as asserções SAML.

O uso de tokens autocontidos tornou-se muito difundido, mesmo fora do OAuth e OIDC. Ao mesmo tempo, a segurança desse mecanismo depende da capacidade de validar a integridade do token e de garantir que o token seja válido para um contexto particular. Existem muitas armadilhas com este processo, e este capítulo fornece detalhes específicos sobre os mecanismos que as aplicações devem ter para evitá-las.

V9.1 Origem e Integridade do Token

Esta seção inclui requisitos para garantir que o token tenha sido produzido por uma parte confiável e não tenha sido adulterado.

#	Descrição	Nível
9.1.1	Verifique se os tokens autocontidos são validados usando sua assinatura digital ou MAC para proteger contra adulteração antes de aceitar o conteúdo do token.	1
9.1.2	Verifique se apenas algoritmos em uma lista de permissões (allowlist) podem ser usados para criar e verificar tokens autocontidos para um determinado contexto. A lista de permissões deve incluir os algoritmos permitidos, idealmente apenas algoritmos simétricos ou assimétricos, e não deve incluir o algoritmo 'None'. Se ambos, simétricos e assimétricos, precisarem ser suportados, controles adicionais serão necessários para evitar a confusão de chaves (key confusion).	1
9.1.3	Verifique se o material criptográfico (key material) que é usado para validar tokens autocontidos vem de fontes confiáveis pré-configuradas para o emissor do token, evitando que atacantes especifiquem fontes e chaves não confiáveis. Para JWTs e outras estruturas JWS, cabeçalhos como 'jku', 'x5u' e 'jwk' devem ser validados contra uma lista de permissões de fontes confiáveis.	1

V9.2 Conteúdo do Token

Antes de tomar decisões de segurança baseadas no conteúdo de um token autocontido, é necessário validar se o token foi apresentado dentro de seu período de validade e se destina-se ao uso pelo serviço receptor e para o propósito para o qual foi apresentado. Isso ajuda a evitar o uso cruzado inseguro (cross-usage) entre diferentes serviços ou com diferentes tipos de token do mesmo emissor.

Requisitos específicos para OAuth e OIDC são abordados no capítulo dedicado.

#	Descrição	Nível
9.2.1	Verifique se, caso haja um intervalo de tempo de validade presente nos dados do token, o token e seu conteúdo sejam aceitos apenas se o tempo de verificação estiver dentro deste intervalo de tempo de validade. Por exemplo, para JWTs, as claims 'nbf' e 'exp' devem ser verificadas.	1
9.2.2	Verifique se o serviço que recebe um token valida que o token seja do tipo correto e tenha o propósito pretendido antes de aceitar os conteúdos do token. Por exemplo, apenas tokens de acesso (access tokens) podem ser aceitos para decisões de autorização e apenas ID Tokens podem ser usados para provar a autenticação do usuário.	2

#	Descrição	Nível
9.2.3	Verifique se o serviço apenas aceita tokens que são destinados ao uso com esse serviço (audiência). Para JWTs, isso pode ser alcançado validando a claim 'aud' em relação a uma lista de permissões definida no serviço.	2
9.2.4	Verifique se, caso um emissor de token use a mesma chave privada para emitir tokens para públicos diferentes, os tokens emitidos contenham uma restrição de audiência (audience restriction) que identifique unicamente as audiências pretendidas. Isso evitará que um token seja reutilizado com um público não intencional. Se o identificador de audiência for provisionado dinamicamente, o emissor do token deverá validar essas audiências para garantir que não resultem em falsificação de audiência (audience impersonation).	2

Referências

Para mais informações, veja também:

- OWASP JSON Web Token Cheat Sheet

V10 OAuth e OIDC

Objetivo de Controle

O OAuth2 (referido como OAuth neste capítulo) é um framework padrão da indústria para autorização delegada. Por exemplo, usando o OAuth, uma aplicação cliente pode obter acesso a APIs (recursos do servidor) em nome de um usuário, desde que o usuário tenha autorizado a aplicação cliente a fazer isso.

Por si só, o OAuth não é projetado para autenticação de usuários. O framework OpenID Connect (OIDC) estende o OAuth adicionando uma camada de identidade de usuário sobre o OAuth. O OIDC fornece suporte para recursos que incluem informações de usuário padronizadas, Single Sign-On (SSO) e gerenciamento de sessão. Como o OIDC é uma extensão do OAuth, os requisitos de OAuth neste capítulo também se aplicam ao OIDC.

Os seguintes papéis (roles) são definidos no OAuth:

- O cliente OAuth é a aplicação que tenta obter acesso aos recursos do servidor (ex., chamando uma API usando o token de acesso emitido). O cliente OAuth geralmente é uma aplicação do lado do servidor (server-side).

- Um cliente confidencial é um cliente capaz de manter a confidencialidade das credenciais que usa para se autenticar no servidor de autorização.
 - Um cliente público não é capaz de manter a confidencialidade das credenciais para autenticação no servidor de autorização. Portanto, em vez de se autenticar (ex., usando parâmetros 'client_id' e 'client_secret'), ele apenas se identifica (usando um parâmetro 'client_id').
- O servidor de recursos OAuth (Resource Server - RS) é a API do servidor que expõe os recursos aos clientes OAuth.
 - O servidor de autorização OAuth (Authorization Server - AS) é uma aplicação de servidor que emite tokens de acesso aos clientes OAuth. Esses tokens de acesso permitem que os clientes OAuth acessem os recursos do RS, seja em nome de um usuário final ou em nome próprio do cliente OAuth. O AS é frequentemente uma aplicação separada, mas (se apropriado) pode ser integrado a um RS adequado.
 - O proprietário do recurso (Resource Owner - RO) é o usuário final que autoriza os clientes OAuth a obter acesso limitado aos recursos hospedados no servidor de recursos em seu nome. O proprietário do recurso consente com esta autorização delegada interagindo com o servidor de autorização.

Os seguintes papéis são definidos no OIDC:

- A Relying Party (RP - Parte Confiável) é a aplicação cliente que solicita a autenticação do usuário final através do Provedor OpenID. Ela assume o papel de um cliente OAuth.
- O Provedor OpenID (OpenID Provider - OP) é um AS do OAuth que é capaz de autenticar o usuário final e fornecer as claims (reivindicações) OIDC a uma RP. O OP pode ser o provedor de identidade (IdP), mas em cenários federados, o OP e o provedor de identidade (onde o usuário final se autentica) podem ser aplicações de servidor diferentes.

O OAuth e o OIDC foram inicialmente projetados para aplicações de terceiros. Hoje, eles são frequentemente usados por aplicações próprias (first-party) também. No entanto, quando usados em cenários first-party, como autenticação e gerenciamento de sessão, o protocolo adiciona certa complexidade, o que pode introduzir novos desafios de segurança.

O OAuth e o OIDC podem ser usados para muitos tipos de aplicações, mas o foco para o ASVS e os requisitos neste capítulo está em aplicações web e APIs.

Como o OAuth e o OIDC podem ser considerados uma lógica acima das tecnologias web, os requisitos gerais de outros capítulos sempre se aplicam, e este capítulo não pode ser tirado de contexto.

Este capítulo aborda as melhores práticas atuais para OAuth2 e OIDC, alinhadas com as especificações encontradas em <https://oauth.net/2/> e <https://openid.net/developers/specs/>. Mesmo que as RFCs sejam consideradas maduras, elas são atualizadas com frequência. Portanto, é importante alinhar-se com as versões mais recentes ao aplicar os requisitos neste capítulo. Veja a seção de referências para mais detalhes.

Dada a complexidade da área, é de vital importância para uma solução segura de OAuth ou OIDC o

uso de servidores de autorização conhecidos e padrão da indústria, além de aplicar a configuração de segurança recomendada.

A terminologia usada neste capítulo se alinha com as RFCs do OAuth e as especificações do OIDC, mas note que a terminologia do OIDC é usada apenas para requisitos específicos do OIDC; caso contrário, a terminologia do OAuth é usada.

No contexto do OAuth e OIDC, o termo “token” neste capítulo refere-se a:

- Tokens de acesso (Access tokens), que devem ser consumidos apenas pelo RS e podem ser tokens de referência validados via introspecção (introspection) ou tokens autocontidos (self-contained tokens) validados usando algum material criptográfico.
- Tokens de atualização (Refresh tokens), que devem ser consumidos apenas pelo servidor de autorização que emitiu o token.
- Tokens de ID OIDC (ID Tokens), que devem ser consumidos apenas pelo cliente que iniciou o fluxo de autorização.

Os níveis de risco para alguns dos requisitos neste capítulo dependem de o cliente ser um cliente confidencial ou considerado um cliente público. Uma vez que o uso de autenticação forte de cliente mitiga muitos vetores de ataque, alguns requisitos podem ser relaxados ao usar um cliente confidencial para aplicações de Nível 1 (L1).

V10.1 Segurança Genérica de OAuth e OIDC

Esta seção cobre os requisitos arquitetônicos genéricos que se aplicam a todas as aplicações usando OAuth ou OIDC.

#	Descrição	Nível
10.1.1	Verifique se os tokens são enviados apenas aos componentes que estritamente necessitam deles. Por exemplo, ao usar o padrão backend-for-frontend para aplicações JavaScript baseadas no navegador, os tokens de acesso e de atualização devem estar acessíveis apenas para o backend.	2
10.1.2	Verifique se o cliente apenas aceita valores do servidor de autorização (como o authorization code ou ID Token) se esses valores resultarem de um fluxo de autorização que foi iniciado pela mesma sessão do user agent e transação. Isso exige que segredos gerados pelo cliente, como o ‘code_verifier’ do proof key for code exchange (PKCE), o ‘state’ ou o ‘nonce’ do OIDC, não sejam adivinháveis, sejam específicos para a transação e estejam vinculados de forma segura tanto ao cliente quanto à sessão do user agent na qual a transação foi iniciada.	2

V10.2 Cliente OAuth

Estes requisitos detalham as responsabilidades das aplicações cliente OAuth. O cliente pode ser, por exemplo, um backend de servidor web (muitas vezes atuando como um Backend For Frontend, BFF), uma integração de serviço backend ou um Frontend Single Page Application (SPA, também conhecida como aplicação baseada em navegador).

Em geral, os clientes de backend são considerados clientes confidenciais e os clientes de frontend são considerados clientes públicos. No entanto, as aplicações nativas em execução no dispositivo do usuário final podem ser consideradas confidenciais ao usar o registro dinâmico de cliente (dynamic client registration) do OAuth.

#	Descrição	Nível
10.2.1	Verifique se, caso o code flow seja usado, o cliente OAuth tem proteção contra ataques de falsificação de requisição baseados em navegador, comumente conhecidos como falsificação de requisição entre sites (Cross-Site Request Forgery - CSRF), que disparam solicitações de token, seja usando a funcionalidade proof key for code exchange (PKCE) ou verificando o parâmetro 'state' que foi enviado na solicitação de autorização.	2
10.2.2	Verifique se, caso o cliente OAuth possa interagir com mais de um servidor de autorização, ele possui uma defesa contra ataques de confusão (mix-up attacks). Por exemplo, ele pode exigir que o servidor de autorização retorne o valor do parâmetro 'iss' e validá-lo na resposta de autorização e na resposta de token.	2
10.2.3	Verifique se o cliente OAuth solicita apenas os escopos (ou outros parâmetros de autorização) necessários nas solicitações ao servidor de autorização.	3

V10.3 Servidor de Recursos OAuth

No contexto do ASVS e deste capítulo, o servidor de recursos (resource server) é uma API. Para fornecer acesso seguro, o servidor de recursos deve:

- Validar o token de acesso, de acordo com o formato do token e as especificações relevantes do protocolo, por exemplo, validação de JWT ou introspecção de token OAuth.
- Se for válido, aplicar decisões de autorização com base nas informações do token de acesso e nas permissões que foram concedidas. Por exemplo, o servidor de recursos precisa verificar se o cliente (agindo em nome do RO) está autorizado a acessar o recurso solicitado.

Portanto, os requisitos listados aqui são específicos de OAuth ou OIDC e devem ser executados após a validação do token e antes de realizar a autorização baseada em informações contidas no token.

#	Descrição	Nível
10.3.1	Verifique se o servidor de recursos aceita apenas tokens de acesso que são destinados para uso com aquele serviço (audiência). A audiência (audience) pode ser incluída em um token de acesso estruturado (como a claim 'aud' no JWT), ou pode ser verificada usando o endpoint de introspecção do token.	2
10.3.2	Verifique se o servidor de recursos impõe decisões de autorização baseadas em claims do token de acesso que definem a autorização delegada. Se claims como 'sub', 'scope' e 'authorization_details' estiverem presentes, eles devem fazer parte da decisão.	2
10.3.3	Verifique se, caso uma decisão de controle de acesso exija a identificação de um usuário único a partir de um token de acesso (JWT ou resposta de introspecção de token relacionada), o servidor de recursos identifica o usuário a partir de claims que não possam ser reatribuídas a outros usuários. Normalmente, isso significa usar uma combinação das claims 'iss' e 'sub'.	2
10.3.4	Verifique se, caso o servidor de recursos exija força, métodos ou atualidade de autenticação específicos, ele verifica se o token de acesso apresentado satisfaz essas restrições. Por exemplo, se presentes, usando as claims OIDC 'acr', 'amr' e 'auth_time', respectivamente.	2
10.3.5	Verifique se o servidor de recursos impede o uso de tokens de acesso roubados ou a repetição (replay) de tokens de acesso (de partes não autorizadas) exigindo tokens de acesso restritos ao remetente (sender-constrained access tokens), seja via Mutual TLS para OAuth 2 ou OAuth 2 Demonstration of Proof of Possession (DPoP).	3

V10.4 Servidor de Autorização OAuth

Estes requisitos detalham as responsabilidades dos servidores de autorização OAuth, incluindo os Provedores OpenID.

Para autenticação do cliente, o método 'self_signed_tls_client_auth' é permitido de acordo com os pré-requisitos exigidos pela seção 2.2 da RFC 8705.

#	Descrição	Nível
10.4.1	Verifique se o servidor de autorização valida as URIs de redirecionamento (redirect URIs) com base em uma lista de permissões específica do cliente contendo URIs pré-registradas, usando comparação exata de strings.	1

#	Descrição	Nível
10.4.2	Verifique se, caso o servidor de autorização retorne o código de autorização (authorization code) na resposta de autorização, este possa ser usado apenas uma vez para uma solicitação de token. Para a segunda solicitação válida com um código de autorização que já foi usado para emitir um token de acesso, o servidor de autorização deve rejeitar a solicitação de token e revogar quaisquer tokens emitidos relacionados ao código de autorização.	1
10.4.3	Verifique se o código de autorização tem vida útil curta. A vida útil máxima pode ser de até 10 minutos para aplicações L1 e L2 e de até 1 minuto para aplicações L3.	1
10.4.4	Verifique se, para um determinado cliente, o servidor de autorização permite apenas o uso de tipos de concessão (grants) que este cliente precisa utilizar. Note que os grants 'token'(Implicit flow) e 'password'(Resource Owner Password Credentials flow) não devem mais ser usados.	1
10.4.5	Verifique se o servidor de autorização mitiga os ataques de replay de token de atualização (refresh token) para clientes públicos, preferencialmente usando tokens de atualização restritos ao remetente (sender-constrained), ou seja, Demonstrating Proof of Possession (DPoP) ou Tokens de Acesso Vinculados a Certificado usando mutual TLS (mTLS). Para aplicações L1 e L2, a rotação de token de atualização (refresh token rotation) pode ser usada. Se a rotação de token de atualização for usada, o servidor de autorização deve invalidar o token de atualização após o uso e revogar todos os tokens de atualização para aquela autorização se um token de atualização já usado e invalidado for fornecido.	1
10.4.6	Verifique se, caso o grant do tipo 'code' seja usado, o servidor de autorização mitiga os ataques de interceptação do código de autorização exigindo o proof key for code exchange (PKCE). Para solicitações de autorização, o servidor de autorização deve exigir um valor 'code_challenge' válido e não deve aceitar o valor 'code_challenge_method' como 'plain'. Para uma solicitação de token, ele deve exigir a validação do parâmetro 'code_verifier'.	2
10.4.7	Verifique se, caso o servidor de autorização suporte registro dinâmico de cliente não autenticado (unauthenticated dynamic client registration), ele mitiga o risco de aplicações clientes maliciosas. Ele deve validar os metadados do cliente, como quaisquer URIs registradas, garantir o consentimento do usuário e avisar o usuário antes de processar uma solicitação de autorização com uma aplicação cliente não confiável.	2

#	Descrição	Nível
10.4.8	Verifique se os tokens de atualização possuem uma expiração absoluta, inclusive se a expiração contínua (sliding expiration) de token de atualização for aplicada.	2
10.4.9	Verifique se os tokens de atualização e os tokens de acesso de referência podem ser revogados por um usuário autorizado usando a interface do usuário do servidor de autorização, para mitigar o risco de clientes maliciosos ou tokens roubados.	2
10.4.10	Verifique se o cliente confidencial é autenticado para as solicitações backchannel entre o cliente e o servidor de autorização, como solicitações de token, solicitações de autorização enviadas por push (PAR) e solicitações de revogação de token.	2
10.4.11	Verifique se a configuração do servidor de autorização atribui apenas os escopos (scopes) necessários ao cliente OAuth.	2
10.4.12	Verifique se, para um determinado cliente, o servidor de autorização permite apenas o valor de 'response_mode' que este cliente precisa utilizar. Por exemplo, fazendo com que o servidor de autorização valide esse valor em relação aos valores esperados ou usando solicitações de autorização enviadas por push (PAR) ou JWT-secured Authorization Request (JAR).	3
10.4.13	Verifique se o grant type 'code' é sempre usado em conjunto com as pushed authorization requests (PAR).	3
10.4.14	Verifique se o servidor de autorização emite apenas tokens de acesso restritos ao remetente (sender-constrained - Proof-of-Possession), seja com tokens de acesso vinculados a certificados usando mutual TLS (mTLS) ou tokens de acesso vinculados a DPOP (Demonstration of Proof of Possession).	3
10.4.15	Verifique se, para um cliente no lado do servidor (server-side, que não é executado no dispositivo do usuário final), o servidor de autorização garante que o valor do parâmetro 'authorization_details' provenha do backend do cliente e que o usuário não o tenha adulterado. Por exemplo, exigindo o uso de uma pushed authorization request (PAR) ou JWT-secured Authorization Request (JAR).	3
10.4.16	Verifique se o cliente é confidencial e o servidor de autorização exige o uso de métodos de autenticação de cliente fortes (baseados em criptografia de chave pública e resistentes a ataques de replay), como mutual TLS ('tls_client_auth', 'self_signed_tls_client_auth') ou JWT de chave privada ('private_key_jwt').	3

V10.5 Cliente OIDC

Como a Relying Party (Parte Confiável) do OIDC age como um cliente OAuth, os requisitos da seção “Cliente OAuth” também se aplicam.

Note que a seção “Autenticação com um Provedor de Identidade” no capítulo “Autenticação” também contém requisitos gerais relevantes.

#	Descrição	Nível
10.5.1	Verifique se o cliente (como Relying Party) mitiga os ataques de replay do ID Token. Por exemplo, garantindo que a claim ‘nonce’ no ID Token corresponda ao valor ‘nonce’ enviado na solicitação de autenticação para o Provedor OpenID (no OAuth2 referido como a solicitação de autorização enviada ao servidor de autorização).	2
10.5.2	Verifique se o cliente identifica o usuário de forma única a partir das claims do ID Token, geralmente a claim ‘sub’, a qual não pode ser reatribuída a outros usuários (no escopo de um provedor de identidade).	2
10.5.3	Verifique se o cliente rejeita as tentativas de um servidor de autorização malicioso de se passar por outro servidor de autorização através de metadados do servidor de autorização. O cliente deve rejeitar os metadados do servidor de autorização se o URL do emissor (issuer) nos metadados do servidor de autorização não corresponder exatamente ao URL do emissor pré-configurado esperado pelo cliente.	2
10.5.4	Verifique se o cliente valida se o ID Token tem a intenção de ser usado para aquele cliente (audiência), verificando se a claim ‘aud’ do token é igual ao valor ‘client_id’ para o cliente.	2
10.5.5	Verifique se, ao usar o logout back-channel do OIDC, a Relying Party mitiga a negação de serviço através de logout forçado e confusão cruzada de JWT (cross-JWT confusion) no fluxo de logout. O cliente deve verificar se o token de logout está digitado corretamente com um valor de ‘logout+jwt’, se contém a claim ‘event’ com o nome de membro correto e se não contém uma claim ‘nonce’. Note que também é recomendado ter uma expiração curta (por exemplo, 2 minutos).	2

V10.6 Provedor OpenID

Como os Provedores OpenID agem como servidores de autorização OAuth, os requisitos da seção “Servidor de Autorização OAuth” também se aplicam.

Note que, se estiver usando o fluxo ID Token (não o fluxo de código), nenhum token de acesso é emitido e muitos dos requisitos para OAuth AS não são aplicáveis.

#	Descrição	Nível
10.6.1	Verifique se o Provedor OpenID permite apenas os valores 'code', 'ciba', 'id_token' ou 'id_token code' para o modo de resposta (response mode). Note que 'code' é preferível a 'id_token code' (o fluxo Híbrido OIDC) e 'token' (qualquer fluxo Implícito) não deve ser usado.	2
10.6.2	Verifique se o Provedor OpenID mitiga a negação de serviço através de logout forçado. Obtendo a confirmação explícita do usuário final ou, se presente, validando os parâmetros na solicitação de logout (iniciada pela Relying Party), como o 'id_token_hint'.	2

V10.7 Gerenciamento de Consentimento

Estes requisitos abrangem a verificação do consentimento do usuário pelo servidor de autorização. Sem a verificação adequada do consentimento do usuário, um ator malicioso pode obter permissões em nome do usuário através de falsificação ou engenharia social.

#	Descrição	Nível
10.7.1	Verifique se o servidor de autorização garante que o usuário consente com cada solicitação de autorização. Se a identidade do cliente não puder ser assegurada, o servidor de autorização deve sempre solicitar explicitamente o consentimento do usuário.	2
10.7.2	Verifique se, quando o servidor de autorização solicita o consentimento do usuário, ele apresenta informações suficientes e claras sobre ao que se está consentindo. Quando aplicável, isso deve incluir a natureza das autorizações solicitadas (geralmente com base no escopo, servidor de recursos, detalhes de autorização do Rich Authorization Requests - RAR), a identidade da aplicação autorizada e a vida útil dessas autorizações.	2
10.7.3	Verifique se o usuário pode revisar, modificar e revogar os consentimentos que o usuário concedeu através do servidor de autorização.	2

Referências

Para obter mais informações sobre o OAuth, consulte:

- oauth.net

- OWASP OAuth 2.0 Protocol Cheat Sheet

Para requisitos relacionados ao OAuth no ASVS, são utilizadas as seguintes RFCs, publicadas e em status de rascunho (draft):

- RFC6749 The OAuth 2.0 Authorization Framework
- RFC6750 The OAuth 2.0 Authorization Framework: Bearer Token Usage
- RFC6819 OAuth 2.0 Threat Model and Security Considerations
- RFC7636 Proof Key for Code Exchange by OAuth Public Clients
- RFC7591 OAuth 2.0 Dynamic Client Registration Protocol
- RFC8628 OAuth 2.0 Device Authorization Grant
- RFC8707 Resource Indicators for OAuth 2.0
- RFC9068 JSON Web Token (JWT) Profile for OAuth 2.0 Access Tokens
- RFC9126 OAuth 2.0 Pushed Authorization Requests
- RFC9207 OAuth 2.0 Authorization Server Issuer Identification
- RFC9396 OAuth 2.0 Rich Authorization Requests
- RFC9449 OAuth 2.0 Demonstrating Proof of Possession (DPoP)
- RFC9700 Best Current Practice for OAuth 2.0 Security
- draft OAuth 2.0 for Browser-Based Applications
- draft The OAuth 2.1 Authorization Framework

Para obter mais informações sobre o OpenID Connect, consulte:

- OpenID Connect Core 1.0
- FAPI 2.0 Security Profile

V11 Criptografia

Objetivo de Controle

O objetivo deste capítulo é definir as melhores práticas para o uso geral da criptografia, bem como incutir uma compreensão fundamental dos princípios criptográficos e inspirar uma mudança em direção a abordagens mais resilientes e modernas. Ele incentiva o seguinte:

- Implementar sistemas criptográficos robustos que falhem de forma segura (fail securely), que se adaptem às ameaças em evolução e que sejam preparados para o futuro (future-proof).
- Utilizar mecanismos criptográficos que sejam ao mesmo tempo seguros e alinhados com as melhores práticas do setor.
- Manter um sistema seguro de gerenciamento de chaves criptográficas com controles de acesso e auditoria apropriados.
- Avaliar regularmente o cenário criptográfico para verificar novos riscos e adaptar algoritmos de acordo.

- Descobrir e gerenciar os casos de uso de criptografia ao longo de todo o ciclo de vida da aplicação para garantir que todos os ativos criptográficos sejam contabilizados e protegidos.

Além de delinear princípios gerais e melhores práticas, este documento também fornece informações técnicas mais detalhadas sobre os requisitos no Apêndice C - Padrões de Criptografia. Isso inclui algoritmos e modos que são considerados “aprovados” para os propósitos dos requisitos deste capítulo.

Requisitos que usam criptografia para resolver um problema separado, como gerenciamento de segredos ou segurança de comunicações, estarão em partes diferentes do padrão.

V11.1 Inventário e Documentação Criptográfica

As aplicações precisam ser projetadas com uma forte arquitetura criptográfica para proteger os ativos de dados de acordo com sua classificação. Criptografar tudo é um desperdício; não criptografar nada é uma negligência legal. Um equilíbrio deve ser alcançado, geralmente durante o design da arquitetura ou o design de alto nível (high-level design), design sprints ou architectural spikes. Projetar a criptografia “no improviso” ou fazer adequações retroativas (retrofitting) custará inevitavelmente muito mais para implementar de forma segura do que construí-la desde o início.

É importante garantir que todos os ativos criptográficos sejam descobertos, inventariados e avaliados regularmente. Por favor, consulte o apêndice para mais informações sobre como isso pode ser feito.

A necessidade de proteger sistemas criptográficos no futuro contra o eventual surgimento da computação quântica também é fundamental. A Criptografia Pós-Quântica (Post-Quantum Cryptography - PQC) refere-se a algoritmos criptográficos projetados para permanecerem seguros contra ataques por computadores quânticos, que têm a expectativa de quebrar os algoritmos amplamente usados, como o RSA e a Criptografia de Curva Elíptica (ECC).

Por favor, consulte o apêndice para orientações atuais sobre primitivas PQC validadas e padrões.

#	Descrição	Nível
11.1.1	Verifique se há uma política documentada para o gerenciamento de chaves criptográficas e um ciclo de vida da chave criptográfica que siga um padrão de gerenciamento de chaves como o NIST SP 800-57. Isso deve incluir a garantia de que as chaves não sejam supercompartilhadas (overshared) (por exemplo, com mais de duas entidades para segredos compartilhados e mais de uma entidade para chaves privadas).	2

#	Descrição	Nível
11.1.2	Verifique se um inventário criptográfico é realizado, mantido, atualizado regularmente e se inclui todas as chaves criptográficas, algoritmos e certificados usados pela aplicação. Ele também deve documentar onde as chaves podem ou não ser usadas no sistema e os tipos de dados que podem ou não ser protegidos usando as chaves.	2
11.1.3	Verifique se mecanismos de descoberta criptográfica são empregados para identificar todas as instâncias de criptografia no sistema, incluindo operações de criptografia, hashing e assinatura.	3
11.1.4	Verifique se um inventário criptográfico é mantido. Ele deve incluir um plano documentado que descreve o caminho de migração para os novos padrões de criptografia, como a criptografia pós-quântica, a fim de reagir a ameaças futuras.	3

V11.2 Implementação Segura de Criptografia

Esta seção define os requisitos para a seleção, implementação e gerenciamento contínuo dos principais algoritmos criptográficos de uma aplicação. O objetivo é garantir que apenas primitivas criptográficas robustas e aceitas no setor sejam implantadas, em alinhamento com as normas vigentes (por exemplo, NIST, ISO/IEC) e melhores práticas. As organizações devem garantir que cada componente criptográfico seja selecionado com base em evidências revisadas por pares (peer-reviewed) e testes de segurança práticos.

#	Descrição	Nível
11.2.1	Verifique se implementações validadas pela indústria (incluindo bibliotecas e implementações aceleradas por hardware) são usadas para operações criptográficas.	2
11.2.2	Verifique se a aplicação é projetada com agilidade criptográfica (crypto agility), de tal forma que números aleatórios, criptografia autenticada, MAC ou algoritmos de hash, comprimentos de chave, rodadas (rounds), cifras e modos possam ser reconfigurados, atualizados ou trocados a qualquer momento, para proteger contra quebras criptográficas. Da mesma forma, também deve ser possível substituir chaves e senhas e criptografar novamente os dados. Isso permitirá a realização de upgrades transparentes (seamless) para a criptografia pós-quântica (PQC), assim que implementações de alta garantia (high-assurance) de esquemas ou padrões aprovados de PQC estiverem amplamente disponíveis.	2

#	Descrição	Nível
11.2.3	Verifique se todas as primitivas criptográficas utilizam um mínimo de 128 bits de segurança com base no algoritmo, no tamanho da chave e na configuração. Por exemplo, uma chave ECC de 256 bits fornece cerca de 128 bits de segurança, ao passo que o RSA requer uma chave de 3072 bits para alcançar 128 bits de segurança.	2
11.2.4	Verifique se todas as operações criptográficas têm tempo constante (constant-time), sem operações de ‘curto-circuito’ em comparações, cálculos ou retornos, para evitar o vazamento de informações.	3
11.2.5	Verifique se todos os módulos criptográficos falham com segurança (fail securely), e se os erros são tratados de maneira a não permitir o surgimento de vulnerabilidades, como ataques de Padding Oracle.	3

V11.3 Algoritmos de Criptografia

Os algoritmos de criptografia autenticada construídos em AES e CHACHA20 formam a espinha dorsal da prática criptográfica moderna.

#	Descrição	Nível
11.3.1	Verifique se os modos de bloco inseguros (por exemplo, ECB) e os esquemas de preenchimento fracos (weak padding schemes, por exemplo, PKCS#1 v1.5) não são usados.	1
11.3.2	Verifique se apenas as cifras e modos aprovados, como o AES com GCM, são usados.	1
11.3.3	Verifique se os dados criptografados estão protegidos contra modificações não autorizadas preferencialmente através do uso de um método de criptografia autenticada aprovado ou da combinação de um método de criptografia aprovado com um algoritmo MAC aprovado.	2
11.3.4	Verifique se os nonces, vetores de inicialização e outros números de uso único (single-use numbers) não são usados para mais de um par de chave de criptografia e elemento de dado. O método de geração deve ser adequado ao algoritmo em uso.	3
11.3.5	Verifique se qualquer combinação de um algoritmo de criptografia com um algoritmo MAC está operando no modo encrypt-then-MAC (criptografa-depois-MAC).	3

V11.4 Hashing e Funções Baseadas em Hash

Os hashes criptográficos são usados em uma ampla variedade de protocolos criptográficos, como assinaturas digitais, HMAC, funções de derivação de chaves (KDF), geração aleatória de bits e armazenamento de senhas. A segurança do sistema criptográfico depende da força das funções hash subjacentes usadas. Esta seção descreve os requisitos para o uso de funções hash seguras em operações criptográficas.

Para o armazenamento de senhas, além do apêndice de criptografia, o OWASP Password Storage Cheat Sheet também fornecerá contexto e orientações úteis.

#	Descrição	Nível
11.4.1	Verifique se apenas funções hash aprovadas são usadas para casos de uso gerais de criptografia, incluindo assinaturas digitais, HMAC, KDF e geração aleatória de bits. Funções de hash não permitidas, como o MD5, não devem ser usadas para nenhum propósito criptográfico.	1
11.4.2	Verifique se as senhas são armazenadas usando uma função de derivação de chave aprovada e de processamento intensivo (também conhecida como “função de hash de senha”), com parâmetros configurados de acordo com a orientação atual. As configurações devem equilibrar segurança e desempenho a fim de tornar os ataques de força bruta suficientemente desafiadores para o nível exigido de segurança.	2
11.4.3	Verifique se as funções hash usadas em assinaturas digitais, como parte da autenticação de dados ou integridade de dados, são resistentes à colisão e possuem os comprimentos de bits adequados. Se for exigida a resistência à colisão, o tamanho da saída deverá ser de, no mínimo, 256 bits. Se apenas a resistência aos ataques de segunda pré-imagem (second pre-image attacks) for exigida, o tamanho da saída deverá ser de, no mínimo, 128 bits.	2
11.4.4	Verifique se a aplicação usa funções de derivação de chave aprovadas com parâmetros de key stretching (alongamento de chave) ao derivar chaves secretas de senhas. Os parâmetros em uso devem equilibrar segurança e desempenho para evitar que ataques de força bruta comprometam a chave criptográfica resultante.	2

V11.5 Valores Aleatórios

A Geração de Números Pseudoaleatórios Criptograficamente Segura (CSPRNG) é incrivelmente difícil de fazer da forma correta. Em geral, boas fontes de entropia dentro de um sistema esgotarão rapidamente se usadas em excesso, mas as fontes com menos aleatoriedade podem levar a chaves e segredos previsíveis.

#	Descrição	Nível
11.5.1	Verifique se todos os números e strings aleatórios que se destinam a não serem adivinháveis devem ser gerados usando um gerador de números pseudoaleatórios criptograficamente seguro (CSPRNG) e têm pelo menos 128 bits de entropia. Note que UUIDs não respeitam esta condição.	2
11.5.2	Verifique se o mecanismo de geração de números aleatórios em uso foi projetado para funcionar de forma segura, mesmo sob alta demanda.	3

V11.6 Criptografia de Chave Pública

A Criptografia de Chave Pública será usada onde não for possível ou indesejável compartilhar uma chave secreta entre várias partes.

Como parte disso, existe a necessidade de mecanismos aprovados para a troca de chaves, como Diffie-Hellman e Curva Elíptica de Diffie-Hellman (ECDH), para garantir que o sistema criptográfico permaneça seguro contra ameaças modernas. O capítulo “Comunicação Segura” fornece os requisitos para o TLS, de forma que os requisitos nesta seção se destinam a situações em que a Criptografia de Chave Pública esteja sendo usada em casos de uso distintos do TLS.

#	Descrição	Nível
11.6.1	Verifique se apenas modos de operação e algoritmos criptográficos aprovados são usados para geração de chaves, seeding, geração e verificação de assinatura digital. Os algoritmos de geração de chaves não devem gerar chaves inseguras que sejam vulneráveis a ataques conhecidos, por exemplo, as chaves RSA que são vulneráveis à fatoração de Fermat.	2
11.6.2	Verifique se os algoritmos criptográficos aprovados são usados para a troca de chaves (como o Diffie-Hellman), com foco em garantir que os mecanismos de troca de chaves utilizem parâmetros seguros. Isso evitará ataques ao processo de estabelecimento da chave que poderiam levar a ataques adversary-in-the-middle ou a quebras criptográficas.	3

V11.7 Criptografia de Dados Em Uso

Proteger os dados enquanto estão sendo processados é primordial. O uso de técnicas como criptografia completa de memória, criptografia de dados em trânsito e assegurar que os dados sejam criptografados o mais rápido possível após o uso, é recomendado.

#	Descrição	Nível
11.7.1	Verifique se o uso de criptografia completa de memória está habilitado para proteger os dados sensíveis enquanto estiverem em uso, impedindo o acesso por usuários ou processos não autorizados.	3
11.7.2	Verifique se a minimização de dados garante a exposição de apenas a menor quantidade de dados durante o processamento, e assegure que os dados sejam criptografados imediatamente após o uso ou logo que for possível.	3

Referências

Para mais informações, veja também:

- OWASP Web Security Testing Guide: Testing for Weak Cryptography
- OWASP Cryptographic Storage Cheat Sheet
- FIPS 140-3
- NIST SP 800-57

V12 Comunicação Segura

Objetivo de Controle

Este capítulo inclui requisitos relacionados aos mecanismos específicos que devem estar em vigor para proteger dados em trânsito, tanto entre um cliente (usuário final) e um serviço backend, quanto entre serviços internos e os de backend.

Os conceitos gerais promovidos por este capítulo incluem:

- Garantir que as comunicações sejam criptografadas externamente e, idealmente, internamente também.
- Configurar mecanismos de criptografia utilizando as diretrizes mais recentes, incluindo os algoritmos e cifras preferidos.
- Assegurar que as comunicações não sejam interceptadas por partes não autorizadas através do uso de certificados assinados.

Além de delinear princípios gerais e melhores práticas, o ASVS também fornece informações técnicas aprofundadas sobre a força criptográfica no Apêndice C - Padrões de Criptografia.

V12.1 Orientação Geral de Segurança TLS

Esta seção fornece orientações iniciais sobre como proteger as comunicações TLS. Ferramentas atualizadas devem ser utilizadas para revisar a configuração TLS de maneira contínua.

Embora o uso de certificados curinga (wildcard certificates) para o TLS não seja inerentemente inseguro, o comprometimento de um certificado que é implantado em todos os ambientes proprietários (ex., produção, homologação, desenvolvimento e testes) pode levar ao comprometimento da postura de segurança das aplicações que o utilizam. A proteção adequada, o gerenciamento e o uso de certificados TLS distintos em diferentes ambientes devem ser empregados, sempre que possível.

#	Descrição	Nível
12.1.1	Verifique se apenas as versões mais recentes recomendadas do protocolo TLS estão habilitadas, como TLS 1.2 e TLS 1.3. A versão mais recente do protocolo TLS deve ser a opção preferida.	1
12.1.2	Verifique se apenas as suites de criptografia recomendadas estão habilitadas, com as suites mais fortes definidas como preferidas. As aplicações L3 devem suportar exclusivamente cipher suites (suites de cifras) que forneçam forward secrecy (sigilo de encaminhamento perfeito).	2
12.1.3	Verifique se a aplicação valida se os certificados de cliente mTLS são confiáveis antes de usar a identidade do certificado para autenticação ou autorização.	2
12.1.4	Verifique se a revogação adequada de certificado, como o Online Certificate Status Protocol (OCSP) Stapling, está habilitada e configurada.	3
12.1.5	Verifique se o Encrypted Client Hello (ECH) está habilitado nas configurações TLS da aplicação para evitar a exposição de metadados sensíveis, como o Server Name Indication (SNI), durante os processos de handshake TLS.	3

V12.2 Comunicação HTTPS com Serviços Voltados Para a Internet (External Facing Services)

Garanta que todo o tráfego HTTP para serviços voltados externamente expostos pela aplicação seja enviado com criptografia, utilizando certificados confiáveis publicamente.

#	Descrição	Nível
12.2.1	Verifique se o TLS é usado em todas as conectividades entre um cliente e serviços HTTP voltados externamente, e não faz retrocesso (fallback) para comunicações inseguras ou não criptografadas.	1
12.2.2	Verifique se os serviços voltados externamente utilizam certificados TLS que são publicamente confiáveis.	1

V12.3 Segurança da Comunicação Geral Serviço-a-Serviço

As comunicações dos servidores (tanto as internas quanto as externas) envolvem muito mais do que apenas HTTP. As conexões de e para outros sistemas também devem ser seguras, utilizando idealmente o TLS.

#	Descrição	Nível
12.3.1	Verifique se um protocolo criptografado como o TLS é usado para todas as conexões de entrada e de saída da aplicação, incluindo os sistemas de monitoramento, ferramentas de gerenciamento, acesso remoto e SSH, middleware, bancos de dados, mainframes, sistemas de parceiros ou APIs externas. O servidor não deve retornar a protocolos inseguros ou não criptografados.	2
12.3.2	Verifique se os clientes TLS validam os certificados recebidos antes de se comunicarem com um servidor TLS.	2
12.3.3	Verifique se o TLS ou outro mecanismo adequado de criptografia de transporte é utilizado em todas as conectividades entre os serviços internos baseados em HTTP na aplicação, e não faz retrocesso (fallback) para comunicações inseguras ou não criptografadas.	2
12.3.4	Verifique se as conexões TLS entre os serviços internos utilizam certificados confiáveis. Onde certificados autoassinados (self-signed) ou gerados internamente forem utilizados, o serviço consumidor deve ser configurado para confiar apenas em CAs internas específicas e em certificados autoassinados específicos.	2
12.3.5	Verifique se os serviços que se comunicam internamente em um sistema (comunicações intra-serviço) usam autenticação forte para garantir a verificação de cada endpoint. Métodos de autenticação fortes, como a autenticação de cliente TLS, devem ser empregados para assegurar a identidade, usando infraestrutura de chave pública e mecanismos que são resistentes a ataques de repetição (replay attacks). Para as arquiteturas de microserviços, considere o uso de uma malha de serviços (service mesh) para simplificar o gerenciamento de certificados e aprimorar a segurança.	3

Referências

Para mais informações, veja também:

- OWASP - Transport Layer Security Cheat Sheet
- Mozilla's Server Side TLS configuration guide

- Mozilla's tool to generate known good TLS configurations.
- O-Saft - OWASP Project to validate TLS configuration

V13 Configuração

Objetivo de Controle

A configuração padrão da aplicação deve ser segura para uso na Internet.

Este capítulo fornece orientações sobre as diversas configurações necessárias para atingir isso, incluindo as que são aplicadas durante as fases de desenvolvimento, build e implantação (deployment).

Os tópicos abordados incluem prevenção de vazamento de dados, o gerenciamento seguro da comunicação entre os componentes e a proteção de segredos.

V13.1 Documentação da Configuração

Esta seção descreve os requisitos de documentação sobre como a aplicação se comunica com serviços internos e externos, bem como as técnicas utilizadas para impedir a perda de disponibilidade provocada pela inacessibilidade de serviços. Também abrange a documentação relacionada aos segredos.

#	Descrição	Nível
13.1.1	Verifique se todas as necessidades de comunicação para a aplicação estão documentadas. Isso deve incluir os serviços externos dos quais a aplicação depende e os casos onde um usuário final poderia ser capaz de fornecer um local (endereço) externo ao qual a aplicação irá se conectar.	2
13.1.2	Verifique se, para cada serviço que a aplicação utiliza, a documentação define o número máximo de conexões concorrentes (ex., limites do pool de conexões) e como a aplicação se comporta quando o limite for atingido, incluindo qualquer mecanismo de contingência ou de recuperação, para evitar as condições de negação de serviço.	3

#	Descrição	Nível
13.1.3	Verifique se a documentação da aplicação define estratégias de gestão de recursos para todos os sistemas externos ou serviços que utilizar (ex., banco de dados, manipuladores de arquivos (file handles), threads, conexões HTTP). Isso deve incluir os procedimentos de liberação de recursos, configurações do timeout, as formas de tratamento das falhas, e, onde existir lógica de repetição implementada, especificar os limites de repetição, atrasos (delays) e os algoritmos back-off. Para operações de request-response síncronas usando HTTP isso deve obrigar o uso de pequenos timeouts (limites de tempo) e proibir as repetições (retries), ou impô-las usando fortes limitações para prevenir as cascatas de atrasos e a exaustão de recursos.	3
13.1.4	Verifique se a documentação da aplicação define quais segredos são críticos para a segurança da aplicação e estabelece um cronograma de rotação para eles, baseado no modelo de ameaças da organização e nos requisitos de negócio.	3

V13.2 Configuração da Comunicação Backend

As aplicações interagem com vários serviços, que incluem as APIs, os bancos de dados ou outros componentes. Estes podem ser vistos como parte do interior da aplicação mas não foram incluídos nos mecanismos padrões para controles do acesso dela, ou podem ser componentes completamente externos. Em cada caso, torna-se necessário configurar a aplicação para que a interação se realize de maneira segura com esses componentes e, se assim for solicitado, deve-se também garantir a proteção sobre essa própria configuração.

Nota: O capítulo “Comunicação Segura” estabelece orientações gerais de encriptação em trânsito.

#	Descrição	Nível
13.2.1	Verifique se as comunicações entre os componentes de backend da aplicação, os quais não suportam mecanismos padrões relativos as sessões com usuários, estão autenticadas. Isto engloba as APIs, camadas de dados e middlewares. A autenticação necessita operar com o uso de contas individuais dos serviços, tokens temporários, e métodos de autenticações baseadas com o emprego de certificados ao invés do emprego inalterável (estático) nas credenciais usadas como, por exemplo: senhas, chaves das APIs, compartilhamento das contas e privilégios que foram outorgados de acessos contínuos.	2

#	Descrição	Nível
13.2.2	Verifique se a comunicação entre componentes das aplicações que rodam como aplicações tipo de servidor final (backend) entre os que figuram a execução de camadas relativas a operação de sistema em si do computador (SO - serviços do operating system local), APIs, middlewares e níveis referentes à dados e transações, ocorrem adotando as configurações das contas empregando a norma da utilização do princípio do menor privilégio, correspondente ao necessário para operar normalmente.	2
13.2.3	Verifique se, sempre no evento que requer utilização ou submissão com credenciais relativas a qualquer prestação e aceitação relativas aos serviços (autenticação de sistema), o valor enviado para e processado referente na conta de um “consumidor”, de maneira e formas alguma for a credencial fornecida a de configuração de tipo padronizada de instalação sem mudanças anteriores (por ex: raiz/raiz ou, seja root/root admin/admin).	2
13.2.4	Verifique se ocorre utilização formal por lista explícita das definições aceitáveis para permitir com restrição positiva de aprovação para definir listas normais (allowlists) o apontamento referente de forma exata de e/ou apontando todos/todos os domínios referentes (e sistemas ou fontes de acessos) do qual o processo externo foi configurado por regras permitindo a aprovação contínua sobre as ações na rede as quais são tratadas internamente e autorizadas a enviar ligações diretas do processamento inicial provenientes delas mesmas originárias (isto é para conexões geradas internamente indo pra fora: “outbounds requests”). Isto tem de cobrir chamadas relacionadas para os transportes relativas transferências com fluxos (data loads, APIs) ou para requisição aos usos dos arquivos externos. A técnica dessa lista aceita poderá ocorrer no nível próprio restrito por programa através da aplicação, na estrutura através do próprio HTTP-Server (webserver de fronteira/camadas de roteamento), pelo nível lógico nas interfaces relativas as portas da rede/firewalls e também com junções complexas contendo intersecção entre múltiplos métodos em variadas divisões de processamento.	2
13.2.5	Verifique se as configurações referentes as normas para acesso a servidor central com suporte à rede da camada web possuam estritamente o emprego por de intersecção da adoção para que todos/todos componentes operando a comunicação operem uma estrita validação baseados em lista que contenham os mapeamentos definidos que possam aceitar a efetuação (aprovadas/allowlisted) quanto para as ordens relativas originadas pelas requisições como também aceitar trâmites provindos à carga referida relativas de documentos externos ou outros suportes referentes.	2

#	Descrição	Nível
13.2.6	Verifique se, para onde as rotinas referentes à transações operacionais do programa façam conectividade apontada para fontes terceiras a outros subsistemas alheios, o projeto observe com obediência de todas definições expostas publicamente nos termos já acordados referidos no manual sobre conexões para configuração padronizada referente. Estas obrigações impõem, que obedecem: os parâmetros com máxima cota dos volumes aceitos e as paralizações conjuntas em chamadas aceitáveis para concorrência de múltiplos clientes; adoções operacionais referente no sentido ao atingir no qual limite de carga se aproxima; comportamentos adotados em limites operantes com base restritiva ao atrasos impostos com tempo máximo na qual uma espera será encerrada em timeout com resposta final com falha (connection timeouts); adoção finalizada referentes às adoções dos mecanismos perante tentativas extras posteriores contidas referentes nos processos nas definições aplicadas das lógicas e tratamentos à contínuas repetições com base sobre de rotinas da “estratégias nos retries”.	3

V13.3 Gerenciamento de Segredos

O gerenciamento de segredos é uma tarefa vital no planejamento para salvaguardar com proteções operacionais os dados durante todo e cada acesso do processamento dos usos do programa. Certos requisitos específicos no qual diz respeito diretamente e unicamente a criptografia de forma detalhada poderão ser lidos internamente perante informações da própria seção específica no capítulo “Criptografia”. Esta ramificação, contudo tem ênfase especial, focando restrita na questão da configuração do modelo gerencial relativas, dos cuidados à administração dos fluxos relativas as chaves sensíveis ao serviço.

#	Descrição	Nível
13.3.1	Verifique se um sistema e procedimentos criados pra fazer guarda referidas das soluções perante os controles referentes as regras de acesso seguras para operar as chaves como as relativas e exemplificadas por meio aos sistemas para cofres a chaves operacionais e a controle as restrições ao acessos de criação são executados nos serviços na gestão relativas sobre proteção e manuseios controlados para assegurar a construção e guarda de segredos no processamento no servidor local das chamadas e de processos referidos do lado-backend da programação contendo segredos com proteções e salvaguardas nos destrancamentos. Entre esses dados incluem: as bases sensíveis com acesso sob formato tipo em senha codificadas restritas; os chaves de algoritmos; os componentes essenciais com as ligações com parâmetros às baseadas transacionadas para servidores no exterior alheios; senhas dinâmicas criadas em processos de seeds de código-aleatório das formas usadas pela validade nos testes do modo do tempo das chamadas nos TOTPS. A verificação impõe: Em situações nas quais dados são vitais, segredos deverão continuar omitidos na adoção dos locais perante arquivos abertos durante manipulações operacionais feitas referidas a bases nos próprios projetos no repasse as operações à nível de desenvolvimento como em bibliotecas no controle centralizados, no que se estenda até os conteúdos formados em pacotes e entregues criados nos “builds/artifact”. Um projeto, ao atingir as especificações relativas em padrão final de enquadro no grau para nível da matriz em certificação na base L3 precisará por de padrão empregar referidos aparelhos exclusivos a execução dos suportes em “aparelhos criados sob níveis das estruturas fisicamente separados por peças isoladas e fechadas via hardware” da forma a qual define uma categoria e classes das configurações nos “Modelos Físicos Em Hardwares Seguros- HSM - hardware Security Modules”	2
13.3.2	Verifique se todos modelos e configurações nas ações com permissões referidas às leituras das rotinas nas proteções aos registros mantidos referenciando chaves sensíveis sigam obrigatoriamente a adoção as restrições com a normatização fundamentadas ao padrão contido pelas diretrizes baseadas nas doutrinas referente ao Princípio Dos Menores e Mínimos Privilégios (least privilege).	2

#	Descrição	Nível
13.3.3	Verifique se, perante a necessidade e momento os quais todas atuações as quais utilizem de bases referentes de lógica de uso por códigos-aleatórios (criptográficas) serão transacionadas exclusivamente, sempre sendo iniciadas rodando através unicamente do meio isolado pelas zonas e setores restritos, através módulos referidos por meio seguro, utilizando os suportes referentes para as execuções (de modos tais através do qual referimos à uso referidos ao “Vault”/ cofres na nuvem e modelos físicos como aos “hardware security modules/ HSM”), cuja obrigação do seu controle nas diretrizes nos processos deverão operar na coordenação perante os registros sigilosos ao resguardo do formato sigiloso sob proteções a vazamentos, aos acessos com intenção quebra e ou acesso de perdas operacionais relativas os dados das senhas e chaves que ocorram aos âmbitos os quais transacionam no mundo “exterior”, além fronteiras perante estas as barreiras de módulo das garantias as manutenções contidas nas seguranças.	3
13.3.4	Verifique se as orientações para adoção referente do registro ao armazenamento perante códigos chaves (segredos) possuam em si no momento da elaboração em sistema com datas as quais estipulem regras referentes os quais as senhas de sistema embutidas se perderão valor de validações através regras com tempo limitador final expostos e cujas validações operacionais farão as manutenções gerando as atualizações cíclicas operantes (rotating) da mesma maneira referidas e impostas de formato ao obedecer fielmente as bases com exigências apontadas publicamente referenciadas nos meios disponíveis nos guias formais e regulamentos operacionais pela as bases referentes as políticas internas referentes através as documentações contidas na aplicação respectiva.	3

V13.4 Vazamento de Informações Não Intencionais (Unintended Information Leakage)

Configurações que estão baseadas perante os perfis aos sistemas dos processos no momento os quais as rotinas referentes da programação estejam executando e em serviço público online, por conseguinte a níveis sob condições em modo-de-produção deverão estar blindadas pelas bases operantes das seguranças em modos-hardened e fortes; isto visa em suma a contenção relativas, das fugas providas pela extração relativas sobre exposição do ambiente referente com detalhes e configurações sensíveis de metadados não operantes que geram desnecessidades a níveis de usabilidade a funções com publicações na divulgação aberta de relativas bases dos sistemas expostos. No âmbito das detecções nas vulnerabilidades da exploração e nos ataques providos de exterior, o processo das análises apontadas da quebra de informações dos componentes a níveis da aplicação e vazamentos referidos não acarretarão um sinal isolado e direto e, ou exclusivo, como uma porta de aberturas na

rede os quais por norma relativas por sistemas a classificação as classifiquem através bases de alertas com risco principal as ameaças como níveis expressivos (sig. risks); contudo as falhas a respeito das regras referentes a este processo a níveis práticos serão combinadas as brechas expostas originadas a respeito à outras falhas (charmed exploits). Se esse tipo específico com relativas ausências das seguranças do exposto anteriormente (issues) já não houver sido exposto a níveis originais por padrões através o projeto em si mesmo e não operar na base padrão padrão na operação inicial da publicação a níveis padrão pela as defesas das regras criadas: esta atitude inicial sem as publicações referidas nas fraquezas elevará ao processo uma camada a qual estipulará as elevações baseadas as dificuldades dos ataques provenientes nos processos referidos das proteções pelas ações nas camadas nas relativas estruturas que contem base os muros aos quais cobrem da aplicação o a um patamar forte na tentativa do qual exigirão, as defesas com melhor barreiras a respeito de um cibercrime apontado (raises the bar para ataques).

À guisa da ilustração: uma tática operada por referidas diretrizes como a ocultação dos níveis dos códigos referentes contendo a versão a respeito, e embutido pelas das aplicações ou referidos servidores em camadas do backend perante este processo da mesma não anulará e a as ausências e também obrigações das necessidades providas a que atualizem através manutenções referentes a níveis e aplicações das atualizações por uso a níveis de patch pra manter perante ao código base sem fraquezas (vulnerabilities); em similaridade e analogia das configurações referidas de proibições e desligamentos as referentes publicações em respostas com as listagens automáticas criadas apontadas ao formato diretório as referentes funções base (folders-listing) do uso contido por servidor este feito inicial da mesma forma em si ao processo também assim referida no processo inicial sem cancelamentos relativas da ausência da eliminação das necessidades para uso no projeto através os serviços usando dos processos relativos a controles na autorização com a blindagens baseadas em segurança relativas para que impeça ao uso e manter arquivos sensíveis ausente a respeito na permanência perante o locais base através os diretórios na internet; ou em nível de arquivos de dados, mas este processo a mesma medida das ações efetuadas impulsionarão as exigências as relativas formas a que os ataques cibernéticos atuantes a respeito deste problema efetuados tenham restrições providas operadas através a fortificação do acesso original através processos para dificultar o trabalho com maiores complexidades geradas na execução perante as rotinas base no qual é imposta (it raises the bar).

#	Descrição	Nível
13.4.1	Verifique se a aplicação opera na inicialização e contínuo serviço operacional (deployed) perante bases com condições das publicações originadas e isentas de processos contendo as transferências nos acompanhamentos gerados nas cópias sobre qualquer e relativas, providas na sua extensão de todo base referida nas estruturas operantes em conjunto contendo embutidos aos originais metadados da engenharia que suportam o software a partir do processo de originais e bases que compõem do controle da codificação primária da versões com referidas formas que englobam repositório contidos embutidas através do repasses as cópias do servidor por diretórios com arquivos através do padrão.git as quais através uso pelas extensões as de tipo das bases operacionais.svn ou na falha referente que esta falha se faça embutir; na obrigações das presenças a qual estas informações a que estiverem publicadas: na configuração com publicações estas presenças devem operar através configurações as quais elas farão bases cujas restrições referidas a formas em publicações inalcançáveis com proibições as formas abertas pra redes referidas exteriores como nas proibições diretas de processamentos referentes através do formato base da estrutura de programação e a aplicação contidas de forma em bases na mesma através o próprio servidor operante (à si mesmo).	1
13.4.2	Verifique se configurações perante aos testes e também nas investigações os processos de correções internas as quais em ambientes através do desenvolvedores referentes através a rotinas referentes relativas o processos no “modo debug” do sistema de componentes a que constituem referidos a todos dos locais a programação em todo processo de servidores as qual já se fazem através dos meios nos quais de execução pública e também ativa do sistema em operações contínuas relativas de base a processos perante a “modo e local no servidores-produções” estão sendo desligados da atuação do sistema das opções de referidas aplicações (disabled): essa imposição da técnica visa o combate no contínuo dos acompanhamentos abertos as falhas e relativas ações em exposições não aceitas através aos detalhes dos módulos operacionais contidas gerados provindo referidas características a processos relativas os modos investigações operantes (debugging resources and feature) e das publicações operacionais dos fluxos do processo com extração através detalhes abertos através perante das vazões do sigilo a nível embutidos os quais produzem falhas por fluxos das lógicas de vazamento sobre vazões aos dados os relativas da “information-leakage”.	2

#	Descrição	Nível
13.4.3	Verifique se estruturas a respeito do nível através as operações no uso com processadores perante publicadores baseados aos provedores online relativas a provedores base web (web-servers), aos mesmos na prestação contínua operacionais referentes dos usuários acessando (clients), não repassam as devoluções relativas as ordens provindas na abertura de informações contendo repasses embutidas referentes aos catálogos ou de visualização relativas as aberturas sobre e das listagens do estrutura base através as navegações sobre árvores aos contendo aos respectivos ao uso nos diretório (direcory-listing) com a exceção aos processo relativas por ordens referidas a definições explícitas contidas através com bases pelas intenções documentadas e planejadas a níveis a aplicações (unless explicitly intended).	2
13.4.4	Verifique se através as atuações das regras provindas no protocolo padrão da base perante requisições do processador web o emprego as definições usando do tipo do modo HTTP apontado no estilo da comunicação TRACE estão proibidas através recusas a qual eles não possuem o uso em formatos suportado provindo ao níveis das aplicações através do processador central ativo os que no níveis a operação contínua com ambiente público em fase na versão de operações no ambientes de formato modo produtivo-produção para os sistemas. Esta exigência em ação fará proteções contidas a níveis às intenções provindas contra aos ataques a qual produzem vazões do sigilo da níveis e aberturas ao fluxos no referidos baseadas através por vazamentos dos dados que produzem sobre processos das “information-leakage”.	2

#	Descrição	Nível
13.4.5	Verifique se os componentes relativos dos relatórios referentes à publicações por usos dos repositórios contendo bases os quais fornecem, referidas através os quais detalham dos métodos os repasses no detalhamento a nível aos comandos dos manuais referentes sobre guias a referidos do emprego nos módulos referentes de “documentações” do serviços do projetos a processos contidos das operações como através e no exemplos operacionais com chamadas às API baseadas nos setores com atuações restrita do uso relativas ao meio de comunicação do nível local das referidas conexões com a camada por sistemas aos ambientes referentes às redes internas locais -e também relativas aos endereços e também dos nós da estrutura apontadas nas referidas como as terminações de acesso a processos operacionais apontadas nas requisições referidas com o emprego baseados à estrutura referidas a “pontos de contato à análises de métrica operacional - endpoints de observações ou monitoring (monitoring endpoints) das requisições-continuem contidos na segurança a blindadas ao formatos restritos aos quais não operarão baseados as devoluções relativas na exibição por meio e formas ao formatos não expostos nas redes públicas, baseadas as exceções apenas dos processos em rotinas relativas referidas as ordens intencionalmente e, criadas, expressamente de uso provindo explícita de e formal intencionalmente.	2
13.4.6	Verifique se a aplicação não faz divulgações sobre detalhes que fornecem baseadas de acesso nas versões contidas ao nível aos referidos sobre informações das edições ou atualizações no suporte através as instalações sobre números referidos às datas nas informações precisas baseadas na relativas aos componente no processo interno nos que atuem nos sistemas e processos os quais trabalham na camadas a bases operantes das atuações relativas aos fundos do projetos (backend).	3

#	Descrição	Nível
13.4.7	Verifique se a estrutura os quais controlam os setores da camadas os processadores relativas contidos ao referidos aos ambientes por servidores com emprego do padrão da linguagem nos protocolos das provedoras web (web tier) operem baseados através das regras contendo parametrização de configurações que os estipulam as relativas normas cuja função atuará por processos no fornecimentos a serviços contendo a aceitação restritas apenas para entrega relativas as permissões contidas, em processos de serviços contendo em arquivos referidas de um modo a base específica relativas as regras às correspondentes restritas e restrições apenas com aceitação correspondentes e de uso relativas de terminações por sufixos no final a processos específicos referentes, com as as extensões com a função em arquivos permitidos definidos para proibir ao processo e devolução providas e não repassar devolução indesejadas que em perdas sem que tenham a níveis e formato da forma de processos relativas com falhas em informações vazadas; falhas providas do código aos originais arquivos a que contenham código fontes e perdas das aberturas por configurações internas do local no servidores.	3

Referências

Para mais informações, veja também:

- OWASP Web Security Testing Guide: Configuration and Deployment Management Testing

V14 Proteção de Dados

Objetivo de Controle

As aplicações não conseguem contabilizar todos os padrões de uso e comportamentos dos usuários e devem, portanto, implementar controles para limitar o acesso não autorizado a dados sensíveis nos dispositivos dos clientes.

Este capítulo inclui requisitos relacionados a definir quais dados precisam ser protegidos, como eles devem ser protegidos e mecanismos específicos a serem implementados ou armadilhas a serem evitadas.

Outra consideração para a proteção de dados é a extração em massa, modificação ou uso excessivo. Os requisitos de cada sistema provavelmente serão muito diferentes, de modo que determinar o que é “anormal” deve levar em consideração o modelo de ameaças e o risco de negócios. Do ponto de vista

do ASVS, a detecção desses problemas é tratada no capítulo “Registro de Segurança e Tratamento de Erros” e o estabelecimento de limites é tratado no capítulo “Validação e Lógica de Negócios”.

V14.1 Documentação de Proteção de Dados

Um pré-requisito essencial para conseguir proteger os dados é categorizar quais dados devem ser considerados sensíveis. Provavelmente haverá diferentes níveis de sensibilidade e, para cada nível, os controles necessários para proteger os dados nesse nível serão diferentes.

Existem vários regulamentos e leis de privacidade que afetam a forma como as aplicações devem abordar o armazenamento, uso e transmissão de informações pessoais sensíveis. Esta seção não tenta mais duplicar esses tipos de proteção de dados ou legislação de privacidade, mas foca em considerações técnicas chave para proteger dados sensíveis. Por favor, consulte as leis e regulamentos locais, e consulte um especialista em privacidade qualificado ou um advogado, conforme necessário.

#	Descrição	Nível
14.1.1	Verifique se todos os dados sensíveis criados e processados pela aplicação foram identificados e classificados em níveis de proteção. Isso inclui dados que são apenas codificados e, portanto, facilmente decodificados, como strings Base64 ou a carga (payload) em texto simples (plaintext) dentro de um JWT. Os níveis de proteção precisam levar em consideração quaisquer regulamentações e normas de proteção de dados e privacidade com os quais a aplicação é obrigada a cumprir.	2
14.1.2	Verifique se todos os níveis de proteção de dados sensíveis têm um conjunto documentado de requisitos de proteção. Isso deve incluir (mas não se limitar a) requisitos relacionados a criptografia geral, verificação de integridade, retenção, como os dados devem ser registrados (logged), controles de acesso a dados sensíveis em registros, criptografia em nível de banco de dados, tecnologias de privacidade e melhoria de privacidade a serem usadas e outros requisitos de confidencialidade.	2

V14.2 Proteção Geral de Dados

Esta seção contém vários requisitos práticos relacionados à proteção de dados. A maioria é específica a problemas particulares, como o vazamento de dados não intencional, mas há também um requisito geral para implementar controles de proteção baseados no nível de proteção exigido para cada item de dados.

#	Descrição	Nível
14.2.1	Verifique se dados sensíveis são enviados ao servidor apenas no corpo da mensagem HTTP ou em campos de cabeçalho, e que a URL e a query string (string de consulta) não contenham informações sensíveis, como uma chave de API ou um token de sessão.	1
14.2.2	Verifique se a aplicação evita que dados sensíveis sejam armazenados em cache em componentes do servidor, como balanceadores de carga e caches de aplicação, ou garanta que os dados sejam eliminados com segurança após o uso.	2
14.2.3	Verifique se dados sensíveis definidos não são enviados a partes não confiáveis (por exemplo, rastreadores de usuário/user trackers) para prevenir a coleta indesejada de dados fora do controle da aplicação.	2
14.2.4	Verifique se os controles em torno de dados sensíveis relacionados à criptografia, verificação de integridade, retenção, como os dados devem ser registrados (logged), controles de acesso em torno de dados sensíveis em logs, tecnologias de privacidade e melhoria de privacidade, são implementados conforme definido na documentação para o nível de proteção de dados específico.	2
14.2.5	Verifique se os mecanismos de cache são configurados para armazenar em cache apenas respostas que possuam o content type (tipo de conteúdo) esperado para aquele recurso e não contenham conteúdo dinâmico e sensível. O servidor web deve retornar uma resposta 404 ou 302 quando um arquivo inexistente for acessado, em vez de retornar um arquivo válido diferente. Isso deve evitar ataques de Engano de Cache Web (Web Cache Deception attacks).	3
14.2.6	Verifique se a aplicação retorna apenas os dados sensíveis mínimos necessários para a funcionalidade da aplicação. Por exemplo, retornar apenas alguns dos dígitos de um número de cartão de crédito e não o número completo. Se os dados completos forem exigidos, eles deverão ser mascarados na interface do usuário a menos que o usuário os visualize especificamente.	3
14.2.7	Verifique se as informações sensíveis estão sujeitas à classificação de retenção de dados, garantindo que os dados desatualizados ou desnecessários sejam excluídos automaticamente, em um cronograma definido ou conforme a situação exigir.	3

#	Descrição	Nível
14.2.8	Verifique se as informações sensíveis são removidas dos metadados de arquivos enviados por usuários, a menos que o usuário tenha consentido com o armazenamento.	3

V14.3 Proteção de Dados no Lado do Cliente (Client-side)

Esta seção contém requisitos para prevenir que dados vazem de maneiras específicas no lado do cliente ou agente do usuário (user agent) de uma aplicação.

#	Descrição	Nível
14.3.1	Verifique se os dados autenticados são apagados do armazenamento do cliente, como o DOM do navegador, após o cliente ou a sessão ser finalizada. O campo de cabeçalho de resposta HTTP 'Clear-Site-Data' pode ajudar com isso, mas o lado do cliente também deve conseguir limpar os dados se a conexão com o servidor não estiver disponível quando a sessão for encerrada.	1
14.3.2	Verifique se a aplicação define campos de cabeçalho de resposta HTTP anticanche adequados (ex., Cache-Control: no-store) para que dados sensíveis não sejam cacheados em navegadores.	2
14.3.3	Verifique se os dados armazenados no armazenamento do navegador (como localStorage, sessionStorage, IndexedDB ou cookies) não contêm dados sensíveis, com exceção de tokens de sessão.	2

Referências

Para mais informações, veja também:

- Consider using the Security Headers website to check security and anti-caching header fields
- Documentation about anti-caching headers by Mozilla
- OWASP Secure Headers project
- OWASP Privacy Risks Project
- OWASP User Privacy Protection Cheat Sheet
- Australian Privacy Principle 11 - Security of personal information
- European Union General Data Protection Regulation (GDPR) overview
- European Union Data Protection Supervisor - Internet Privacy Engineering Network
- Information on the "Clear-Site-Data" header
- White paper on Web Cache Deception

V15 Codificação e Arquitetura Seguras

Objetivo de Controle

Muitos requisitos do ASVS se relacionam a uma área particular da segurança, como autenticação ou autorização, ou dizem respeito a um tipo particular de funcionalidade de aplicação, como o registro (logging) ou manipulação de arquivos.

Este capítulo fornece requisitos gerais de segurança a serem considerados ao projetar e desenvolver aplicações. Esses requisitos não focam apenas na arquitetura limpa (clean architecture) e qualidade de código, mas também em práticas de arquitetura e codificação específicas necessárias para a segurança da aplicação.

V15.1 Documentação de Codificação e Arquitetura Seguras

Muitos requisitos para o estabelecimento de uma arquitetura segura e defensável dependem da documentação clara de decisões feitas a respeito da implementação de controles de segurança específicos e os componentes usados dentro da aplicação.

Esta seção descreve os requisitos de documentação, incluindo a identificação de componentes que se considera conter “funcionalidade perigosa” ou ser um “componente de risco”.

Um componente com “funcionalidade perigosa” (dangerous functionality) pode ser um componente desenvolvido internamente ou de terceiros que realiza operações como desserialização de dados não confiáveis, parsing de arquivos raw ou de dados binários, execução dinâmica de código ou manipulação direta de memória. As vulnerabilidades nesses tipos de operações apresentam alto risco de comprometer a aplicação e, potencialmente, de expor a infraestrutura subjacente.

Um “componente de risco” (risky component) é uma biblioteca de terceiros (ou seja, não desenvolvida internamente) onde há falta de controles de segurança, ou estão mal implementados, ao redor dos seus processos de desenvolvimento ou sua funcionalidade. Exemplos incluem os componentes de manutenção deficiente, sem suporte formal, os que se encontram no estágio de ciclo de vida final (end-of-life) e ou possuem histórico atestado referidas em casos notórios relativos no padrão na fraquezas de extrema relevância no descobrimento falhas provindas pelas (significant vulnerabilities).

Esta seção também enfatiza a importância de se definir prazos de tempo apropriados para lidar com vulnerabilidades em componentes terceiros.

#	Descrição	Nível
15.1.1	Verifique se a documentação da aplicação define janelas de tempo de remediação baseadas em risco referentes nas atuações nas as versões baseadas a vulnerabilidades e na forma do controle da documentação relativas provindas nos arquivos em atualizações pelas quais afetam bibliotecas e os usos no geral e também provindas em e relativas em componente terceiro as que contenham referências perante vulnerabilidades, isto visando o ato em e relativas mitigações as relativas aos quais visam as mitigações com redução através minimização provindos nas consequências base aos usos dessas e na forma do processo através aos riscos com danos e perdas aos oriundos no processo e componentes as bases dos processos referentes.	1
15.1.2	Verifique se há processos referidos do qual baseiam no catálogo do tipo em inventário como em sistemas com listas através as referências operacionais base com listagens em arquivos de e materiais com bases referidas as do projeto das bibliotecas contendo aos códigos ou base das relativas dos uso nas listagens através ao (Software Bill of Materials - SBOM), as quais operem a gestão e manutenção a e e aos com o e o controle no registro sob todas referidas bibliotecas (de terceiro) as quais são transacionadas no fluxo referentes por utilidades na uso das ações correntes, isso referenciando o conjunto nas as confirmações aos processos com verificação a que as origens dessas ferramentas a componentes tenham o repasses aos através as origens na de depósitos do padrão referidos base (repositories) operadas confiavelmente sob prévias e de definições as referentes bases de confiáveis manutenções as atuações contínuas (continually maintained).	2
15.1.3	Verifique se a documentação da aplicação identifica funcionalidades que consomem tempo ou exigem muitos recursos. Isso deve incluir maneiras de evitar a perda de disponibilidade devido ao uso excessivo dessa funcionalidade e evitar uma situação em que construir uma resposta leve mais tempo do que o tempo limite (timeout) de resposta ao consumidor. As defesas possíveis podem incluir o processamento assíncrono, uso de filas e imposição nos limites sobre os processos executados no formato os processos ocorrem paralelos (parallel processes) e sob cota em processos definidos por a um processo e níveis referentes a e no máximo limitadas referida “por usuários” por cliente e níveis as taxas totais através o funcionamento e referentes de “aplicações inteiras globais (per application)” .	2

#	Descrição	Nível
15.1.4	Verifique se a documentação da aplicação destaca bibliotecas terceiras a quais com avaliação da gestão de processo operem a serem classificadas a apontamentos com riscos de componentes aos referidos na classe no grupo ao nível “componentes de riscos”.	3
15.1.5	Verifique se a documentação da aplicação destaca partes do sistema a aplicação na que a uso no modelo na que atua com o operações através o uso dos processos perante base aos de modelos com as “funcionalidades perigosas (dangerous functionality)” são estarem referidas.	3

V15.2 Arquitetura de Segurança e Dependências

Esta seção inclui requisitos em manuseios referentes por rotinas com processos perante uso através bases às exigência nos sistemas as atuações ao manuseio relativos as atuações contra bases e componentes defasados as, sem suporte, que contém riscos ao atuações referidas de a um processo na ou o formato do processamento referentes ao “manuseio por controle as gestão sob e das e com sobre os acompanhamentos a base relativas às dependências do de códigos (dependency management)”.

Também engloba a adoção referentes as rotinas a através do a respeito na através nas bases providas aos e com atuações de arquitetura a com as técnicas sobre do do sistema e referentes a como nas execuções sob encapsulamento e isolamento de contêineres e isolamento das em nível de atuações na redes operacionais isto operando visando às com ações atuações referidas em formas reducionais sobre no ao tamanho com atuações de mitigar dos danos causados com e do reflexos nos repasses causados na de usos referentes com relativas bases perante sobre às no a base do modelos das e às na atuações perigosas em o as bases às com formas do perigo na “dangerous operations” nas atuações nas sobre os componentes base ou também no os da bases operantes com processos a riscos de aos os que definimos base do referidos em e como a nas “risky components” nos modelos já na conforme as de e contidas na e a os de referidas nas no formato do contidas os de com formas em na base das referidas definições contidas em e providas da na base as definições na seção do e na de prévia na base e e do de referentes a prevenção nos de formas referidas as falhas que acarretem e os de aos das perdas providas às no formas a exaustão às do das nas disponibilidade com bases as originais nas e dos com de das do a em devido nas super os em ou os super os com formas os uso no referidas a com os excessivo sobre funcionalidades com de os a exigência sobre do processamentos ou formas ao grande formas na dos e ou o a do do de do peso e as sobre com da relativas à do processador e de e na com os do consumo em processos.

#	Descrição	Nível
15.2.1	Verifique se a aplicação a apenas a a nas com de nas contém dos os e componentes nas na formas em com que a não a de e de o violaram os limites estipulados e as métricas o do no a os que do sobre contidas a nas do relativas as o das relativas os cronograma do e e as no prazos do na documentação nas sobre relativas no atualizações de bases nas na nos em atualizações com e da com formas em remediação.	1
15.2.2	Verifique se a aplicação implementou relativas a o do e com formas de a nas do da proteções e a sobre defesas da em as no as contidas contra a o com perdas aos relativas nas no a da providas de da de da perdas as em nas do formas e de as na das perdas os de de na disponibilidade e as a providas devido ao a uso a da em na em relativas a à e as na na forma com uso com das as funcionalidades a qual com operam o no atuações a consumam de grandes proporções da de e do tempo a na ou a no ou as e que exijam das os a em formas em grandes os na os com e e no da e e as de exigentes grandes em de em consumo de formas de com na nas consumo nas sobre recursos e do a na de e relativas aos de os o da na ao de a e a e de o uso os decisões o da as a os com do das em de da em sobre nas a da em relativas na com base relativas na em das em a os documentadas a as e na formas em no das sobre de na e em a na a na documentadas as nas do do as de as com da em as a na a de no com os de das e a de a no e do a na da e e a na em de as em sobre o e com estratégias com e de e e a na a e e e.	2
15.2.3	Verifique se o de as em ambiente a a da no do em produção a e do de e contém da nas apenas as a da o de e com a na de nas de a com a no funcionalidades de com a na as que de e e em nas em são de e com a e do e da a a e exigidas na nas com e e nas e para do de que do a a a na e o em com de as da em a em do e a da nas de a a a na a e na aplicação a e a da a a nas com e nas a a funcionar de a a e as a com o em com e não e de na de na de de expõe e nas a de e a de e funcionalidades com de e a do as a o de em com na a as a a nas do e e as em e estranhas a a e do e a a de e do as e do e a as e a as do e de o a e (extraneous a a a functionality) de e na da a o como a na as as e nas e códigos a de a a teste as nas a a e e a e e a as e e as e o e a (test a e de code) e nas a na as com em a trechos as a as a e nas as as as na de na e de a a de exemplo a na de a (sample as do a a snippets) e a as as e a nas e o a e e de a e a as de do funcionalidades do e de de a e de a desenvolvimento a de a a.	2

#	Descrição	Nível
15.2.4	Verifique se na e de os na a de componentes de na de e terceiros na as na e e de na as todas as de as a na de a a de a na as na a as suas dependências do a da a a a transitivas na as são as e de a na na a nas incluídas na as a do na a de repositório da da a de de de a e de esperado na a a as de as seja de a a de a de de e e de ele e de as da as da da internamente de na da a da a na a de e e de a na na de propriedade a e da as da a da as ou a a da na uma da da a e a fonte na a de externa a e as e que a não do a de da há da as as da as e a na na as do as as de a a de risco da a as de a da da de de de a um as a da as a a ataque as a as de da da a de a e de as da confusão a de a de a da da de a dependência a as de a.	3
15.2.5	Verifique se da as da a da as aplicação de de as e as de implementa da de as da de a de as as proteções do de da de adicionais da em da a da as a torno da a de as da de as a da partes as as da de as as da as da a a de aplicação a da que de de a a as as as de as a da a são a a de de as as as a a documentadas de a a as como as a da as a as as contendo da de da a de de a a as as a as “funcionalidade as as as perigosa” a as as de a de a ou as a a as de usando a a a as bibliotecas as as as de as a a de de de terceiros da as de as as as as as consideradas as de as de a as as “componentes a as de a de a de risco”. a da as as as de as de Isso da as as a pode de as as a as da incluir as a as as técnicas as as de a a as de a como da as as a a sandboxing, da as a a a da encapsulamento, a as a as as containerização da a as as da as da as a as a as ou as da a a a as as a a isolamento as a as a a de de de de nível de da as a a rede as as de de as da a da a para da as de as de a a a as de as da atrasar as de de da a a as e as da a da de a de da a deter da as a a de de de da a atacantes a de de a da de de a de de que a a a da as de de de a a as as de da da comprometem a as de de da as da a de da da uma de da a da de da da de a de de da de parte da de as as da as da da a de de as as da as a uma as as da de a de da de as aplicação da a da da a de de de de a pivotar de de a de da as da as da da as de para as a as as a a de a as outro a as as a da as de de as lugar de da de a da as da a de a as a da a de da da da a as as a na a de de da as de de de da de de da de de da da a aplicação da da a de as a as de as da de de da a as.	3

V15.3 Codificação Defensiva

Esta da da seção as a as de da abrange de da da de de os a de a a da da de da da de a da de de tipos a da as de da de as as de de vulnerabilidade da as da de da de as as da as a de a as da as de as da a (vulnerability da da a types) da as de a da as de a as, de as da de as de as a as da as de da da de as incluindo da a da as de a as da a type as da a a as da a juggling, de da de da da prototype de a as as a a pollution de as de e a da a a de de da da de de as a da de a de a outros da da a as da a da da as a de da,

a da a que a as as resultam a de da a a do da de as da de de uso da as as a de a de as as a a padrões as de de de a as de as da a a codificação as as as da de as as a da inseguros de as de de de em as da a da uma da da da as da as de da de de as de da linguagem de de as de de da de as a da as de da particular da as de a a as as as as da as. da de as as de de da a as Alguns da as a de a da as as de podem da da de de a não de a as de ser de a de a de da relevantes de de a para as as de de as todas da as a as de as da linguagens a as da as a de, da a de as da de enquanto da a de a de da a outros a de de de as de a a a de as da de a de de de terão a a a da de a da da as de as correções de as da de as de de de a de de de as da a específicas de de de a as da da de da de a da da a de a linguagem de a da da as da da ou da a de da da as a de as podem da da as de da de se a de as de as a da relacionar as a a as de de a da a da como da da as de da de da a a da a as de uma de as as as da linguagem a as da de ou as as de da da as a framework a da de a da a em da a as as particular da a da de de da as a da de as a as as a a lida de de a com da da as da a de a um da as a as a recurso de da as a as a, as de as da de como a da de a as as parâmetros da de a de HTTP a de. da as a a a de de Ele a da de a a de da a da a as as a de de também da de as a de as as de as a de as considera da as a da de o de da da as a a de de a risco a a da de a as de não as as as validar da de as as as a a da as da a de criptograficamente de da a de de as de as atualizações da da a de da as da aplicação.

Também da a da as de a a de a a a considera de as de a os de as as a da de de de a riscos da da de da as associados da da da as a as as a da as a da ao a da da as da de uso de a a de da de as objetos da de de de a para de de da as da de da representar de a de de a as da a da da de da da a de a as a da as da de a de as itens as de as a a a as as de da da dados as a a e de as a da as as de aceitar de as as de a a da da de as a e a da de da as retornar de de a da da de a as as a estes da da a de de as as de através a as a a da de APIs as da da da as a externas de a de as da a de a a. da as de Neste de de a caso de a de da as a, a a de a as da a a de de da as aplicação da a da da as de a as de deve da as de da de as de de garantir da da de a as que a as a de os da de de de da a campos a da a da de da as da a a as as de da da dados as as as a de de a que de as a da não de as da a a devem a as de ser de as a de as as de da de as a as as a as as graváveis a as a as da (writable) as a da as de a não de de a a as sejam da de de as a a de as modificados as a de as por da a da de as a de a de de as as entrada da as de do da de da usuário de as da as da da as de da de da a de a (mass a as de a a de a as a assignment) de da a a e a a as que de da a a as de da de de da a API de da da seja de da as seletiva da da da sobre de a da de quais a as da a da a de da a a de campos a da de da as de as da as de da as as dados a as de as são as as a da de as retornados as as de. Onde a as da a a a o as a de acesso as a de a da a da de as a da a a ao da de da campo de da as de de da de de de as depende a da as da as de de da as de de da da as permissões da da da de as as de as as a de um as da da de as da de as usuário de da da as as da as as as de da, a as isso da da de de a de de deve de as a as de da as as de as as as as de de as da a de ser as da de de de da as considerado as as da da da a as a as a de as as a as de as da no de a a a a as contexto a da de as de do da as da de as requisito a a da as de de de a as da a de a a as controle da a de de as as da a as a a as de as as acesso as a da da de em as da a da a nível as a as a da da a a as de da da de da a de de as as as campo as da da a as as as de da as da no da a as da as de as a da as da de as as capítulo de de de da de as as as a as Autorização de.

#	Descrição	Nível
15.3.1	Verifique se a aplicação retorna apenas o subconjunto necessário de campos de um objeto de dados. Por exemplo, ela não deve retornar um objeto de dados inteiro, pois alguns campos individuais não devem estar acessíveis aos usuários.	1
15.3.2	Verifique se, nos casos em que o backend da aplicação faz chamadas a URLs externas, ele esteja configurado para não seguir redirecionamentos, a menos que isso seja uma funcionalidade intencional.	2
15.3.3	Verifique se a aplicação possui contramedidas para se proteger contra ataques de atribuição em massa (mass assignment) limitando os campos permitidos por controlador (controller) e ação, por ex., não é possível inserir ou atualizar o valor de um campo quando isso não foi projetado para fazer parte daquela ação.	2
15.3.4	Verifique se todos os componentes de proxy e middleware transferem o endereço IP original do usuário corretamente usando campos de dados confiáveis que não possam ser manipulados pelo usuário final, e a aplicação e o servidor web usam este valor correto para decisões de logging e de segurança como limitação de taxa (rate limiting), levando em conta que mesmo o endereço IP original pode não ser confiável devido a IPs dinâmicos, VPNs ou firewalls corporativos.	2
15.3.5	Verifique se a aplicação garante explicitamente que as variáveis são do tipo correto e realiza operações de comparação e de igualdade estritas (strict equality and comparator operations). Isso é para evitar vulnerabilidades de confusão de tipos (type confusion) ou type juggling causadas pelo código da aplicação fazer uma suposição sobre um tipo de variável.	2
15.3.6	Verifique se o código JavaScript é escrito de uma forma que evita a poluição de protótipo (prototype pollution), por exemplo, usando Set() ou Map() em vez de objetos literais.	2
15.3.7	Verifique se a aplicação possui defesas contra ataques de poluição de parâmetros HTTP (HTTP parameter pollution attacks), particularmente se o framework da aplicação não faz distinção sobre a fonte (source) dos parâmetros de requisição (query string, body parameters, cookies ou campos de cabeçalho).	2

V15.4 Concorrência Segura

Problemas de concorrência (concurrency) tais como as condições de corrida (race conditions), vulnerabilidades do tipo de tempo-de-verificação-para-o-tempo-de-uso (time-of-check to time-of-use - TOCTOU), deadlocks (bloqueios), livelocks, thread starvation e sincronização inadequada podem levar a comportamento imprevisível e riscos de segurança. Esta seção inclui várias técnicas e estratégias para ajudar a mitigar esses riscos.

#	Descrição	Nível
15.4.1	Verifique se objetos compartilhados em códigos multi-thread (como caches, arquivos ou objetos na memória acessados por várias threads) são acessados de forma segura usando tipos thread-safe e mecanismos de sincronização como locks (travas) ou semáforos para evitar condições de corrida e corrupção de dados.	3
15.4.2	Verifique se as checagens no estado de um recurso, como sua existência ou permissões, e as ações que dependem delas são executadas como uma operação atômica única para evitar as condições de corrida do tipo time-of-check to time-of-use (TOCTOU). Por exemplo, checando se um arquivo existe antes de abri-lo, ou verificando o acesso de um usuário antes de concedê-lo.	3
15.4.3	Verifique se bloqueios (locks) são usados consistentemente para evitar que as threads fiquem presas, seja esperando umas pelas outras ou tentando novamente (retrying) sem parar, e se a lógica de travamento permanece dentro do código responsável por gerenciar o recurso para garantir que os bloqueios não possam ser modificados inadvertidamente ou maliciosamente por classes ou código externo.	3
15.4.4	Verifique se as políticas de alocação de recursos previnem o thread starvation ao garantir o acesso justo (fair access) aos recursos, como, por exemplo, pelo aproveitamento de pools de threads, permitindo que as threads de menor prioridade procedam dentro de um tempo razoável.	3

Referências

Para mais informações, veja também:

- OWASP Prototype Pollution Prevention Cheat Sheet
- OWASP Mass Assignment Prevention Cheat Sheet
- OWASP CycloneDX Bill of Materials Specification
- OWASP Web Security Testing Guide: Testing for HTTP Parameter Pollution

V16 Registro de Segurança e Tratamento de Erros

Objetivo de Controle

Os logs (registros) de segurança são distintos de logs de erro ou de desempenho e são usados para registrar eventos relevantes para a segurança, como decisões de autenticação, decisões de controle de acesso e tentativas de burlar os controles de segurança, como a validação de entrada ou a validação da lógica de negócios. O seu propósito é o de apoiar a detecção, resposta e investigação, fornecendo dados estruturados e de alto sinal (high-signal) para ferramentas de análise, como SIEMs.

Os logs não devem incluir dados pessoais sensíveis a menos que exigido legalmente, e quaisquer dados registrados devem ser protegidos como um ativo de alto valor. O registro (logging) não deve comprometer a privacidade ou a segurança do sistema. As aplicações também devem falhar de forma segura (fail securely), evitando divulgação ou interrupção desnecessária.

Para orientações detalhadas de implementação, consulte as OWASP Cheat Sheets na seção de referências.

V16.1 Documentação de Registros de Segurança (Security Logging)

Esta seção garante um inventário claro e completo dos logs em toda a pilha da aplicação (application stack). Isso é essencial para o monitoramento eficaz da segurança, para a resposta a incidentes e a conformidade.

#	Descrição	Nível
16.1.1	Verifique se existe um inventário documentando o registro (logging) realizado em cada camada da pilha de tecnologia da aplicação, quais eventos estão sendo registrados, os formatos dos logs, onde esse log é armazenado, como é utilizado, como o acesso a ele é controlado, e por quanto tempo os logs são mantidos.	2

V16.2 Registro (Logging) Geral

Esta seção fornece requisitos para garantir que os logs de segurança sejam estruturados de forma consistente e contenham os metadados esperados. O objetivo é tornar os logs legíveis por máquina e analisáveis em sistemas e ferramentas distribuídas.

Naturalmente, os eventos de segurança envolvem frequentemente dados sensíveis. Se tais dados forem registrados sem consideração, os próprios logs se tornam classificados e, portanto, sujeitos a requisitos de criptografia, políticas de retenção mais rigorosas e potencial divulgação durante as auditorias.

Portanto, é crítico registrar apenas o que é necessário e tratar os dados de log com o mesmo cuidado com outros ativos sensíveis.

Os requisitos abaixo estabelecem os requisitos fundamentais para os metadados de logs, para a sincronização, para o formato e seu controle.

#	Descrição	Nível
16.2.1	Verifique se cada entrada de log (log entry) inclui os metadados necessários (como quando, onde, quem, o que) que permitiriam uma investigação detalhada da linha do tempo quando um evento acontecer.	2
16.2.2	Verifique se as fontes de tempo para todos os componentes de registro estão sincronizadas, e que os carimbos de data/hora (timestamps) nos metadados do evento de segurança usem UTC ou incluam o deslocamento explícito (offset) do fuso horário. O UTC é recomendado para garantir a consistência através de sistemas distribuídos e para prevenir confusões durante as transições do horário de verão.	2
16.2.3	Verifique se a aplicação apenas armazena ou transmite os logs para os arquivos e serviços que estejam documentados no inventário de logs.	2
16.2.4	Verifique se os logs podem ser lidos e correlacionados pelo processador de log que está em uso, preferencialmente usando um formato de log comum.	2
16.2.5	Verifique se, ao registrar os dados sensíveis, a aplicação impõe o logging baseado no nível de proteção daquele dado. Por exemplo, pode não ser permitido logar determinados dados, como credenciais ou detalhes de pagamento. Outros dados, como os tokens de sessão, podem ser logados apenas ao estarem em hash ou mascarados, de forma total ou parcial.	2

V16.3 Eventos de Segurança

Esta seção define os requisitos para registrar os eventos de relevância de segurança dentro da aplicação. Capturar estes eventos é essencial para detectar comportamentos suspeitos, apoiar investigações e cumprir as obrigações de compliance.

Esta seção descreve os tipos de eventos que devem ser registrados, mas não tenta fornecer detalhes exaustivos. Toda aplicação possui fatores de risco e o contexto operacional únicos.

Observe que, embora o ASVS inclua o registro (logging) de eventos de segurança no escopo, o alerta (alerting) e a correlação (ex., regras de SIEM ou infraestrutura de monitoramento) são considerados fora do escopo e são tratados por sistemas operacionais e de monitoramento.

#	Descrição	Nível
16.3.1	Verifique se todas as operações de autenticação são registradas, incluindo as tentativas bem e malsucedidas. Metadados adicionais, como o tipo de autenticação ou os fatores usados, também deverão ser coletados.	2
16.3.2	Verifique se as tentativas falhas de autorização são registradas. Para o L3, isto deve incluir o registro de todas as decisões da autorização, incluindo o registro quando os dados sensíveis são acessados (sem registrar os dados sensíveis em si).	2
16.3.3	Verifique se a aplicação registra os eventos de segurança que foram definidos na documentação e que também registre as tentativas de ignorar (bypass) os controles de segurança, tais como a validação da entrada, a lógica de negócios e as medidas antiautomação.	2
16.3.4	Verifique se a aplicação registra os erros inesperados e as falhas dos controles de segurança, tais como falhas do TLS no backend.	2

V16.4 Proteção de Logs

Os logs são artefatos forenses valiosos e devem ser protegidos. Se os logs puderem ser facilmente modificados ou excluídos, eles perdem a integridade e tornam-se não confiáveis para as investigações de incidentes ou procedimentos legais. Os logs podem expor o comportamento interno de aplicações ou metadados sensíveis, tornando-os um alvo atraente para os atacantes.

Esta seção define os requisitos a fim de garantir que os logs estejam protegidos contra acessos não autorizados, adulterações e as suas divulgações, e que sejam transmitidos de forma segura e armazenados e sistemas seguros e isolados.

#	Descrição	Nível
16.4.1	Verifique se todos os componentes de logging codificam os dados apropriadamente para evitar a injeção em logs (log injection).	2
16.4.2	Verifique se os logs estão protegidos contra o acesso não autorizado e não podem ser modificados.	2
16.4.3	Verifique se os logs são transmitidos de forma segura para um sistema logicamente isolado e separado para análise, detecção, alerta e encaminhamento (escalation). O objetivo é garantir que, caso a aplicação for invadida, os logs não sejam comprometidos.	2

V16.5 Tratamento de Erros

Esta seção define os requisitos para garantir que as aplicações apresentem falhas em forma harmoniosa e contínua de modo seguro (fail gracefully and securely), sem divulgação dos detalhes internos e sensíveis.

#	Descrição	Nível
16.5.1	Verifique se uma mensagem genérica é retornada ao consumidor perante as falhas quando ocorre um erro inesperado e de impacto na segurança, garantindo assim nenhuma exposição nos dados sensíveis do sistema interno (ex: como stack traces, comandos em queries, senhas chaves dos sistemas de processos, e os tokens em aberto).	2
16.5.2	Verifique se a aplicação continuará a atuar e no processamento em modos segurança perante os eventos das falhas através os acessos na bases dos recursos os de formatos externo e/ou externos a, e no o através a exemplo das chamadas aos os na do dos de processos em do padrões na (patterns) nas das sobre de as os do e com as nos referidos de a no como os a em as os do de de do e “circuit a e a de do breakers”a e na (disjuntores de de da circuitos) as a ou de de em de a degradação e e a harmoniosa da do de (graceful o da degradation).	2
16.5.3	Verifique se a aplicação a a o falha o e a as e e de da da de da de forma as de da de a de de a harmoniosa a o de a de da a e a segura a de da as da (fails e as e as de gracefully da e a de and a as de de securely), as de a a o incluindo as a as o de quando a as e da a ocorre o a a as uma a a as da a e de a exceção, o de e a da da da prevenindo da de a as a da condições de da as a da o a de da a da de da “fail-open”o da da (falhar a da de de de a de de e de aberto) e a de da as de de como da e a a o a da de processar a de a a as uma da a a a transação de as da a da as apesar a de as de as a da dos de a de da da erros o de de as as resultantes a e da as da a da a lógica de de a da as de de da a a validação da a da as.	2

#	Descrição	Nível
16.5.4	Verifique se a a a um a a de da de manipulador o a de de a a da as de o as erro a e da as as “de a da as de da as da última o de a da da as de as da a instância” a de de de a as de as (last as a de de as as da as resort) de as de as as as a é as a de a da as de definido da o a a o o que a de de de as as as irá a da a as as capturar da de a a de da as as todas de da as de as a a de de as a da a exceções o a de as de as as de de as da a as não de de da a da de a tratadas. de as de a a de Isto a as da a é a a o as as a a da de a tanto a de de as as a de para as as a de a evitar a a a de de a de as as de a a de de da perder de o de a da da da a as da a detalhes da a a de a do de a a o de da de da a de de as da a as de erro o a da as as da as da que a as as as de as a as de da de devem a de a de de a ir da as da da da de de a de a as de de para da da da de de os o as de arquivos de da de da da de da log, de a de de o a de de e as as a quanto a as da da de as as para as de de de a a as a a de a as as de a garantir as de a as da a que as a da as da a de da da de a um de da a o erro da da da de as de de a a a as as as não de de as a a de o derrube as da a o de o de as as a da da de de todo de da da de de de o as da a de da da a de de de da a processo da a a as de de as da as a as as as a de aplicação, a de de da de o da a a as a a de levando da a da as a da de a de de de da uma da de a as a a de de perda a da de da a as de as a as da de de as disponibilidade a a as a a as da de as a.	3

Nota: Determinadas a da as linguagens da da as as, as a as da as de as a (incluindo da de a o da a a Swift, da da da as Go e, a a as de da as da de da de através da da as as a as as a as as a de de de de de de a de as prática de de da de comum de da a as de as as de a as a de de as da as design, de a a as a da a as a muitas da a as de a de de da as a as as a as da as linguagens a de da a da da a de as a da de funcionais, da de da de) de as a a não a a as de de a suportam a de as as a as as de exceções a a a de a de de a de da a de de ou as as de da da a as da de da manipuladores a da da da as a de de de eventos as a da de de de as as da da a as as a de as a de as as última da a a as de as instância de da da de de da da de as as a de a as da da de de a a. as a Neste a de as de de de a caso de a as de a de a da, as da de da arquitetos a as de da da de de a as e a as a de a desenvolvedores de da da a as da as devem a de a da as da a as de a usar da as de a da de da as de de a de de a da da de um da a a da as de padrão a da as de a da as da as a as, as a a linguagem a de a as a da da ou a da as da de da a um da da a da da a a da de de a de da as da da de modo a a a da a de a a da da de da a amigável as de a da da a a ao de as de as a framework as as as da de as as de a da da da a as de as a a de da para a de as as da as as da as as garantir a as de a da a as a a da as as a da a que a as as as aplicações de de de a a de da da as a as da possam de as as de a de as de de a as as a a a de as a lidar da de as da de a de da a de a de as de de forma a as as da as a de da a de a de de as segura da a da as com as de as as a eventos as a da de excepcionais a a de a de a da de as as, a a a de a as de as inesperados a a de a a ou a as de da a a as a de de as a da relacionados as da a de da da de a da as da a segurança a da a.

Referências

Para mais informações, veja também:

- OWASP Web Security Testing Guide: Testing for Error Handling
- OWASP Authentication Cheat Sheet section about error messages
- OWASP Logging Cheat Sheet
- OWASP Application Logging Vocabulary Cheat Sheet

V17 WebRTC

Objetivo de Controle

A Comunicação em Tempo Real pela Web (Web Real-Time Communication - WebRTC) possibilita a troca de voz, vídeo e dados em tempo real em aplicações modernas. À medida que a adoção aumenta, a proteção da infraestrutura WebRTC torna-se crítica. Esta seção fornece os requisitos de segurança para as partes interessadas (stakeholders) que desenvolvem, hospedam ou integram os sistemas WebRTC.

O mercado de WebRTC pode ser categorizado em geral em três segmentos:

1. **Desenvolvedores de Produtos (Product Developers):** Fornecedores proprietários ou de código aberto que criam e fornecem soluções e produtos WebRTC. O foco principal é o desenvolvimento de tecnologias WebRTC robustas e seguras que possam ser utilizadas por outras partes.
2. **Plataformas de Comunicação como Serviço (CPaaS):** Provedores que ofertam APIs, SDKs e a infraestrutura ou as plataformas necessárias à viabilização de funcionalidades em WebRTC. Os provedores CPaaS poderão utilizar produtos vindos através de primeiro segmento (Desenvolvedores) ou mesmo o fazer na base criadora própria em software pro seu provimento (WebRTC software) e oferecer estes aos serviços (services).
3. **Provedores de Serviço (Service Providers):** As organizações as quais tiram vantagens de produtos e os integram vindos via desenvolvedores do produto e através CPaaS, ou mesmo realizam próprio seu desenvolvimentos da solução WebRTC. Eles desenvolvem as bases dos ambientes interligados nos quais implementarão a criação de uso, a base no sistema a programas online as salas/ambientes de conferências virtuais em telecomunicações/conferencing, setores em cuidado as redes de tratamento e atendimentos para as bases em saúde, plataformas educacionais de e-learning entre e para as demais demandas aos nichos da atuação de negócios cujo requisito na interconexão e transação por áudio e vídeo requeiram comunicações a meios a atuações em em meios os e usos em transações perante sistemas online operando no momento do processo no tipo de comunicações nos formatos ditos “em tempo real (real-time)”

Os requisitos em termos à base perante segurança abordadas aqui referenciados estão com enfoque primário de atuação nas proteções aos que figuram nos papéis nas cadeias e nos processos dos Desenvolvedores dos Produto, do grupo no modelo através da CPaaS, além e os Provedores no Serviço a qual operem ao perfil e aos grupos nos quais:

- Façam uso aos componentes nas plataformas da tipo e do base nos modelos através softwares a bases da comunidade/públicos livres - as de código aberto (open-source) à o a o uso em construção em as da e sobre em suas da da aplicação de a em de em WebRTC.
- A de e da em baseiem-se em da produtos comerciais da a o as para e na da WebRTC a na a como nas na em parte no na em as da de o infraestrutura da da o na a de seu e de uso do de o a da a do sistema da a na.
- A em nas façam e as a e uso e da de nas do na de a soluções da da na de a WebRTC do desenvolvidas e a internamente da a do de a a e de as ou em de na de o a integram da de da a o na as em da e a os os vários do da os na da a em do de a a e do e a e na do do e de componentes a da e a em da a de em do de um o de da a do serviço de de a de de da na de a de o o a as da e da da o o coeso de a de do do da.

É a importante de as de o da notar o as do e da na a da de de de o a na e a que e de as da de a de de e de as a da o de e da o da e o a a de a na de a do esses o na e na e do de e da do e requisitos o a e de a na de da de e as na o segurança da e a o da da não da da na de se da a a as e do do a de a do de o do a da e aplicam na da na e de a a da da aos de do as do as e a de da as da de da de e de e a a as desenvolvedores o e e da que as a o e de da e na utilizam o as as o o as e exclusivamente o de e SDKs a da de as o a a as de de o do de da de as a da da a do a da e APIs do as de e fornecidas do e do as de de de a de as por e na do do de da as a fornecedores a de a o de as as a as da CPaaS do as o as de da de. as da de Para as e da na a e na a e do de tais o da a e as as da da a as da a e de a da as desenvolvedores, o da a a as os da de provedores a na a de da o as do as o a da de CPaaS a e na de as as da a as de são as e da do a e as de tipicamente a a as o de as e e de responsáveis da a o a as a de de de a o da de por de as as da as e o da do as de grande e do o as da as da da a as de de da a a as a a da de parte as de e das da o da da as as do as a de de as as da de de de de de a as as preocupações do de a as de as da e a o de e as e as subjacentes a de da de as da de a e o segurança de e a do as a de e as as o da de dentro do de de da de de da de suas as de da e as as o da as de e da de as as da plataformas, do a de de de de a a as e do da a de a e a e a o as um da a da da de a as as padrão as de as as as a de a a da da as de as genérico o e as e as e de de o da de da de as de e a a as a as segurança a da a e as as a e as as da como as da de da a a as as as da a o da as de a da a a de da as de as de ASVS do da as as pode as de as as de de da a e de a não a as de a a de da as as de as a atender da da da as de as as as e de da da da as a da as da a da as totalmente da as a da da de às de a da a da de as de de da de de da de da a de de de suas as de da a as da da de da as as as as as as de necessidades de as de a as as.

V17.1 Servidor TURN

Esta seção define os requisitos de segurança para sistemas que operam seus próprios servidores TURN (Traversal Using Relays around NAT). Os servidores TURN auxiliam no retransmissão de mí-

dia em ambientes de rede restritivos, mas podem representar riscos se configurados incorretamente. Esses controles se concentram na filtragem segura de endereços e na proteção contra o esgotamento de recursos.

#	Descrição	Nível
17.1.1	Verifique se o serviço Traversal Using Relays around NAT (TURN) permite acesso apenas aos endereços IP que não são reservados a propósitos especiais (por exemplo, redes internas, broadcast, loopback). Note que isso se aplica tanto aos endereços IPv4 quanto aos IPv6.	2
17.1.2	Verifique se o serviço Traversal Using Relays around NAT (TURN) não é suscetível ao esgotamento de recursos quando usuários legítimos tentam abrir um grande número de portas no servidor TURN.	3

V17.2 Mídia

Esses requisitos se aplicam apenas aos sistemas que hospedam seus próprios servidores de mídia WebRTC, como Unidades de Encaminhamento Seletivo (Selective Forwarding Units - SFUs), Unidades de Controle Multiponto (Multipoint Control Units - MCUs), servidores de gravação ou servidores de gateway. Os servidores de mídia manipulam e distribuem os streams de mídia, tornando a segurança desses componentes crítica para a proteção da comunicação entre os pares. A salvaguarda dos streams de mídia é fundamental nas aplicações WebRTC para evitar espionagem (eavesdropping), adulteração e ataques de negação de serviço (denial-of-service) que podem comprometer a privacidade do usuário e a qualidade da comunicação.

Em especial, é necessário implementar proteções contra ataques de inundação (flood attacks), tais como limitação de taxa (rate limiting), validação de registros de tempo (timestamps), uso de relógios sincronizados para equiparar intervalos em tempo real e o gerenciamento de buffers para evitar overflow (transbordamento) e manter a sincronia adequada. Se os pacotes de uma sessão de mídia específica chegarem muito rapidamente, os pacotes em excesso deverão ser descartados. Também é importante proteger o sistema de pacotes malformados implementando a validação de entrada, tratando falhas de integer overflow (estouro de número inteiro) de forma segura, impedindo ataques de buffer overflow e empregando outras técnicas robustas de tratamento de erros.

Sistemas que dependem puramente da comunicação de mídia peer-to-peer (ponto a ponto) entre navegadores web, sem o envolvimento ou mediação através de servidores de mídia intermediários, estão excluídos desses requisitos de segurança específicos de mídia.

Esta seção refere-se ao uso de Datagram Transport Layer Security (DTLS) no contexto do WebRTC. Um requisito relacionado a possuir uma política documentada para o gerenciamento de chaves criptográficas pode ser encontrado no capítulo “Criptografia”. Informações sobre métodos criptográficos aprovados podem ser encontradas no Apêndice de Criptografia do ASVS ou em documentos como o NIST SP 800-52 Rev. 2 ou BSI TR-02102-2 (Versão 2025-01).

#	Descrição	Nível
17.2.1	Verifique se a chave para o certificado Datagram Transport Layer Security (DTLS) é gerenciada e protegida com base na política documentada para o gerenciamento de chaves criptográficas.	2
17.2.2	Verifique se o servidor de mídia está configurado para utilizar e suportar as cipher suites aprovadas pelo Datagram Transport Layer Security (DTLS) e um perfil seguro de proteção para a Extensão DTLS a fim de estabelecer as chaves pro Secure Real-time Transport Protocol (DTLS-SRTP).	2
17.2.3	Verifique se a autenticação do Secure Real-time Transport Protocol (SRTP) é checada no servidor de mídia com objetivo de prevenir aos ataques com Injeção da Real-time Transport Protocol (RTP) resultem base perante à seja tanto no formato numa criação com situação relativa em Negações dos Serviços e nas inclusão maliciosa através das inserções referentes de conteúdos de mídias base vídeo ou de áudio dentro aos contínuos percursos provindas das transmissões operadas do sistema aos fluxos contínuos nas mídias originais e legítimas (media streams).	2
17.2.4	Verifique se o servidor de mídia consegue atuar e proceder processando nas execuções aos fluxos operantes com e perante tráfegos recebidos das chamadas através contínuas nas transmissões as mídias ao deparar os pacotes através os formatos inválidos através falhas malformados com proveniências usando Secure Real-time Transport Protocol (SRTP).	2
17.2.5	Verifique se o servidor de mídia tem base operacional operante as quais possibilitem as execuções processando a níveis aos fluxos e as atividades através tráfegos a chamadas contínuas recebidas perante e através ao ataques com eventos por baseados na contínuas envios sucessivos nas bases às inundações (flood) contendo formatos nos chamadas feitas operados contendo a níveis por pacotes usando pelo protocolo tipo e formato por meio do Secure Real-time Transport Protocol (SRTP) os mesmos, através remessas que atestam sendo provindos dos usuários classificados base legítimos.	3

#	Descrição	Nível
17.2.6	Verifique se o servidor de mídia atua contendo com bases blindados no servidor onde ele não atue no estado ao formato através da vulnerabilidade em condições de vulnerabilidade em corrida “ClientHello” associados e com o tipo da falha das base operantes com atuações de Datagram Transport Layer Security (DTLS). E isso o fazendo referenciando a atuação através nas bases contendo confirmações o qual e caso através o banco dos conhecimento a onde constata com dados através referências publicamente com notório os quais ele opere através contendo as falhas ao ambiente, e através o modo onde façam atuações nas checagens testando através e executando com teste o modo da atuações no testes operando base com teste da a vulnerabilidade tipo Race condition (condição de corrida).	3
17.2.7	Verifique se o mecanismo atuando as quais em processos na forma nas execuções com gravações através vídeos ou na base contendo o uso na formato em do processo a nível contendo gravações de áudio através a embutidas relativas a atuações na base por sistemas do servidores a mídia contenha no formato contendo blindagens operantes a quais permitam a contínua processos em formato na processamentos das contínuas formas nos tráfegos recebidas os operados nas transmissões recebidas referentes as mídias perante atuação provindo sobre e referentes ao baseados nos de inundações em contínuas formas os eventos com contínuas (flood) aos pacotes na base e tipo operando pelo método contendo no pacotes ao através no uso e formato por transmissões a com através Secure Real-time Transport Protocol (SRTP) provindas no atuações vindas no formato perante chamadas ao nível contendo usuários base os formatado legitimados.	3

#	Descrição	Nível
17.2.8	Verifique se o certificado através as base aos relativas de certificações do Datagram Transport Layer Security (DTLS) é submetido em atuações em checagens na através as comparações operando a formas de de avaliações em bases às relativas contra referidas de atributos nas propriedades das e através e aos atributo do padrão Session Description Protocol (SDP) referente com dados na nas em formato da impressão das a níveis ao das fingerprint, atuando com e ao processamento nas interrupções provindas nas atuações nas desliga contendo desativações às através no corte na formato nos no repasse da atuação no a o contínuos baseados operacionais contendo nos envio aos fluxo em transmissões perante chamadas por uso por nas relativas contidas relativas a caso nas checagens atuantes que contarem em as em através do nível referidos as as apontamentos nas falhas referidas, na formas relativas isso contendo o formato do base no objetivo a garantir e assegurar os dados da as atuações das relativas na autenticidades dos envios perante os fluxos relativas as e do transmissões base da contínua e as de mídia relativas do streams.	3

V17.3 Sinalização

Esta seção define os requisitos para sistemas que operam seus próprios servidores de sinalização WebRTC. A sinalização coordena a comunicação ponto a ponto (peer-to-peer) e deve ser resiliente contra ataques que podem interromper o estabelecimento ou o controle da sessão.

Para garantir a sinalização segura, os sistemas devem tratar as entradas malformadas de forma harmoniosa e permanecer disponíveis sob carga.

#	Descrição	Nível
17.3.1	Verifique se o servidor de sinalização consegue continuar o processamento de mensagens de sinalização de entrada legítimas durante um ataque de inundação (flood attack). Isso deve ser alcançado implementando a limitação de taxa (rate limiting) no nível de sinalização.	2

#	Descrição	Nível
17.3.2	Verifique se o servidor de sinalização consegue continuar o processamento de mensagens de sinalização legítimas ao encontrar uma mensagem de sinalização malformada que poderia causar uma condição de negação de serviço. Isso poderia incluir a implementação da validação da entrada (input validation), do tratamento seguro contra estouros de número inteiro (integer overflows), prevenindo estouros de buffer (buffer overflows) e o uso de outras técnicas robustas de tratamento de erros.	2

Referências

Para mais informações, veja também:

- The WebRTC DTLS ClientHello DoS is best documented at Enable Security's blog post aimed at security professionals and the associated white paper aimed at WebRTC developers
- RFC 3550 - RTP: A Transport Protocol for Real-Time Applications
- RFC 3711 - The Secure Real-time Transport Protocol (SRTP)
- RFC 5764 - Datagram Transport Layer Security (DTLS) Extension to Establish Keys for the Secure Real-time Transport Protocol (SRTP)
- RFC 8825 - Overview: Real-Time Protocols for Browser-Based Applications
- RFC 8826 - Security Considerations for WebRTC
- RFC 8827 - WebRTC Security Architecture
- DTLS-SRTP Protection Profiles

Apêndice A: Glossário

- **Absolute Maximum Session Lifetime (Tempo de Vida Máximo Absoluto da Sessão)** – Também referido como “Timeout Geral” pelo NIST, esta é a quantidade máxima de tempo que uma sessão pode permanecer ativa após a autenticação, independentemente da interação do usuário. Este é um componente da expiração da sessão.
- **Allowlist (Lista de Permissões)** – Uma lista de dados ou operações permitidas, por exemplo, uma lista de caracteres que têm permissão para realizar validação de entrada.
- **Anti-forgery token (Token Antifalsificação)** – Um mecanismo pelo qual um ou mais tokens são passados em uma solicitação e validados pelo servidor da aplicação para garantir que a solicitação veio de um endpoint esperado.
- **Application Security (Segurança de Aplicações)** – A segurança em nível de aplicação foca na análise dos componentes que compõem a camada de aplicação do Modelo de Referência de Interconexão de Sistemas Abertos (Modelo OSI), em vez de focar, por exemplo, no sistema operacional subjacente ou nas redes conectadas.

- **Application Security Verification (Verificação de Segurança de Aplicações)** –A avaliação técnica de uma aplicação em relação ao OWASP ASVS.
- **Application Security Verification Report (Relatório de Verificação de Segurança de Aplicações)** –Um relatório que documenta os resultados gerais e a análise de suporte produzida pelo verificador para uma aplicação específica.
- **Authentication (Autenticação)** –A verificação da identidade reivindicada de um usuário da aplicação.
- **Automated Verification (Verificação Automatizada)** –O uso de ferramentas automatizadas (sejam ferramentas de análise dinâmica, ferramentas de análise estática ou ambas) que usam assinaturas de vulnerabilidade para encontrar problemas.
- **Black box testing (Teste de Caixa Preta)** –Um método de teste de software que examina a funcionalidade de uma aplicação sem espiar suas estruturas ou funcionamentos internos.
- **Common Weakness Enumeration (CWE) (Enumeração de Fraquezas Comuns)** –Uma lista desenvolvida pela comunidade de fraquezas comuns de segurança de software. Serve como uma linguagem comum, uma régua de medição para ferramentas de segurança de software e uma linha de base (baseline) para esforços de identificação, mitigação e prevenção de fraquezas.
- **Component (Componente)** –Uma unidade autônoma de código, com interfaces de disco e rede associadas, que se comunica com outros componentes.
- **Credential Service Provider (CSP) (Provedor de Serviços de Credenciais)** –Também chamado de Provedor de Identidade (IdP). Uma fonte de dados de usuário que pode ser usada como fonte de autenticação por outras aplicações.
- **Cross-Site Script Inclusion (XSSI) (Inclusão de Script Entre Sites)** - Uma variante do ataque de Cross-Site Scripting (XSS) na qual uma aplicação web recupera código malicioso de um recurso externo e inclui esse código como parte de seu próprio conteúdo.
- **Cross-Site Scripting (XSS)** –Uma vulnerabilidade de segurança tipicamente encontrada em aplicações web que permite a injeção de scripts no lado do cliente (client-side) dentro do conteúdo.
- **Cryptographic module (Módulo Criptográfico)** –Hardware, software e/ou firmware que implementa algoritmos criptográficos e/ou gera chaves criptográficas.
- **Cryptographically secure pseudo-random number generator (CSPRNG) (Gerador de Números Pseudoaleatórios Criptograficamente Seguro)** - Um gerador de números pseudoaleatórios com propriedades que o tornam adequado para uso em criptografia, também referido como gerador de números aleatórios criptográficos (CRNG).
- **Datagram Transport Layer Security (DTLS)** –Um protocolo criptográfico que fornece segurança de comunicação em uma conexão de rede. É baseado no protocolo TLS, mas adaptado para proteger protocolos orientados a datagramas (geralmente sobre UDP). Definido na RFC 9147 para o DTLS 1.3.
- **Datagram Transport Layer Security Extension to Establish Keys for the Secure Real-time Transport Protocol (DTLS-SRTP)** –Um mecanismo para usar um handshake DTLS para estabelecer material criptográfico para uma sessão SRTP. Definido na RFC 5764.
- **Design Verification (Verificação de Design)** –A avaliação técnica da arquitetura de segurança

de uma aplicação.

- **Dynamic Application Security Testing (DAST) (Teste Dinâmico de Segurança de Aplicações)** –Tecnologias projetadas para detectar condições indicativas de uma vulnerabilidade de segurança em uma aplicação em seu estado de execução.
- **Dynamic Verification (Verificação Dinâmica)** –O uso de ferramentas automatizadas que usam assinaturas de vulnerabilidades para encontrar problemas durante a execução de uma aplicação.
- **Fast IDentity Online (FIDO)** –Um conjunto de padrões de autenticação que permite o uso de uma variedade de métodos de autenticação diferentes, incluindo biometria, Trusted Platform Modules (TPMs), tokens de segurança USB, etc.
- **Hardware Security Module (HSM) (Módulo de Segurança de Hardware)** –Componente de hardware que armazena chaves criptográficas e outros segredos de maneira protegida.
- **Hibernate Query Language (HQL)** –Uma linguagem de consulta de aparência semelhante ao SQL usada pela biblioteca Hibernate ORM.
- **HTTP Strict Transport Security (HSTS)** –Uma política que instrui o navegador a conectar-se apenas ao domínio que retorna o cabeçalho via TLS e quando um certificado válido é apresentado. É ativada usando o campo de cabeçalho de resposta Strict-Transport-Security.
- **HyperText Transfer Protocol (HTTP) (Protocolo de Transferência de Hipertexto)** –Um protocolo de aplicação para sistemas de informação hipermídia, colaborativos e distribuídos. É a base da comunicação de dados para a World Wide Web.
- **HyperText Transfer Protocol over SSL/TLS (HTTPS)** –Um método de proteger a comunicação HTTP criptografando-a usando o Transport Layer Security (TLS).
- **Identity Provider (IdP) (Provedor de Identidade)** –Também chamado de Credential Service Provider (CSP) nas referências do NIST. Uma entidade que fornece uma fonte de autenticação para outras aplicações.
- **Inactivity Timeout (Tempo Limite de Inatividade)** –Este é o período de tempo que uma sessão pode permanecer ativa na ausência de interação do usuário com a aplicação. Este é um componente da expiração da sessão.
- **Input Validation (Validação de Entrada)** –A canonicalização e validação de entradas de usuários não confiáveis.
- **JSON Web Token (JWT)** –A RFC 7519 define um padrão para um objeto de dados JSON composto por uma seção de cabeçalho (header) que explica como validar o objeto, uma seção de corpo (body) contendo um conjunto de reivindicações (claims) e uma seção de assinatura (signature) que contém uma assinatura digital que pode ser usada para validar o conteúdo da seção do corpo. É um tipo de token autocontido.
- **Local File Inclusion (LFI) (Inclusão de Arquivo Local)** –Um ataque que explora procedimentos vulneráveis de inclusão de arquivos em uma aplicação, levando à inclusão de arquivos locais já presentes no servidor.
- **Malicious Code (Código Malicioso)** –Código introduzido em uma aplicação durante o seu desenvolvimento, sem o conhecimento do proprietário da aplicação, que burla (circumvents) a política de segurança pretendida para a aplicação. Não é o mesmo que malware, como um

vírus ou worm!

- **Malware** –Código executável que é introduzido em uma aplicação durante a execução (runtime) sem o conhecimento do usuário ou administrador da aplicação.
- **Message authentication code (MAC) (Código de Autenticação de Mensagem)** - Um checksum criptográfico em dados, calculado por um algoritmo de geração de MAC, que é usado para fornecer garantia sobre sua integridade e autenticidade.
- **Multi-factor authentication (MFA) (Autenticação Multifator)** –Autenticação que inclui dois ou mais dos fatores únicos.
- **Mutual TLS (mTLS) (TLS Mútuo)** –Veja autenticação de cliente TLS.
- **Object-relational Mapping (ORM) (Mapeamento Objeto-Relacional)** –Um sistema usado para permitir que um banco de dados relacional/baseado em tabelas seja referenciado e consultado dentro de um programa de aplicação usando um modelo de objeto compatível com a aplicação.
- **One-time Password (OTP) (Senha de Uso Único)** –Uma senha que é gerada de forma única para ser usada em uma única ocasião.
- **Open Worldwide Application Security Project (OWASP)** –O Open Worldwide Application Security Project (OWASP) é uma comunidade mundial livre e aberta com foco em melhorar a segurança de softwares aplicativos. Nossa missão é tornar a segurança de aplicações “visível” , para que pessoas e organizações possam tomar decisões informadas sobre os riscos de segurança de aplicações. Veja: <https://www.owasp.org/>.
- **Password-Based Key Derivation Function 2 (PBKDF2)** –Um algoritmo especial unidirecional usado para criar uma chave criptográfica forte a partir de um texto de entrada (como uma senha) e um valor aleatório adicional (salt), podendo, portanto, ser usado para dificultar a quebra (crack) de uma senha offline se o valor resultante for armazenado em vez da senha original.
- **Public Key Infrastructure (PKI) (Infraestrutura de Chave Pública)** –Um arranjo que vincula chaves públicas às respectivas identidades das entidades. O vínculo é estabelecido através de um processo de registro e emissão de certificados em e por uma autoridade de certificação (CA).
- **Public Switched Telephone Network (PSTN) (Rede Telefônica Pública Comutada)** –A rede telefônica tradicional que inclui tanto telefones fixos quanto telefones móveis.
- **Real-time Transport Protocol (RTP) e Real-time Transport Control Protocol (RTCP)** –Dois protocolos usados em associação para o transporte de streams multimídia. Usados pela pilha WebRTC. Definidos na RFC 3550.
- **Reference Token (Token de Referência)** –Um tipo de token que atua como um ponteiro ou identificador para estado ou metadados armazenados em um servidor, às vezes chamados de tokens aleatórios ou tokens opacos. Ao contrário dos tokens autocontidos, que embutem parte de seus dados relevantes dentro do próprio token, os tokens de referência não contêm informações intrínsecas, dependendo do servidor para obter o contexto. O token de referência será ou conterá um identificador de sessão.
- **Relying Party (RP) (Parte Confiável)** –Geralmente uma aplicação que confia em um usuário que se autenticou em um provedor de autenticação separado. A aplicação depende de algum tipo de token ou conjunto de asserções assinadas fornecidas por esse provedor de autenticação

para confiar que o usuário é quem diz ser.

- **Remote File Inclusion (RFI) (Inclusão Remota de Arquivo)** - Um ataque que explora procedimentos de inclusão vulneráveis na aplicação, resultando na inclusão de arquivos remotos.
- **Scalable Vector Graphics (SVG)** - Uma linguagem de marcação baseada em XML para descrever gráficos vetoriais bidimensionais.
- **Secure Real-time Transport Protocol (SRTP) e Secure Real-time Transport Control Protocol (SRTCP)** - Um perfil dos protocolos RTP e RTCP fornecendo suporte para criptografia de mensagens, autenticação e proteção de integridade. Definido na RFC 3711.
- **Security Architecture (Arquitetura de Segurança)** - Uma abstração do design de uma aplicação que identifica e descreve onde e como os controles de segurança são usados, e também identifica e descreve a localização e a sensibilidade dos dados do usuário e da aplicação.
- **Security Assertion Markup Language (SAML)** - Um padrão aberto para autenticação de logon único (single sign-on) com base na passagem de asserções assinadas (geralmente objetos XML) entre o provedor de identidade e a relying party.
- **Security Configuration (Configuração de Segurança)** - A configuração de tempo de execução (runtime) de uma aplicação que afeta como os controles de segurança são usados.
- **Security Control (Controle de Segurança)** - Uma função ou componente que realiza uma verificação de segurança (por exemplo, uma verificação de autorização) ou que, quando chamado, resulta em um efeito de segurança (por exemplo, a geração de um registro de auditoria).
- **Security information and event management (SIEM) (Gerenciamento de Informações e Eventos de Segurança)** - Um sistema para detecção de ameaças, conformidade e gerenciamento de incidentes de segurança através da coleta e análise de dados relacionados à segurança de várias fontes dentro da infraestrutura de TI de uma organização.
- **Self-Contained Token (Token Autocontido)** - Um token que encapsula um ou mais atributos que não dependem do estado no lado do servidor (server-side state) ou de outro armazenamento externo. Esses tokens garantem a autenticidade e a integridade de seus atributos contidos, permitindo a troca segura e “sem estado”(stateless) de informações entre sistemas. Os tokens autocontidos geralmente são protegidos usando técnicas criptográficas, como assinaturas digitais ou códigos de autenticação de mensagens (MACs), para garantir a autenticidade, a integridade e, em alguns casos, a confidencialidade de seus dados. Exemplos comuns incluem as Asserções SAML e os JWTs.
- **Server-side Request Forgery (SSRF) (Falsificação de Solicitação do Lado do Servidor)** - Um ataque que abusa da funcionalidade no servidor para ler ou atualizar recursos internos. O atacante fornece ou modifica uma URL, da qual o código em execução no servidor lerá ou submeterá dados.
- **Session Description Protocol (SDP)** - Um formato de mensagem para configurar sessões multimídia (usado, por exemplo, no WebRTC). Definido na RFC 4566.
- **Session Identifier ou Session ID (Identificador de Sessão)** - Uma chave que identifica uma sessão stateful armazenada no backend. Será transferida de e para o cliente como um “Token de Referência” ou dentro dele.
- **Session Token (Token de Sessão)** - Uma frase genérica (catch-all) usada neste padrão para se

referir ao token ou valor usado em mecanismos de sessão sem estado/stateless (que usam um token autocontido) ou em mecanismos de sessão stateful (que usam um token de referência).

- **Session Traversal Utilities for NAT (STUN)** –Um protocolo usado para auxiliar a travessia de NAT a fim de estabelecer comunicações ponto a ponto (peer-to-peer). Definido na RFC 3489.
- **Single-factor authenticator (Autenticador de fator único)** –Um mecanismo para verificar se um usuário está autenticado. Deve ser algo que você sabe (segredos memorizados, senhas, frases secretas, PINs), algo que você é (biometria, impressão digital, escaneamento facial) ou algo que você possui (tokens OTP, um dispositivo criptográfico como um smart card).
- **Single Sign-on Authentication (SSO) (Autenticação de Logon Único)** –Isso ocorre quando um usuário faz login em uma aplicação e, em seguida, é automaticamente logado em outras aplicações sem ter que se reautenticar. Por exemplo, ao fazer login no Google, o usuário será logado automaticamente em outros serviços do Google, como YouTube, Google Docs e Gmail.
- **Software bill of materials (SBOM) (Lista de Materiais de Software)** - Uma lista estruturada e abrangente de todos os componentes, módulos, bibliotecas, frameworks e outros recursos necessários para construir ou montar uma aplicação de software.
- **Software Composition Analysis (SCA) (Análise de Composição de Software)** –Um conjunto de tecnologias projetadas para analisar a composição de aplicações, dependências, bibliotecas e pacotes em busca de vulnerabilidades de segurança de versões de componentes específicos em uso. Isso não deve ser confundido com a análise de código-fonte, que agora é comumente referida como SAST.
- **Software development lifecycle (SDLC) (Ciclo de Vida de Desenvolvimento de Software)** –O processo passo a passo pelo qual o software é desenvolvido, indo desde os requisitos iniciais até a implantação e manutenção.
- **SQL Injection (SQLi) (Injeção de SQL)** –Uma técnica de injeção de código usada para atacar aplicações orientadas a dados, na qual instruções SQL maliciosas são inseridas em um ponto de entrada.
- **Stateful Session Mechanism (Mecanismo de Sessão Stateful)** –Em um mecanismo de sessão stateful, a aplicação retém o estado da sessão no backend, o qual normalmente corresponde a um token de sessão, gerado usando um gerador de números pseudoaleatórios criptograficamente seguro (CSPRNG), que é emitido para o usuário final.
- **Stateless Session Mechanism (Mecanismo de Sessão Sem Estado/Stateless)** –Um mecanismo de sessão sem estado usará um token autocontido que é passado aos clientes e contém informações de sessão que não são necessariamente armazenadas no serviço que, em seguida, recebe e valida o token. Na realidade, um serviço precisará ter acesso a algumas informações de sessão (como uma lista de revogação de JWT) para ser capaz de impor os controles de segurança exigidos.
- **Static application security testing (SAST) (Teste Estático de Segurança de Aplicações)** – Um conjunto de tecnologias projetadas para analisar o código-fonte da aplicação, byte code e binários em busca de condições de codificação e de design que sejam indicativas de vulnerabilidades de segurança. As soluções SAST analisam uma aplicação de “dentro para fora” em um estado não executável.

- **Threat Modeling (Modelagem de Ameaças)** –Uma técnica que consiste em desenvolver arquiteturas de segurança cada vez mais refinadas para identificar agentes de ameaça, zonas de segurança, controles de segurança e ativos técnicos e de negócios importantes.
- **Time-of-check to time-of-use (TOCTOU) (Tempo-de-verificação-para-o-tempo-de-uso)** – Uma situação em que uma aplicação verifica o estado de um recurso antes de usar esse recurso, mas o estado do recurso pode ser alterado entre a verificação e o uso. Isso pode invalidar os resultados da verificação e causar uma situação em que a aplicação realiza ações inválidas devido a essa incompatibilidade de estado.
- **Time based One-time Passwords (TOTPs) (Senhas de Uso Único Baseadas em Tempo)** - Um método de gerar uma OTP onde o tempo atual age como parte do algoritmo para gerar a senha.
- **TLS client authentication (Autenticação de Cliente TLS)**, também chamada de **Mutual TLS (mTLS) (TLS Mútuo)** –Em uma conexão TLS padrão, um cliente pode usar o certificado fornecido pelo servidor para validar a identidade do servidor. Onde a autenticação de cliente TLS é usada, o cliente também usa sua própria chave privada e certificado para permitir que o servidor também valide a identidade do cliente.
- **Transport Layer Security (TLS)** –Protocolos criptográficos que fornecem segurança de comunicação através de uma conexão de rede.
- **Traversal Using Relays around NAT (TURN)** –Uma extensão do protocolo STUN usando um servidor TURN como um retransmissor (relay) quando conexões diretas peer-to-peer não podem ser estabelecidas. Definido na RFC 8656.
- **Trusted execution environment (TEE) (Ambiente de Execução Confiável)** - Um ambiente de processamento isolado no qual aplicações podem ser executadas com segurança, independentemente do resto do sistema.
- **Trusted Platform Module (TPM) (Módulo de Plataforma Confiável)** –Um tipo de HSM que geralmente está anexado a um componente de hardware maior, como uma placa-mãe, e atua como a “raiz de confiança”(root of trust) daquele sistema.
- **Trusted Service Layer (Camada de Serviço Confiável)** –Qualquer ponto de imposição de controle confiável, como um microserviço, API serverless, lado do servidor, uma API confiável em um dispositivo cliente que possua inicialização segura (secure boot), APIs de parceiros ou externas, e assim por diante. Confiável significa que não há preocupação de que um usuário não confiável será capaz de ignorar ou pular a camada ou os controles implementados naquela camada.
- **Uniform Resource Identifier (URI) (Identificador Uniforme de Recurso)**- Uma sequência única de caracteres que identifica um recurso, como página da web, endereço de correio, locais.
- **Uniform Resource Locator (URL) (Localizador Uniforme de Recursos)** –Uma string que especifica a localização do recurso na Internet.
- **Universally Unique Identifier (UUID) (Identificador Universalmente Único)** –Um número de referência único usado como identificador em software.
- **Verifier (Verificador)** –A pessoa ou equipe que está revisando uma aplicação em relação aos requisitos do OWASP ASVS.

- **Web Real-Time Communication (WebRTC) (Comunicação em Tempo Real pela Web)** –Uma pilha de protocolos e API da web associada usada para o transporte de fluxos multimídia em aplicações web, geralmente no contexto de teleconferências. Baseado em SRTP, SRTCP, DTLS, SDP e STUN/TURN.
- **WebSocket over TLS (WSS)** –A prática de proteger a comunicação de WebSocket embutindo (layering) o WebSocket sobre o protocolo TLS.
- **What You See Is What You Get (WYSIWYG) (O Que Você Vê É O Que Você Obtém)** –Um tipo de editor de conteúdo rico (rich content) que mostra como o conteúdo realmente parecerá ao ser renderizado, em vez de mostrar a codificação usada para governar a renderização.
- **X.509 Certificate (Certificado X.509)** –Um certificado X.509 é um certificado digital que usa o amplamente aceito padrão internacional de infraestrutura de chaves públicas (PKI) X.509 para verificar que uma chave pública pertence à identidade do usuário, computador ou serviço contida no certificado.
- **XML eXternal Entity (XXE) (Entidade Externa XML)** –Um tipo de entidade XML que pode acessar conteúdo local ou remoto por meio de um identificador de sistema declarado. Isso pode levar a vários ataques de injeção.

Apêndice B: Referências

Os seguintes projetos da OWASP provavelmente serão mais úteis aos usuários/adotantes deste padrão:

Projetos Principais da OWASP (Core Projects)

1. OWASP Top 10 Project: <https://owasp.org/www-project-top-ten/>
2. OWASP Web Security Testing Guide: <https://owasp.org/www-project-web-security-testing-guide/>
3. OWASP Proactive Controls: <https://owasp.org/www-project-proactive-controls/>
4. OWASP Software Assurance Maturity Model (SAMM): <https://owasp.org/www-project-samm/>
5. OWASP Secure Headers Project: <https://owasp.org/www-project-secure-headers/>

Projeto OWASP Cheat Sheet Series

Este projeto possui várias folhas de dicas (cheat sheets) que serão relevantes para diferentes tópicos no ASVS.

Existe um mapeamento para o ASVS que pode ser encontrado aqui: <https://cheatsheetseries.owasp.org/IndexASVS.html>

Projetos Relacionados à Segurança Mobile

1. OWASP Mobile Security Project: <https://owasp.org/www-project-mobile-security/>
2. OWASP Mobile Top 10 Risks: <https://owasp.org/www-project-mobile-top-10/>
3. OWASP Mobile Security Testing Guide e Mobile Application Security Verification Standard: <https://owasp.org/www-project-mobile-security-testing-guide/>

Projetos Relacionados à Internet das Coisas (IoT) da OWASP

1. OWASP Internet of Things Project: <https://owasp.org/www-project-internet-of-things/>

Projetos Serverless da OWASP

1. OWASP Serverless Project: <https://owasp.org/www-project-serverless-top-10/>

Outros

Da mesma forma, os seguintes sites provavelmente serão úteis aos usuários/adotantes deste padrão:

1. SecLists Github: <https://github.com/danielmiessler/SecLists>
2. MITRE Common Weakness Enumeration: <https://cwe.mitre.org/>
3. PCI Security Standards Council: <https://www.pcisecuritystandards.org/>
4. PCI Data Security Standard (DSS) v3.2.1 Requirements and Security Assessment Procedures: https://www.pcisecuritystandards.org/documents/PCI_DSS_v3-2-1.pdf
5. PCI Software Security Framework - Secure Software Requirements and Assessment Procedures: https://www.pcisecuritystandards.org/documents/PCI-Secure-Software-Standard-v1_0.pdf
6. PCI Secure Software Lifecycle (Secure SLC) Requirements and Assessment Procedures: https://www.pcisecuritystandards.org/documents/PCI-Secure-SLC-Standard-v1_0.pdf
7. OWASP ASVS 4.0 Testing Guide <https://github.com/BlazingWind/OWASP-ASVS-4.0-testing-guide>

Apêndice C: Padrões de Criptografia

O capítulo “Criptografia” vai além de simplesmente definir as melhores práticas. O seu objetivo é o de aprimorar a compreensão dos princípios de criptografia e incentivar a adoção de métodos de segurança modernos e mais resilientes. Este apêndice fornece informações técnicas detalhadas com relação a cada requisito, complementando os padrões abrangentes descritos no capítulo “Criptografia”

Este apêndice define o nível de aprovação para diferentes mecanismos criptográficos:

- Mecanismos Aprovados (Approved - A) podem ser usados em aplicações.
- Mecanismos Legados (Legacy mechanisms - L) não devem ser usados em aplicações, mas ainda podem ser usados apenas para compatibilidade com código ou aplicações legadas existentes. Embora o uso de tais mecanismos atualmente não seja considerado uma vulnerabilidade em si, eles devem ser substituídos por mecanismos mais seguros e à prova de futuro (future-proof) assim que possível.
- Mecanismos Não Permitidos (Disallowed mechanisms - D) não devem ser usados porque são atualmente considerados quebrados ou não fornecem segurança suficiente.

Esta lista pode ser anulada (overridden) no contexto de uma determinada aplicação por vários motivos, incluindo:

- novas evoluções no campo da criptografia;
- conformidade com regulamentação.

Inventário e Documentação Criptográfica

Esta seção fornece informações adicionais para V11.1 Inventário e Documentação Criptográfica.

É importante garantir que todos os ativos criptográficos, como algoritmos, chaves e certificados, sejam descobertos, inventariados e avaliados regularmente. Para o Nível 3, isso deve incluir o uso de varredura (scanning) estática e dinâmica para descobrir o uso da criptografia em uma aplicação. Ferramentas como SAST e DAST podem ajudar com isso, mas é possível que ferramentas dedicadas sejam necessárias para obter uma cobertura mais abrangente. Exemplos gratuitos de ferramentas incluem:

- CryptoMon - Network Cryptography Monitor - using eBPF, written in python
- Cryptobom Forge Tool: Generating Comprehensive CBOMs from CodeQL Outputs

Forças Equivalentes dos Parâmetros Criptográficos

As forças de segurança relativas para vários sistemas criptográficos estão nesta tabela (do NIST SP 800-57 Part 1, p.71):

Força de Segurança	Algoritmos de Chave Simétrica	Campo Finito	Fatoração de Inteiros	Curva Elíptica
<= 80	2TDEA	L = 1024 N = 160	k = 1024	f = 160-223
112	3TDEA	L = 2048 N = 224	k = 2048	f = 224-255
128	AES-128	L = 3072 N = 256	k = 3072	f = 256-383
192	AES-192	L = 7680 N = 384	k = 7680	f = 384-511

Força de Segurança	Algoritmos de Chave Simétrica	Campo Finito	Fatoração de Inteiros	Curva Elíptica
256	AES-256	$L = 15360$ $N = 512$	$k = 15360$	$f = 512+$

Exemplo de aplicações:

- Criptografia de Campo Finito: DSA, FFDH, MQV
- Criptografia de Fatoração de Inteiros: RSA
- Criptografia de Curva Elíptica: ECDSA, EdDSA, ECDH, MQV

Nota: observe que esta seção pressupõe que não exista computador quântico; se tal computador existisse, as estimativas para as últimas 3 colunas não seriam mais válidas.

Valores Aleatórios

Esta seção fornece informações adicionais para V11.5 Valores Aleatórios.

Nome	Versão/Referência	Notas	Status
/dev/random	Linux 4.8+ (Oct 2016), também encontrado no iOS, Android e outros sistemas operacionais POSIX baseados em Linux. Baseado na RFC7539	Utilizando stream ChaCha20. Encontrado no iOS SecRandomCopyBytes e Android Secure Random com as configurações corretas fornecidas a cada um.	A
/dev/urandom	Arquivo especial do kernel do Linux para fornecer dados aleatórios	Fornecer fontes de entropia de alta qualidade a partir de aleatoriedade de hardware	A
AES-CTR-DRBG	NIST SP800-90A	Conforme usado em implementações comuns, como a Windows CNG API BCryptGenRandom definida por BCRYPT_RNG_ALGORITHM.	A
HMAC-DRBG	NIST SP800-90A		A
Hash-DRBG	NIST SP800-90A		A

Nome	Versão/Referência	Notas	Status
getentropy()	OpenBSD, disponível em Linux glibc 2.25+ e macOS 10.12+	Fornecer bytes aleatórios seguros diretamente da fonte de entropia do kernel com uma API direta e mínima. É mais moderno e evita as armadilhas associadas às APIs mais antigas.	A

A função hash subjacente usada com o HMAC-DRBG ou Hash-DRBG deve ser aprovada para este uso.

Algoritmos de Cifra

Esta seção fornece informações adicionais para V11.3 Algoritmos de Criptografia.

Os algoritmos de cifra aprovados estão listados em ordem de preferência.

Algoritmos de Chave Simétrica	Referência	Status
AES-256	FIPS 197	A
Salsa20	Salsa 20 specification	A
XChaCha20	XChaCha20 Draft	A
XSalsa20	Extending the Salsa20 nonce	A
ChaCha20	RFC 8439	A
AES-192	FIPS 197	A
AES-128	FIPS 197	L
2TDEA		D
TDEA (3DES/3DEA)		D
IDEA		D
RC4		D
Blowfish		D
ARC4		D
DES		D

Modos de Cifra AES

Cifras de bloco, como o AES, podem ser usadas com diferentes modos de operação. Muitos modos de operação, como o Electronic codebook (ECB), são inseguros e não devem ser usados. Os modos de operação Galois/Counter Mode (GCM) e Counter with cipher block chaining message authentication code (CCM) fornecem criptografia autenticada e devem ser usados em aplicações modernas.

Os modos aprovados estão listados em ordem de preferência.

Modo	Autenticado	Referência	Status	Restrição
GCM	Sim	NIST SP 800-38D	A	
CCM	Sim	NIST SP 800-38C	A	
CBC	Não	NIST SP 800-38A	L	
CCM-8	Sim		D	
ECB	Não		D	
CFB	Não		D	
OFB	Não		D	
CTR	Não		D	

Notas:

- Todas as mensagens criptografadas devem ser autenticadas. Para QUALQUER uso do modo CBC, DEVE haver um algoritmo MAC de hash associado para validar a mensagem. Em geral, isso DEVE ser aplicado no método Encrypt-Then-Hash (mas o TLS 1.2 usa Hash-Then-Encrypt em seu lugar). Se isso não puder ser garantido, o CBC NÃO DEVE ser usado. A única aplicação onde a criptografia sem um algoritmo MAC é permitida é a criptografia de disco.
- Se o CBC for usado, deve-se garantir que a verificação do preenchimento (padding) seja realizada em tempo constante (constant time).
- Ao usar CCM-8, a tag MAC possui apenas 64 bits de segurança. Isso não está em conformidade com o requisito 6.2.9, que exige pelo menos 128 bits de segurança.
- Criptografia de disco é considerada fora do escopo para o ASVS. Portanto, este apêndice não lista nenhum método aprovado para criptografia de disco. Para este uso, a criptografia sem autenticação é geralmente aceita e os modos XTS, XEX e LRW são tipicamente usados.

Empacotamento de Chaves (Key Wrapping)

O empacotamento de chave criptográfica (cryptographic key wrap) (e o correspondente desempacotamento de chave / key unwrap) é um método de proteger uma chave existente encapsulando-a (ou

seja, empacotando-a) ao empregar um mecanismo de criptografia adicional, para que a chave original não seja exposta de forma óbvia, por exemplo, durante uma transferência. Esta chave adicional usada para proteger a chave original é referida como a chave de empacotamento (wrap key).

Essa operação pode ser realizada quando for desejável proteger chaves em locais considerados não confiáveis ou para enviar chaves sensíveis em redes não confiáveis ou dentro de aplicações. No entanto, deve-se considerar seriamente a compreensão da natureza (ex., a identidade e o propósito) da chave original antes de se comprometer com um procedimento de empacotar/desempacotar (wrap/unwrap), pois isso pode ter repercussões tanto para os sistemas/aplicações de origem quanto de destino em termos de segurança e, especialmente, conformidade, o que pode incluir trilhas de auditoria da função de uma chave (ex., assinatura), bem como o armazenamento apropriado da chave.

Especificamente, o AES-256 DEVE ser usado para o empacotamento de chaves, seguindo o NIST SP 800-38F e considerando provisões voltadas para o futuro contra a ameaça quântica. Os modos de cifra usando AES são os seguintes, em ordem de preferência:

Empacotamento de Chave		
(Key Wrapping)	Referência	Status
KW	NIST SP 800-38F	A
KWP	NIST SP 800-38F	A

AES-192 e AES-128 PODEM ser usados se o caso de uso exigir, mas sua motivação DEVE ser documentada no inventário de criptografia da entidade.

Criptografia Autenticada

Com exceção da criptografia de disco, os dados criptografados devem ser protegidos contra modificação não autorizada utilizando alguma forma de esquema de criptografia autenticada (AE), geralmente usando um esquema de criptografia autenticada com dados associados (AEAD).

A aplicação deve, de preferência, usar um esquema AEAD aprovado. Alternativamente, ela pode combinar um esquema de cifra aprovado e um algoritmo MAC aprovado com uma construção Encrypt-then-MAC.

MAC-then-encrypt ainda é permitido para compatibilidade com aplicações legadas. É usado no TLS v1.2 com suítes de cifras (ciphers suites) antigas.

Mecanismo AEAD	Referência	Status
AES-GCM	SP 800-38D	A
AES-CCM	SP 800-38C	A

Mecanismo AEAD	Referência	Status
ChaCha-Poly1305	RFC 7539	A
AEGIS-256	AEGIS: A Fast Authenticated Encryption Algorithm (v1.1)	A
AEGIS-128	AEGIS: A Fast Authenticated Encryption Algorithm (v1.1)	A
AEGIS-128L	AEGIS: A Fast Authenticated Encryption Algorithm (v1.1)	A
Encrypt-then-MAC		A
MAC-then-encrypt		L

Funções Hash

Esta seção fornece informações adicionais para V11.4 Hashing e Funções Baseadas em Hash.

Funções Hash para Casos de Uso Gerais

A tabela a seguir lista as funções hash aprovadas em casos gerais de uso criptográfico, como assinaturas digitais:

- Funções hash aprovadas fornecem forte resistência à colisão e são adequadas para aplicações de alta segurança.
- Alguns desses algoritmos oferecem forte resistência a ataques quando usados com o gerenciamento adequado de chaves criptográficas, e por isso são adicionalmente aprovados para funções HMAC, KDF e RBG.
- Função hash com menos de 254 bits de saída possui resistência a colisão insuficiente e não deve ser usada para assinatura digital ou outras aplicações que requeiram resistência a colisão. Para outros usos, elas podem ser usadas para compatibilidade e verificação APENAS com sistemas legados, mas não devem ser usadas em novos designs.

Função Hash	Referência	Status	Restrições
SHA3-512	FIPS 202	A	
SHA-512	FIPS 180-4	A	
SHA3-384	FIPS 202	A	
SHA-384	FIPS 180-4	A	
SHA3-256	FIPS 202	A	
SHA-512/256	FIPS 180-4	A	
SHA-256	FIPS 180-4	A	

Função Hash	Referência	Status	Restrições
SHAKE256	FIPS 202	A	
BLAKE2s	BLAKE2: simpler, smaller, fast as MD5	A	
BLAKE2b	BLAKE2: simpler, smaller, fast as MD5	A	
BLAKE3	BLAKE3 one function, fast everywhere	A	
SHA-224	FIPS 180-4	L	Não adequado para HMAC, KDF, RBG, assinaturas digitais
SHA-512/224	FIPS 180-4	L	Não adequado para HMAC, KDF, RBG, assinaturas digitais
SHA3-224	FIPS 202	L	Não adequado para HMAC, KDF, RBG, assinaturas digitais
SHA-1	RFC 3174 & RFC 6194	L	Não adequado para HMAC, KDF, RBG, assinaturas digitais
CRC (qualquer comprimento)		D	
MD4	RFC 1320	D	
MD5	RFC 1321	D	

Funções Hash para Armazenamento de Senhas

Para o hashing seguro de senhas, funções hash dedicadas devem ser usadas. Estes algoritmos de hashing lento (slow-hashing) mitigam ataques de força bruta e de dicionário, aumentando a dificuldade computacional do crack de senhas.

KDF	Referência	Parâmetros Obrigatórios	Status
argon2id	RFC 9106	$t = 1: m \geq 47104$ (46 MiB), $p = 1$	A
		$t = 2: m \geq 19456$ (19 MiB), $p = 1$	A
		$t \geq 3: m \geq 12288$ (12 MiB), $p = 1$	A
scrypt	RFC 7914	$p = 1: N \geq 2^{17}$ (128 MiB), $r = 8$	A
		$p = 2: N \geq 2^{16}$ (64 MiB), $r = 8$	A
		$p \geq 3: N \geq 2^{15}$ (32 MiB), $r = 8$	A

KDF	Referência	Parâmetros Obrigatórios	Status
bcrypt	A Future-Adaptable Password Scheme	cost ≥ 10	A
PBKDF2-HMAC-SHA-512	NIST SP 800-132, FIPS 180-4	iterações $\geq 210,000$	A
PBKDF2-HMAC-SHA-256	NIST SP 800-132, FIPS 180-4	iterações $\geq 600,000$	A
PBKDF2-HMAC-SHA-1	NIST SP 800-132, FIPS 180-4	iterações $\geq 1,300,000$	L

As funções de derivação de chaves baseadas em senhas aprovadas podem ser usadas para armazenamento de senhas.

Funções de Derivação de Chaves (KDFs)

Funções de Derivação de Chaves Gerais

KDF	Referência	Status
HKDF	RFC 5869	A
TLS 1.2 PRF	RFC 5248	L
MD5-based KDFs	RFC 1321	D
SHA-1-based KDFs	RFC 3174 & RFC 6194	D

Funções de Derivação de Chaves Baseadas em Senha

KDF	Referência	Parâmetros Obrigatórios	Status
argon2id	RFC 9106	t = 1: m ≥ 47104 (46 MiB), p = 1	A
		t = 2: m ≥ 19456 (19 MiB), p = 1	A
scrypt	RFC 7914	p = 1: N $\geq 2^{17}$ (128 MiB), r = 8	A
		p = 2: N $\geq 2^{16}$ (64 MiB), r = 8	A
		p ≥ 3 : N $\geq 2^{15}$ (32 MiB), r = 8	A

KDF	Referência	Parâmetros Obrigatórios	Status
PBKDF2-HMAC-SHA-512	NIST SP 800-132, FIPS 180-4	iterações $\geq 210,000$	A
PBKDF2-HMAC-SHA-256	NIST SP 800-132, FIPS 180-4	iterações $\geq 600,000$	A
PBKDF2-HMAC-SHA-1	NIST SP 800-132, FIPS 180-4	iterações $\geq 1,300,000$	L

Mecanismos de Troca de Chaves

Esta seção fornece informações adicionais para V11.6 Criptografia de Chave Pública.

Esquemas KEX

Uma força de segurança de 112 bits ou superior DEVE ser garantida para todos os esquemas de Troca de Chaves, e sua implementação DEVE seguir as escolhas de parâmetros na tabela a seguir.

Esquema	Parâmetros de Domínio	Forward Secrecy	Status
Diffie-Hellman de Campo Finito (FFDH)	$L \geq 3072$ & $N \geq 256$	Sim	A
Diffie-Hellman de Curva Elíptica (ECDH)	$f \geq 256-383$	Sim	A
Transporte de chave criptografada com RSA-PKCS#1 v1.5		Não	D

Onde os seguintes parâmetros são:

- k é o tamanho da chave para chaves RSA.
- L é o tamanho da chave pública e N é o tamanho da chave privada para criptografia de campo finito.
- f é o alcance (range) de tamanhos de chaves para ECC.

Nenhuma nova implementação DEVE usar qualquer esquema que NÃO esteja em conformidade com as diretrizes NIST SP 800-56A & B e NIST SP 800-77. Especificamente, o IKEv1 NÃO DEVE ser usado em produção.

Grupos Diffie-Hellman

Os seguintes grupos são aprovados para implementações de troca de chaves Diffie-Hellman. As forças de segurança estão documentadas no NIST SP 800-56A, Apêndice D, e no NIST SP 800-57 Part 1 Rev.5.

Grupo	Status
P-224, secp224r1	A
P-256, secp256r1	A
P-384, secp384r1	A
P-521, secp521r1	A
K-233, sect233k1	A
K-283, sect283k1	A
K-409, sect409k1	A
K-571, sect571k1	A
B-233, sect233r1	A
B-283, sect283r1	A
B-409, sect409r1	A
B-571, sect571r1	A
Curve448	A
Curve25519	A
MODP-2048	A
MODP-3072	A
MODP-4096	A
MODP-6144	A
MODP-8192	A
ffdhe2048	A
ffdhe3072	A
ffdhe4096	A
ffdhe6144	A
ffdhe8192	A

Códigos de Autenticação de Mensagens (Message Authentication Codes - MAC)

Os Códigos de Autenticação de Mensagem (MACs) são constructos criptográficos usados para verificar a integridade e autenticidade de uma mensagem. Um MAC recebe uma mensagem e uma chave secreta como entradas (inputs) e produz uma tag de tamanho fixo (o valor MAC). Os MACs são amplamente utilizados em protocolos de comunicação segura (ex., TLS/SSL) para garantir que as mensagens trocadas entre as partes sejam autênticas e estejam intactas.

Algoritmo MAC	Referência	Status
HMAC-SHA-256	RFC 2104 & FIPS 198-1	A
HMAC-SHA-384	RFC 2104 & FIPS 198-1	A
HMAC-SHA-512	RFC 2104 & FIPS 198-1	A
KMAC128	NIST SP 800-185	A
KMAC256	NIST SP 800-185	A
BLAKE3 (modo keyed_hash)	BLAKE3 one function, fast everywhere	A
AES-CMAC	RFC 4493 & NIST SP 800-38B	A
AES-GMAC	NIST SP 800-38D	A
Poly1305-AES	The Poly1305-AES message-authentication code	A
HMAC-SHA-1	RFC 2104 & FIPS 198-1	L
HMAC-MD5	RFC 1321	D

Assinaturas Digitais

Os esquemas de assinatura DEVEM usar parâmetros e tamanhos de chave aprovados de acordo com o NIST SP 800-57 Part 1.

Algoritmo de Assinatura	Referência	Status
EdDSA (Ed25519, Ed448)	RFC 8032	A
XEdDSA (Curve25519, Curve448)	XEdDSA	A
ECDSA (P-256, P-384, P-521)	FIPS 186-4	A
RSA-RSSA-PSS	RFC 8017	A
RSA-SSA-PKCS#1 v1.5	RFC 8017	D
DSA (qualquer tamanho de chave)	FIPS 186-4	D

Padrões de Criptografia Pós-Quântica (Post-Quantum Encryption Standards)

As implementações de PQC devem estar em conformidade com FIPS-203/204/205, uma vez que há o mínimo de código fortalecido (hardened code) ou referência de implementação até o momento. <https://www.nist.gov/news-events/news/2024/08/nist-releases-first-3-finalized-post-quantum-encryption-standards>

O método de acordo de chaves TLS híbrido pós-quântico proposto mlkem768x25519 é suportado pelos principais navegadores, como a versão 132 do Firefox e a versão 131 do Chrome. Ele pode ser usado em ambientes de teste criptográfico ou quando disponível em bibliotecas aprovadas pela indústria ou pelo governo.

Apêndice D: Recomendações

Introdução

Durante a preparação da versão 5.0 do Application Security Verification Standard (ASVS), ficou claro que havia uma série de itens existentes e novos sugeridos que não deveriam ser incluídos como requisitos na versão 5.0. Isso pode ter ocorrido porque eles não estavam no escopo do ASVS de acordo com a definição da versão 5.0 ou, alternativamente, porque embora se sentisse que eram uma boa ideia, não poderiam ser tornados obrigatórios.

Não querendo perder todos esses itens inteiramente, alguns foram capturados neste apêndice.

Mecanismos recomendados, dentro do escopo

Os itens a seguir estão no escopo do ASVS. Eles não devem ser tornados obrigatórios, mas é altamente recomendável considerá-los como parte de uma aplicação segura.

- Um medidor de força de senha (password strength meter) deve ser fornecido para ajudar os usuários a definir uma senha mais forte.
- Crie um arquivo security.txt publicamente disponível na raiz ou diretório .well-known da aplicação que defina claramente um link ou endereço de e-mail para as pessoas entrarem em contato com os proprietários sobre problemas de segurança.
- A validação de entrada no lado do cliente (client-side) deve ser aplicada, além da validação em uma camada de serviço confiável, pois isso fornece uma boa oportunidade para descobrir quando alguém burlou os controles no lado do cliente na tentativa de atacar a aplicação.
- Evite que páginas acidentalmente acessíveis e sensíveis apareçam nos mecanismos de busca usando um arquivo robots.txt, o cabeçalho de resposta X-Robots-Tag ou uma meta tag html robots.
- Ao usar GraphQL, implemente a lógica de autorização na camada de lógica de negócios (business logic layer) em vez da camada do GraphQL ou do resolver para evitar a necessidade de tratar a autorização em cada interface separada.

Referências:

- More information on security.txt including a link to the RFC

Princípios de Segurança de Software

Os itens a seguir estavam anteriormente no ASVS, mas na verdade não são requisitos. Em vez disso, são princípios a serem considerados ao implementar controles de segurança que, quando seguidos, levarão a controles mais robustos. Eles incluem:

- Os controles de segurança devem ser centralizados, simples (economia de design), sabidamente seguros e reutilizáveis. Isso deve evitar controles duplicados, ausentes ou ineficazes.
- Sempre que possível, use implementações de controle de segurança previamente escritas e bem examinadas (well-vetted), em vez de depender da implementação de controles a partir do zero.
- Idealmente, um único mecanismo de controle de acesso deve ser usado para acessar dados e recursos protegidos. Todas as requisições devem passar por esse único mecanismo para evitar caminhos alternativos do tipo copiar e colar ou inseguros.
- O controle de acesso baseado em atributos ou recursos (feature-based) é um padrão recomendado pelo qual o código verifica a autorização do usuário para um recurso ou item de dados, em vez de apenas sua função (role). As permissões ainda devem ser alocadas usando as roles.

Processos de Segurança de Software

Existem vários processos de segurança que foram removidos do ASVS 5.0, mas ainda são uma boa ideia. O projeto OWASP SAMM pode ser uma boa fonte sobre como implementar efetivamente esses processos. Os itens que estavam anteriormente no ASVS incluem:

- Verifique o uso de um ciclo de vida de desenvolvimento de software seguro que aborde a segurança em todas as etapas do desenvolvimento.
- Verifique o uso de modelagem de ameaças (threat modeling) para cada mudança de design ou planejamento de sprint para identificar ameaças, planejar contramedidas, facilitar respostas apropriadas a riscos e guiar testes de segurança.
- Verifique se todas as histórias de usuários (user stories) e funcionalidades (features) contêm restrições funcionais de segurança, como “Como usuário, eu devo ser capaz de visualizar e editar meu perfil. Eu não devo ser capaz de visualizar ou editar o perfil de mais ninguém.”
- Verifique a disponibilidade de uma lista de verificação de codificação segura, requisitos de segurança, diretrizes ou políticas para todos os desenvolvedores e testadores.
- Verifique se existe um processo contínuo para garantir que o código-fonte da aplicação esteja livre de backdoors, código malicioso (por exemplo, ataques salami, bombas lógicas, bombas-relógio) e recursos não documentados ou ocultos (por exemplo, Easter eggs, ferramentas de depuração inseguras). Estar em conformidade com esta seção não é possível sem acesso total ao

código-fonte, incluindo bibliotecas de terceiros, e portanto, provavelmente é adequado apenas para aplicações que exigem os mais altos níveis de segurança.

- Verifique se existem mecanismos para detectar e responder ao desvio de configuração (configuration drift) em ambientes implantados. Isso pode incluir o uso de infraestrutura imutável (immutable infrastructure), reimplantação automatizada a partir de uma base segura (secure baseline) ou ferramentas de detecção de desvio que comparam o estado atual em relação a configurações aprovadas.
- Verifique se o fortalecimento de configurações (configuration hardening) é executado em todos os produtos de terceiros, bibliotecas, frameworks e serviços conforme suas recomendações individuais.

Referências:

- OWASP Threat Modeling Cheat Sheet
- OWASP Threat modeling
- OWASP Software Assurance Maturity Model Project
- Microsoft SDL

Apêndice E - Contribuidores

Agradecemos imensamente as contribuições das seguintes pessoas que comentaram ou abriram pull requests desde o lançamento do ASVS 4.0.0.

Se você estiver ciente de quaisquer erros ou gostaria que seu nome aparecesse de forma diferente, por favor nos avise.

Johan Sydseter (sydseter)	luis servin (lfservin)	Oleksii Dovydkov (oleksiidov)	IZUKA Masahiro (maizuka)
James Sulinski (jsulinski)	Eli Saad (ThunderSon)	kkshitish9	Andrew van der Stock (vanderaj)
Rick M (kingthorin)	Bankde Eakasit (Bankde)	Michael Gargiullo (mgargiullo)	Raphael Dunant (Racater)
Cesar Kohl (cesarkohl)	inaz0	Joerg Bruenner (JoergBruenner)	David Deatherage (securitydave)
John Carroll (yosignals)	Jim Fenton (jimfenton)	Matteo Pace (M4tteoP)	Sebastien gioria (SPoint42)
Steven van der Baan (vdbaan)	Jeremy Bonghwan Choi (jeremychoi)	craig-shony	Riccardo Sirigu (ricsirigu)
Tomasz Wrobel (tw2as)	Alena Dubeshko (belalena)	Rafael Green (RafaelGreen1)	mjang-cobalt

clallier94	Kevin W. Wall (kwwall)	Jordan Sherman (jsherm-fwdsec / deleterepo)	Ingo Rauner (ingo-rauner)
Dirk Wetter (drwetter)	Moshe Zioni (moshe-apiiro)	Patrick Dwyer (coderpatros)	David Clarke (davidclarke-au)
Takaharu Ogasa (takaharuogasa)	Arkadii Yakovets (arkid15r)	Motoyasu Saburi (motoyasu-saburi)	leirn
wet-certitude	timhemel	RL Thornton (thornshadow99)	Thomas Bandt (aspnetde)
Roel Storms (roelstorms)	Jeroen Willemsen (commjoen)	anonymous-31	Kamran Saifullah (deFr0ggy)
Steve Springett (stevespringett)	Spyros (northdpole)	Hans Herrera (hansphp)	Marx314
CarlosAllendes	Yonah Russ (yruss972)	Sander Maijers (sanmai-NL)	Luboš Bretschneider (bretik)
Eva Sarafianou (esarafianou)	ataseren	Steve Thomas (Sc00bz)	Dominique RIGHETTO (righettod)
Steven van der Baan (svdb-ncc)	Michael Vacarella (Aif4thah)	Tonimir Kisasondi (tkisason)	Stefan Streichsbier (streichsbaer)
hi-unc1e	sb3k (starbuck3000)	mario-platt	Devdatta Akhawe (devd)
Michael Gissing (scolytus)	Jet Anderson (thatsjet)	Dave Wichers (davewichers)	Jonny Schnittger (JonnySchnittger)
Silvia Väli (silviavali)	jackgates73	1songb1rd	Timur - (timurozkul)
Gareth Heyes (hackvector)	appills	suvikaartinen	chaals (chaals)
DanielPharos (AtlasHackert)	will Farrell (willfarrell)	Alina Vasiljeva (avasiljeva)	Paul McCann (ismisepaul)
Sage (SajjadPourali)	rbsec	Benedikt Bauer (mastacheata)	James Jardine (jamesjardine)
Mark Burnett (m8burnett)	dschwarz91	Cyber-AppSec (Cyber-AppSec)	Tib3rius
BitnessWise (bitnesswise)	damienbod (damienbod)	Jared Meit (jmeit-fwdsec)	Stefan Seelmann (sseelmann)

Brendan O'Connor (ussjoin)	Andrei Titov (andrettv)	Hans-Petter Fjeld (atluxity)	markehack
Neil Madden (NeilMadden)	Michael Geramb (mgeramb)	Osama Elnaggar (ossie-git)	mackowski
Ravi Balla (raviballa)	Hazana (hazanasec)	David Means (dmeans82)	Alexander Stein (tohch4)
BaeSenseii (baesenseii)	Vincent De Schutter (VincentDS)	S Bani (sbani)	Mitsuaki Akiyama (mak1yama)
Christopher Loessl (hashier)	victorxm	Michal Rada (michalradacz)	Veeresh Devireddy (drveresh)
MaknaSEO	darkzero2022	Liam (LiamDobbelaere)	Frank Denis (jedisct1)
Otto Sulin (ottosulin)	carllaw6885	Anders Johan Holmefjord (aholmis)	Richard Fritsch (rfricz)
mesutgungor	Scott Helme (ScottHelme)	Carlo Reggiani (carloreggiani)	Suyash Srivastava (suyash5053)
Mark Potter (markonweb)	Arjan Lamers (alamers)	Gøran Breivik (gobrtg)	flo-blg
Guillaume Déflache (guillaume-d)	Toufik Airane (toufik-airane)	Keith Hoodlet (securingdev)	Sinner (SoftwareSinner)
iloving	Jeroen Beckers (TheDauntless)	Joubin Jabbari (joubin)	yu fujioka (fujiokayu)
execjosh (execjosh)	Alicja Kario (tomato42)	Sidney Ribeiro (srjsoftware)	Gabriel Marquet (Gby56)
Drew Schulz (drschulz)	bedirhan	muralito	Ronnie Flathers (ropnop)
Philippe De Ryck (philippederyck)	Malte (mal33)	MazeOfThoughts	Andreas Falk (andifalk)
Javi (javixeneize)	Daniel Hahn (averell23)	borislav-c	Robin Wood (digininja)
miro2ns	Jan Dockx (jandockx)	vipinsaini434	priyanshukumar397
Nat Sakimura (sakimura)	Benjamin Häublein (BenjaminHae)	unknown-user-from	Ali Ramazan TAŞDELEN (alitasdln)
Pedro Escalera (oEscal)	Josh (josh-hemphill)	Tim Würtele (SECTim)	AviD (avidouglen)

SheHacksPurple (shehackspurple)	fcerullo-cycubix	Hector Eryx Paredes Camacho (heryxpc)	Irene Michlin (irene221b)
Jonah Y-M (TG-Techie)	Dhiraj Bahroos (bahroos)	Jef Meijvis (jefmeijvis)	IzmaDoesItbeta
Abdessamad TEMMAR (TmmmmmR)	sectroyer	Soh Satoh (sohsatoh)	regoravalaz
james-t (james-bitherder)	Aram Hovsepyan (aramhovsepyan)	JaimeGomezGarciaSan	ValdiGit01
iwatachan (ishowta)	Vinod Anandan (VinodAnandan)	Kevin Kien (KevinKien)	paul-williamson- swoop
endergzr	Radhwan Alshamamri (Rado0z)	Grant Ongers (rewtd)	Cure53 (cure53)
AliR2Linux	Ads Dawson (Gang- GreenTemperTatum)	William Reyor (BillReyor)	gabe (gcrow)
mascotter	luissaiz	Suren Manukyan (vx-sec)	Piotr Gliźniewicz (pglizniewicz)
Tadeusz Wachowski (tadeuszwachowski)	Nasir aka Nate (andesec)	settantasette	Lars Haulin (LarsH)
Terence Eden (edent)	JasmineScholz	Arun Sivadasan (teavanist)	Yusuf GÜR (yusuffgur)
Troy Marshall (troymarshall)	Tanner Prynn (tprynn)	Nick K. (nickific)	raoul361
Azeem Ilyas (TheAxZim)	Evo Stamatov (avioli)	Tim Potter (timpotter87)	Gavin Ray (GavinRay97)
monis (demideus)	Marcin Hoppe (MarcinHoppe)	Grambulf (ramshazar)	Jordan Pike (computersarebad)
Jason Rogers (jason-invision)	Ben Hall (benbhall)	JamesPoppyCock (jamesly123)	WhiteHackLabs (whitehacklabs)
Alex Gaynor (alex)	Filip van Laenen (filipvanlaenen)	jeurgen	GraoMelo
Andreas Kurtz (ay-kay)	Tom Tervoort (TomTervoort)	old man (deveras)	Marco Schnüriger (marcortw)
stiiin	infoseclearn (teaminfoseclearn)	hljupkij	Noe (nmarher)

Lyz (lyz-code)	Martin Riedel (mrtnrdl)	KIM Jaesuck (tcaesvk)	Barbara Schachner (bschach)
René Reuter (AresSec)	carhackpils	Tyler (tyler2cr)	Hugo (hasousa)
Wouter Bloeyaert (Someniak)	Mark de Rijk (markderijkinforec)	Ramin (picohub)	Philip D. Turner (philipdturner)
Will Chatham (willc)			
